DÉMONSTRATEUR WI-FI 6

ObCP + X-NUCLEO-67W61M1

Documentation du programme : organisation, déroulement et référence des fonctions

› Carte du programme : couches, fichiers, qui utilise quoi

› Déroulement : démarrage, boucle principale, trajet d'une commande, d'une requête web, d'un message MQTT

› Dialogue avec le module Wi-Fi, mémoire, brochage

› Référence des 470 fonctions, paramètres, codes d'erreur, commandes

› Recettes pour modifier le programme, tests sur PC

Programme version 1.2.0 (projet STM32CubeMX) — module ST67W611M1, firmware mission T01 v2.0.x

Complément du guide d'utilisation (Guide_Demonstrateur_WiFi6_ObCP.docx)

28 octobre 2026

Sommaire

1. Présentation 4
1.1 À qui s'adresse ce document 4
1.2 Comment lire ce document 4
1.3 Le programme en chiffres 4
1.4 Conventions du code 5

2. Carte du programme 6
2.1 Les couches du logiciel 7
2.2 Les fichiers du projet 7
2.3 Qui utilise quoi 8
2.4 Les variables globales 9

3. Déroulement du programme 11
3.1 Le démarrage 11
3.2 La boucle principale et les tâches de fond 12
3.3 Les interruptions 13
3.4 Parcours d'une commande tapée au terminal 14
3.5 Parcours d'une requête web 15
3.6 Parcours d'un message MQTT 16
3.7 Démarrage automatique des services (version 1.2) 17

4. Dialogue avec le module Wi-Fi 19
4.1 Un coprocesseur piloté par commandes AT 19
4.2 Les trames SPI 19
4.3 Le moteur AT : réponses et événements 20
4.4 Les machines à états 22
4.5 Commandes AT utilisées 23

5. Mémoire 26
5.1 Plan de la mémoire 26
5.2 Occupation par partie du programme 27
5.3 Les plus grosses variables 28
5.4 La configuration enregistrée en flash 28

6. Matériel et brochage 30
6.1 Broches du STM32L476RG 30
6.2 Périphériques et fichiers qui les pilotent 30

7. Référence des fonctions 32
7.1 app.h — démarrage, boucle principale, état général 32
7.2 console.h — console sur le port COM virtuel 33
7.3 demos.h — commandes et tâches de fond des quatre démos 35
7.4 config.h — configuration courante et sauvegarde en flash 41
7.5 cfg_codec.h — table des paramètres, valeurs d'usine, stockage 41
7.6 app_config.h — réglages de compilation et valeurs d'usine 43
7.7 io.h — LED RVB, sortie PWM, boutons, capteurs 43
7.8 color.h — couleurs de la LED RVB 44
7.9 lis3dh.h — accéléromètre 45
7.10 http_parse.h — analyse des requêtes HTTP 46
7.11 web_page.h — page web embarquée 47
7.12 qr_term.h — QR code dans le terminal 47
7.13 module_ota.h — mise à jour du firmware du module 48
7.14 ymodem.h — réception de fichier YMODEM 48
7.15 bridge_ctl.h — fin de la passerelle « module boot » 49
7.16 timeutil.h — dates, fuseau horaire, heure d'été 50
7.17 util.h — fonctions utilitaires 51
7.18 board.h — carte ObCP 53
7.19 cdc_port.h — port COM virtuel USB 55
7.20 uart_bridge.h — passerelle USB ↔ UART du module 56
7.21 ncp_port.h — adaptation du pilote à la carte 57
7.22 ncp.h — démarrage du module, événements 58
7.23 ncp_spi.h — trames SPI 59
7.24 ncp_at.h — moteur de commandes AT 60

7.25 ncp_wifi.h — Wi-Fi : scan, station, point d'accès 63
7.26 ncp_net.h — réseau IP : DNS, ping, SNTP, TCP 65
7.27 ncp_mqtt.h — client MQTT du module 67
7.28 ncp_fwu.h — mise à jour du firmware par SPI 68
7.29 Fichiers générés par CubeMX et gestionnaires de fautes 69
8. Paramètres de configuration 71
9. Codes d'erreur 73
10. Commandes du terminal et fonctions C 74
11. Modifier le programme 76
11.1 Règles à respecter 76
11.2 Ajouter une commande 76
11.3 Ajouter un paramètre de configuration 77
11.4 Ajouter une adresse au serveur web 77
11.5 Publier une nouvelle mesure 78
11.6 Ajouter un périphérique avec STM32CubeMX 78
12. Tests sur PC et outils 79
12.1 Le principe : un module simulé 79
12.2 Les programmes de test 79
12.3 Lancer les tests 79
12.4 Les outils du dossier Tools 79
13. Glossaire 81
Annexe A – Index des fonctions publiques 83

1. Présentation

Ce document décrit le **programme** du démonstrateur Wi-Fi 6 pour l'ObCP, version 1.2.0 : comment il est organisé, comment il se déroule et ce que fait chacune de ses fonctions. Il s'adresse à ceux qui veulent **lire, comprendre ou modifier le code**. L'installation, la programmation de la carte et l'utilisation des quatre démonstrations sont décrites dans le guide d'utilisation (`Doc/Guide_Demonstrateur_WiFi6_ObCP.docx`).

1.1 À qui s'adresse ce document

Il suppose des bases de langage C (fonctions, pointeurs, structures, tableaux) et une première prise en main de STM32CubeIDE. Les notions propres à l'embarqué et aux réseaux (interruption, tampon circulaire, commande AT, MQTT…) sont expliquées au fil du texte et rappelées dans le glossaire (chapitre 13).

1.2 Comment lire ce document

1. Commencer par la **carte du programme** (chapitre 2) : les couches du logiciel et le rôle de chaque fichier.
2. Suivre le **déroulement** (chapitre 3) : démarrage, boucle principale, puis le trajet d'une commande tapée, d'une requête web et d'un message MQTT à travers le code.
3. Le chapitre 4 explique le **dialogue avec le module Wi-Fi** (trames SPI, commandes AT, machines à états) ; les chapitres 5 et 6 décrivent la **mémoire** et le **brochage**.
4. Pour un détail précis : la **référence des fonctions** (chapitre 7), les tables des paramètres, des codes d'erreur et des commandes (chapitres 8 à 10) et l'**index des fonctions** (annexe A).
5. Avant de modifier le code : les **règles et recettes** du chapitre 11, puis les **tests sur PC** du chapitre 12.

> **Bon à savoir :** le code source est lui-même commenté pour un lecteur débutant : chaque fichier commence par un en-tête qui explique son rôle, chaque fonction publique est décrite dans le fichier `.h` qui la déclare (commentaires au format Doxygen : `@brief`, `@param`, `@retval`) et chaque fonction interne par un commentaire placé juste avant elle.
>
> La référence du chapitre 7, les tables des chapitres 8 à 10, les figures de dépendances et d'occupation mémoire sont **extraites automatiquement** du code et du programme compilé : elles correspondent exactement à la version 1.2.0.

Pour lire les sources dans STM32CubeIDE, l'ordre conseillé (rappelé en tête de `main.c`) est : `main.c` (ordre du démarrage), `app.c` (`app_init()`, `app_loop()` : le cœur de l'application), `cli.c` (les commandes du terminal), le fichier de la démo qui vous intéresse (`demo_scan.c`…), puis le pilote du module (`ncp.h` présente ses couches). Dans CubeIDE, **Ctrl + clic** sur un nom de fonction ouvre sa définition ; **Ctrl + Alt + H** affiche ses appelants.

1.3 Le programme en chiffres

Élément	Valeur
Fichiers écrits pour la démo	32 fichiers .c et 28 en-têtes .h : 15 750 lignes, dont environ 3 300 lignes de commentaires
Fonctions	295 fonctions publiques (déclarées dans un .h) et 175 fonctions internes (static)
Commandes du terminal	29 (chapitre 10)
Paramètres de configuration	29, enregistrés en flash (chapitre 8)
Codes d'erreur	24, plus 0 = succès (chapitre 9)
Mémoire utilisée	flash : 136,5 Ko sur 1 024 Ko ; RAM : 44,0 Ko sur 96 Ko, plus la pile et le tas (chapitre 5)

Élément	Valeur
Tests sur PC	5 programmes de test, 1 004 vérifications, plus des contrôles du serveur web, du script de mise à jour et du projet CubeMX (chapitre 12)
Système d'exploitation	aucun (« bare metal ») : une boucle principale et des tâches de fond non bloquantes

1.4 Conventions du code

Quelques règles de nommage permettent de savoir, à la seule lecture d'un nom, où chercher sa définition :

Forme	Signification	Exemples
`module_action()`	fonction publique : le préfixe est le nom du module (fichier) qui la définit	`console_printf()`, `wifi_scan()`, `cfg_set()`
`cmd_xxx()`	fonction exécutée par une commande du terminal (`argc`, `argv` comme `main()`)	`cmd_scan()`, `cmd_web()`
`xxx_poll()`	tâche de fond : appelée à chaque tour de boucle, rend la main aussitôt	`netapp_poll()`, `web_poll()`
`xxx_urc()`	traitement d'un événement non sollicité du module (URC)	`wifi_urc()`, `net_urc()`
`g_xxx`	variable globale (déclarée `extern` dans un .h, partagée)	`g_cfg`, `g_wifi`, `g_app`
`s_xxx`	variable `static` d'un fichier : visible seulement dans ce fichier, conservée entre deux appels	`s_cmds[]`, `s_conn[]`
`MAJUSCULES`	constante (`#define`) ou valeur d'une énumération	`NET_LINKS`, `AT_OK`, `CFG_VERSION`
`xxx_t`	type défini par `typedef` (structure, énumération)	`cfg_t`, `wifi_ctx_t`

- **Codes de retour** : les fonctions qui peuvent échouer rendent un `int` : 0 (`AT_OK`) en cas de succès, une valeur négative sinon (chapitre 9). `app_report()` affiche le message correspondant.
- **Textes** : une fonction qui écrit du texte reçoit le tampon de destination et sa taille (`out`, `len`) ; elle ne déborde jamais et termine toujours le texte par `'\0'`.
- **Commentaires** : `/* Voir xxx.h */` devant une fonction publique signale que sa description complète est dans l'en-tête qui la déclare.
- **Fichiers générés par CubeMX** (`main.c`, `gpio.c`, `spi.c`, `stm32l4xx_it.c`, `USB_DEVICE/…`) : on n'y écrit **que** entre `/* USER CODE BEGIN xxx */` et `/* USER CODE END xxx */`, seules parties conservées à la régénération.
- **Style** : accolades sur leur propre ligne, une instruction par ligne, conditions entièrement parenthésées, entiers non signés écrits `10U` (règles inspirées de MISRA C) ; le code compile sans aucun avertissement avec `-Wall -Wextra`.

2. Carte du programme

Le programme est organisé en **couches** : chacune s'appuie sur celles du dessous et ne connaît pas celles du dessus. La figure ci-dessous en donne la carte complète : chaque boîte est un fichier source, avec son rôle en quelques mots.

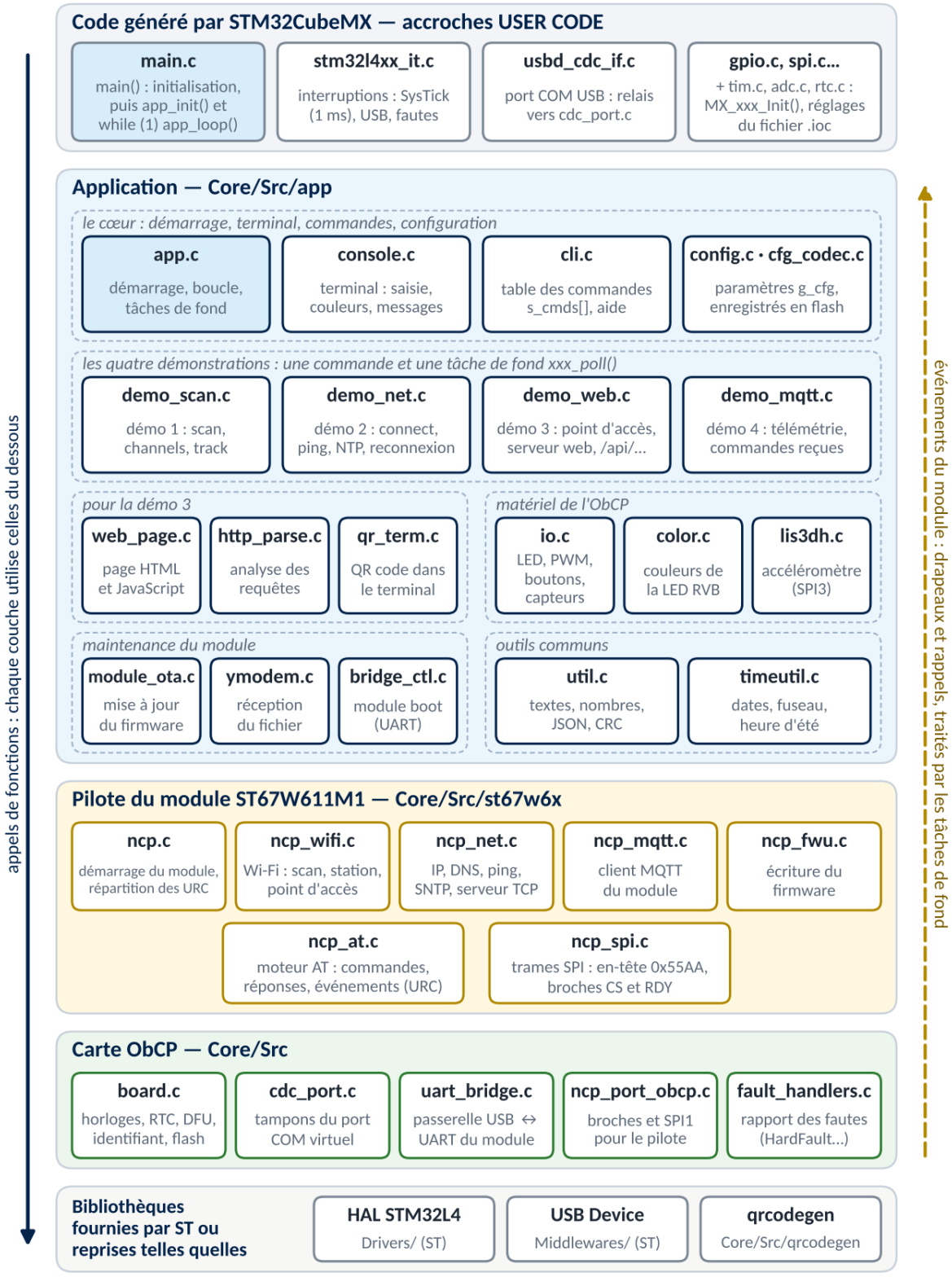


Figure 1 – Carte du programme : les couches et les fichiers

2.1 Les couches du logiciel

Couche	Rôle	Dossier
Code généré par CubeMX	Initialise le matériel (horloges, broches, SPI, timers, ADC, RTC, USB) selon le fichier `ObCP_WiFi_Demo.ioc`, puis appelle l'application. Régénéré par CubeMX : seules les sections USER CODE sont à nous.	`Core/Src`, `USB_DEVICE`
Application	Tout ce que l'on voit : console, commandes, configuration, les quatre démos, page web, télémétrie. Ne manipule jamais directement le SPI du module : passe par le pilote.	`Core/Src/app`
Pilote du module	« ST67W6X_Lite » : traduit chaque service (scan, connexion, serveur TCP, MQTT…) en commandes AT envoyées au module par trames SPI. Indépendant de la carte.	`Core/Src/st67w6x`
Carte ObCP	Ce qui est propre à la carte : horloges et RTC, identifiant, port COM virtuel, passerelle UART, broches du module (`ncp_port_obcp.c`), rapport de fautes.	`Core/Src`
Bibliothèques	HAL STM32L4 et bibliothèque USB de ST, générateur de QR codes de Project Nayuki : utilisées telles quelles.	`Drivers`, `Middlewares`, `Core/Src/qrcodegen`

Les appels vont **du haut vers le bas** : l'application appelle le pilote, qui appelle `ncp_port_obcp.c`, qui appelle la HAL. Dans l'autre sens, le module ne peut pas « appeler » l'application : quand il signale un événement (réseau trouvé, connexion perdue, message MQTT reçu…), le pilote met à jour un **drapeau** dans une variable d'état (`g_wifi`, `g_net`, `g_mqtt`) ou appelle une fonction de **rappel** fournie par l'application. Les tâches de fond de l'application lisent ces drapeaux au tour de boucle suivant (chapitre 3).

> **Astuce :** cette séparation rend le pilote et une grande partie de l'application **indépendants du matériel** : ils sont compilés et testés sur PC face à un module simulé (chapitre 12). Pour porter le pilote sur une autre carte STM32, il suffit de réécrire les sept fonctions de `ncp_port.h`.

2.2 Les fichiers du projet

Le tableau suivant donne, pour chaque dossier, les fichiers écrits pour la démo et leur taille (lignes de texte, commentaires compris). Les fichiers produits par CubeMX ou fournis par ST ne sont pas comptés.

Fichiers	Contenu	Lignes
app.c, console.c, cli.c	démarrage, boucle principale, terminal, commandes	2 144
config.c, cfg_codec.c	paramètres et enregistrement en flash	615
demo_scan.c, demo_net.c, demo_web.c, demo_mqtt.c	les quatre démonstrations	3 013
web_page.c, http_parse.c, qr_term.c	page web, analyse HTTP, QR code	494
io.c, color.c, lis3dh.c	LED RVB, PWM, boutons, capteurs	697
module_ota.c, ymodem.c, bridge_ctl.c	mise à jour et programmation du module	964
util.c, timeutil.c	outils communs (textes, JSON, dates)	785
ncp.c, ncp_spi.c, ncp_at.c	pilote : démarrage, trames SPI, moteur AT	1 305
ncp_wifi.c, ncp_net.c, ncp_mqtt.c, ncp_fwu.c	pilote : Wi-Fi, IP, MQTT, mise à jour	1 550
board.c, cdc_port.c, uart_bridge.c, ncp_port_obcp.c, fault_handlers.c	carte ObCP	1 301

Fichiers	Contenu	Lignes
28 en-têtes (Core/Inc)	déclarations des fonctions publiques, avec leur description	2 882

Autres dossiers du paquet :

Dossier	Contenu
`Core/Src/qrcodegen`	bibliothèque QR-Code-generator (Project Nayuki, licence MIT), non modifiée
`Drivers`, `Middlewares`	HAL STM32L4 et bibliothèque USB Device de ST, copiées par CubeMX
`USB_DEVICE`	configuration USB générée par CubeMX ; `usbd_cdc_if.c` et `usbd_conf.c` renvoient vers `cdc_port.c` (sections USER CODE)
`Tests`	tests sur PC : module simulé, fausse HAL, programmes de test (chapitre 12)
`Tools`	scripts Python : vérification après régénération CubeMX, mise à jour du module, enregistrement du RSSI
`Binaries`	programme compilé prêt à charger (.hex, .bin, .elf)
`Doc`	guide d'utilisation et ce document

2.3 Qui utilise quoi

La figure suivante est tirée directement du programme compilé : pour chaque fichier (une ligne), elle compte les fonctions et variables globales qu'il utilise dans chacun des autres (une colonne). Elle montre par exemple que `cli.c` utilise presque tous les modules de l'application (il appelle les commandes), que seul `ncp_at.c` utilise `ncp_spi.c`, ou qu'aucun fichier du pilote n'utilise l'application.

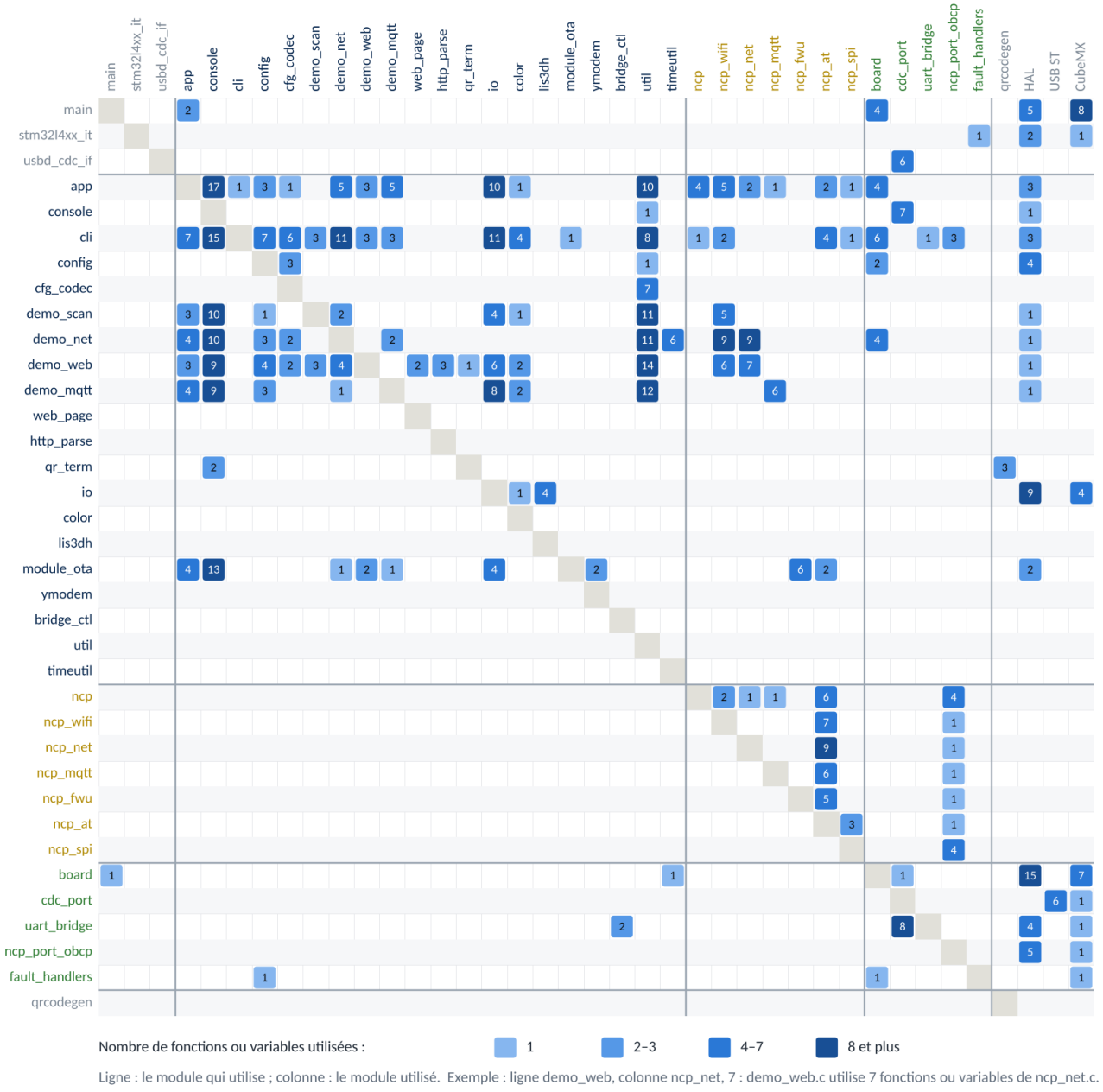


Figure 2 – Qui utilise quoi : nombre de fonctions ou variables utilisées d'un module à l'autre

La même information est donnée module par module dans la référence (chapitre 7, rubriques « Utilise » et « Utilisé par »).

2.4 Les variables globales

Le programme n'a que cinq variables globales ; toutes les autres sont `static`, donc privées à leur fichier. Ces cinq structures sont le « tableau de bord » partagé entre les couches :

Variable	Type, fichier	Contenu	Écrite par
`g_cfg`	`cfg_t`, `config.c`	configuration courante : réseau, IP, heure, point d'accès, MQTT… (copie en RAM de la page de flash)	commandes `set`, `config_load()`, page web
`g_app`	`app_t`, `app.c`	état général : module démarré (`ncp_ok`), versions du firmware, adresses MAC, configuration modifiée non enregistrée (`cfg_dirty`)	`app.c`, `cli.c`

Variable	Type, fichier	Contenu	Écrite par
`g_wifi`	`wifi_ctx_t`, `ncp_wifi.c`	état de la station (`sta`), résultats du scan, clients du point d'accès, drapeaux d'événements `evt_xxx`	`wifi_urc()` (événements +CW:)
`g_net`	`net_ctx_t`, `ncp_net.c`	état des 5 connexions TCP du module : ouverte, octets en attente	`net_urc()` (+CIP:, +IPD:)
`g_mqtt`	`mqtt_ctx_t`, `ncp_mqtt.c`	client MQTT connecté, drapeaux de connexion et déconnexion	`mqtt_urc()` (+MQTT:)

> **Bon à savoir :** les champs `evt_xxx` sont des **drapeaux** : le pilote les met à vrai quand l'événement arrive, le code qui le traite les remet à faux. Ils sont déclarés `volatile` car ils changent pendant les attentes du moteur AT : le compilateur doit les relire à chaque fois au lieu de garder une copie dans un registre.

3. Déroulement du programme

3.1 Le démarrage

À la mise sous tension, le processeur exécute `Reset_Handler` (fichier de démarrage `startup_stm32l476rgtx.s`) : il copie les valeurs initiales des variables de la flash vers la RAM, met les autres à zéro, puis appelle `main()`. La figure suivante détaille la suite des opérations.

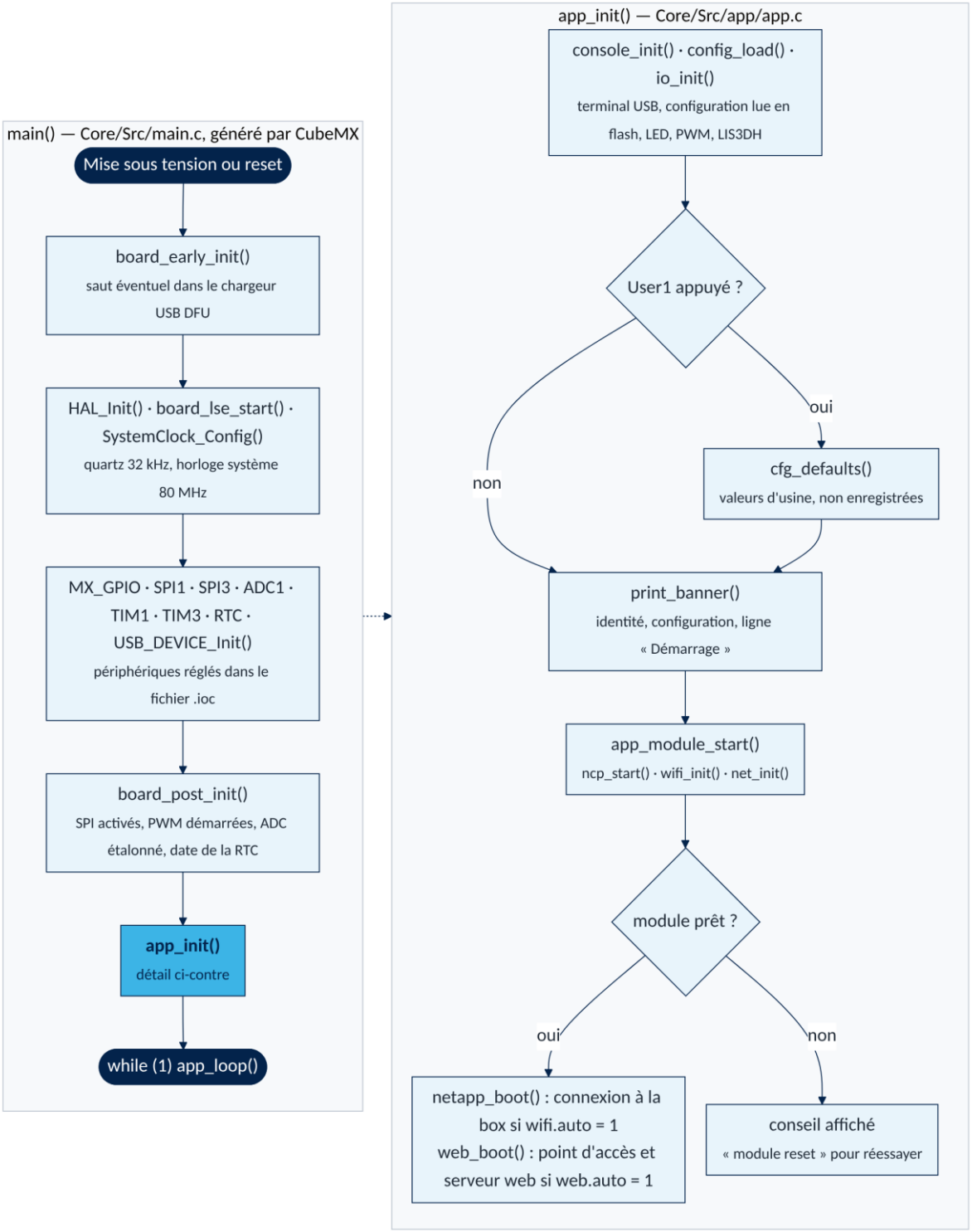


Figure 3 – Démarrage : main() puis app_init()

Dans `main()`, le code généré par CubeMX règle les horloges (quartz 8 MHz et PLL : 80 MHz) puis chaque périphérique (`MX_xxx_Init()`). La démo intervient à trois endroits, dans les sections USER CODE : `board_early_init()` tout au début (saut éventuel dans le chargeur USB DFU demandé par la commande `dfu`), `board_lse_start()` avant le réglage des horloges (démarrage du quartz 32 kHz de la RTC), et enfin `board_post_init()` puis `app_init()`.

`app_init()` initialise la console, lit la configuration en flash (`config_load()`), prépare la LED, la sortie PWM et l'accéléromètre, puis affiche la bannière. Comme le terminal n'est pas encore ouvert, ces messages sont gardés dans le tampon d'émission USB (4 Ko) et apparaissent dès son ouverture. Elle démarre ensuite le module Wi-Fi (`app_module_start()` : impulsion sur `CHIP_EN`, attente du message « ready », réglage du pays et du réseau) et, si le module répond, lance les **services automatiques** : connexion à la box si `wifi.auto = 1` (`netapp_boot()`) et lancement programmé du serveur web si `web.auto = 1` (`web_boot()`).

> **Astuce :** bouton **User1 maintenu** pendant le démarrage : `app_init()` part des valeurs d'usine (sans effacer la configuration enregistrée). Utile si une configuration empêche le fonctionnement normal.

3.2 La boucle principale et les tâches de fond

Le programme n'utilise pas de système d'exploitation temps réel : après `app_init()`, `main()` appelle `app_loop()` sans fin (« super-boucle »). Un tour de boucle dure de quelques microsecondes à quelques millisecondes ; il ne doit **jamais rester bloqué** longtemps, sinon les autres services s'arrêtent (page web qui ne répond plus, connexion MQTT perdue…).

`app_idle()` regroupe les **tâches de fond** : chaque démo y fait avancer sa propre « machine » d'un petit pas, puis rend la main. Les commandes longues (`scan`, `track`, `ping`, `module ota`…) appellent elles aussi `app_idle()` pendant leurs attentes, pour que le serveur web et MQTT continuent de fonctionner. Une protection (`busy`) empêche un appel imbriqué.

Tâche de fond	Fichier	Ce qu'elle fait à chaque tour
`ncp_poll()`	`ncp.c`	lit les trames que le module a préparées (broche RDY à 1), traite les réponses et les événements
`netapp_poll()`	`demo_net.c`	connexion automatique à la box, reconnexion après une perte, synchronisation NTP de l'horloge
`web_poll()`	`demo_web.c`	lancement automatique du serveur, lecture des requêtes HTTP arrivées, réponses, fermeture des connexions inactives
`mqttapp_poll()`	`demo_mqtt.c`	connexion au broker, télémétrie périodique, commandes reçues (file d'attente)
`io_buttons_poll()`	`io.c`	appuis sur User1 (bascule la LED) et SW1 (événement MQTT)

Pendant une tâche de fond, un message à afficher passe par `console_event()` : il efface la ligne en cours de saisie, écrit le message puis réaffiche l'invite et ce qui était tapé, pour ne pas mélanger les messages et la saisie.

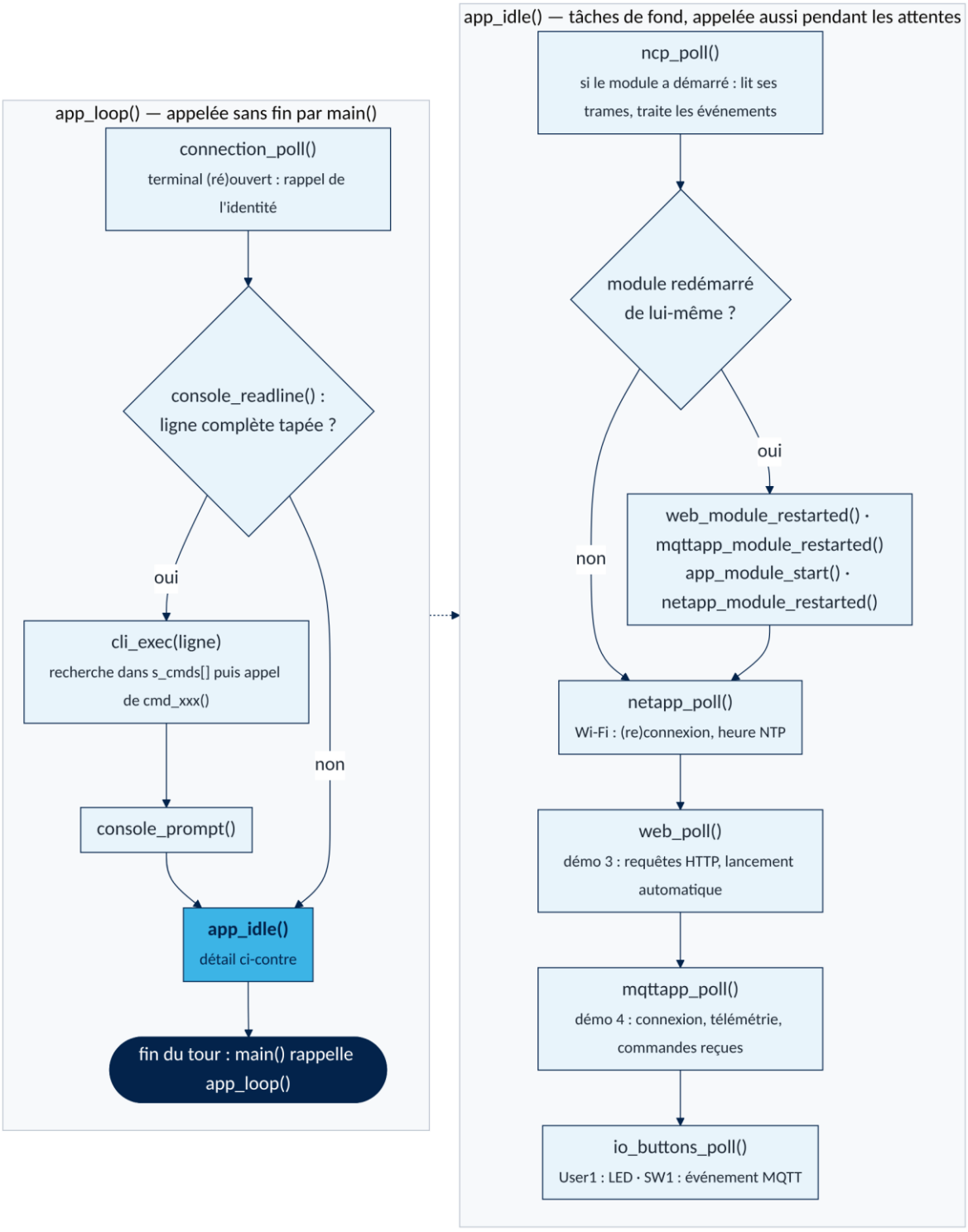


Figure 4 – Un tour de la boucle principale

3.3 Les interruptions

Une **interruption** suspend le programme principal pour exécuter une courte fonction (le « gestionnaire »), puis le reprend là où il était. Le programme en utilise très peu : presque tout est lu par **scrutation** (« polling ») dans la boucle principale, ce qui est plus simple à suivre et à mettre au point.

Interruption	Gestionnaire	Rôle
SysTick (toutes les ms)	`SysTick_Handler()` → `HAL_IncTick()`	compteur de millisecondes lu par `HAL_GetTick()` : délais, échéances des tâches de fond
USB OTG FS	`OTG_FS_IRQHandler()` → HAL → `usbd_cdc_if.c` → `cdc_port_itf_xxx()`	octets reçus du terminal rangés dans le tampon circulaire de réception, envoi du bloc suivant du tampon d'émission, ouverture du port (DTR)
Fautes (HardFault…)	`fault_handlers.c`	rapport rangé dans les registres de sauvegarde de la RTC, LED rouge, redémarrage ; affiché au démarrage suivant

Le signal RDY du module et les boutons sont donc lus par la boucle principale ; pendant `module boot`, le DMA1 copie seul les octets entre l'USART2 et la mémoire, sans interruption. Les données échangées entre l'interruption USB et la boucle passent par des **tampons circulaires** (`cdc_port.c`) : l'interruption avance seulement l'index d'écriture, la boucle seulement l'index de lecture, ce qui évite tout conflit sans bloquer les interruptions.

> **Attention :** dans un gestionnaire d'interruption : rien de long, pas de `console_printf()`, pas de commande AT. On y range une donnée ou on y lève un drapeau, et la boucle principale fait le reste.

3.4 Parcours d'une commande tapée au terminal

Exemple de la commande `scan` (démo 1). Les caractères tapés arrivent par l'USB et sont rangés par `cdc_port.c` ; `console_readline()` (éditeur de ligne) rend la ligne complète à l'appui sur Entrée. `cli_exec()` la découpe en mots (`util_tokenize()`), cherche le premier mot dans la table `s_cmds[]` et appelle la fonction associée, ici `cmd_scan()`.

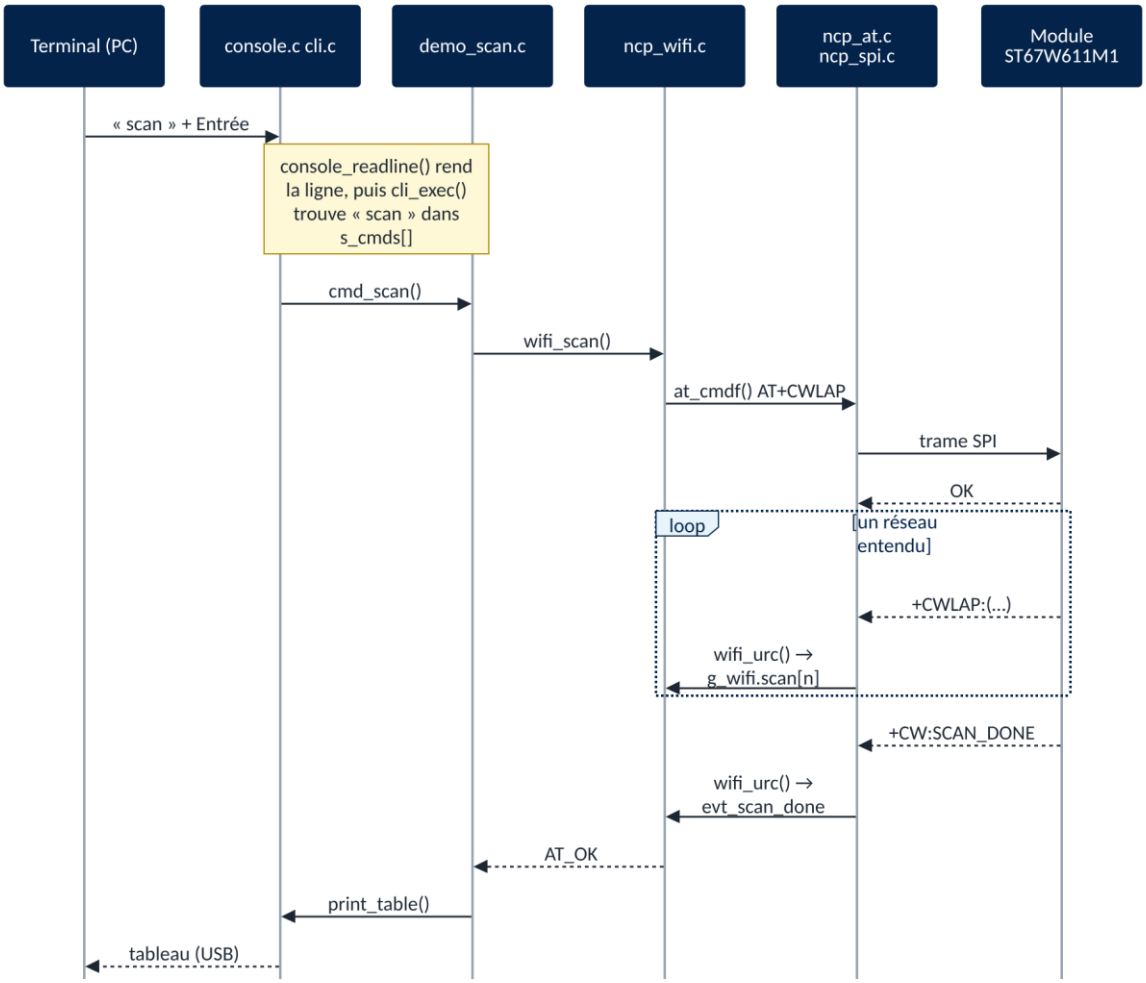


Figure 5 – Parcours de la commande scan, du terminal au module et retour

`wifi_scan()` envoie `AT+CWLAP` au module. La liste des réseaux n'arrive pas dans la réponse à la commande : le module envoie une ligne `+CWLAP:(…)` par réseau entendu, puis `+CW:SCAN_DONE`, sous forme d'**événements non sollicités** (URC). `wifi_scan()` attend ce dernier en appelant `at_poll()` ; chaque ligne est rangée dans `g_wifi.scan[]` par `wifi_urc()`. `cmd_scan()` trie et affiche ensuite le tableau.

3.5 Parcours d'une requête web

Le serveur TCP tourne **dans le module** (commande `AT+CIPSERVER`). Quand un navigateur se connecte, le module l'annonce (`+CIP:0,CONNECTED`) ; quand des données arrivent, il annonce leur nombre (`+IPD:0,312`) mais les garde : c'est le mode de « réception passive », où l'application vient les chercher quand elle est prête.

`web_poll()` lit les données (`net_recv()`, commande `AT+CIPRECVDATA`), les accumule dans `s_conn[lien].buf` jusqu'à ce que `http_parse_request()` reconnaisse une requête complète, puis `handle()` choisit la réponse selon le chemin : page HTML, `/api/status`, `/api/led`… La réponse est envoyée par `net_send()` (`AT+CIPSEND`) et la connexion fermée (`AT+CIPCLOSE`). Jusqu'à 4 navigateurs peuvent être servis en même temps, chacun sur son « lien ».

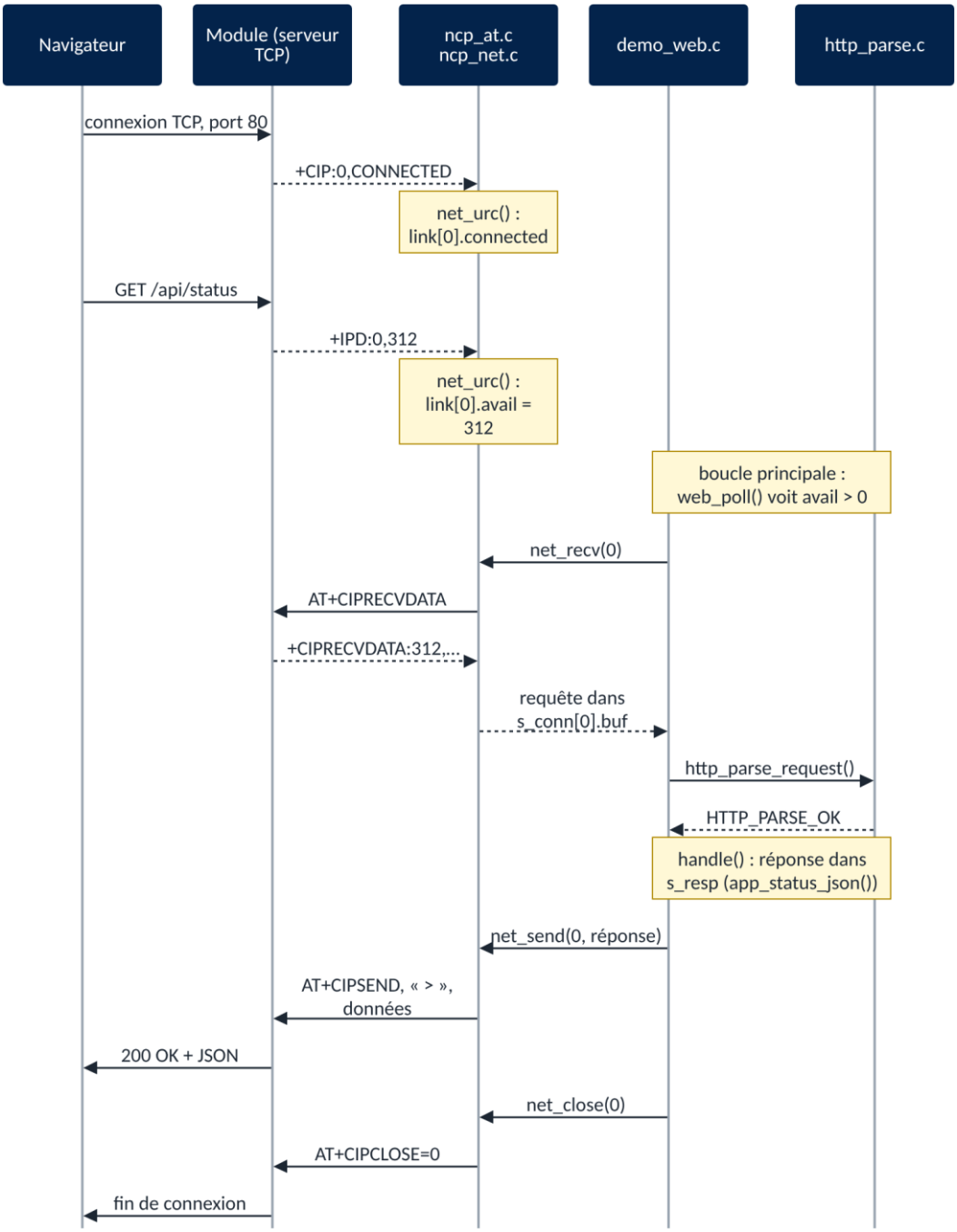


Figure 6 – Parcours d'une requête HTTP (ici GET /api/status)

3.6 Parcours d'un message MQTT

Le client MQTT est lui aussi **dans le module**. Un message reçu sur le topic de commande arrive dans le flux AT (`+MQTT:SUBRECV:…`) pendant que le moteur AT analyse les trames, éventuellement au milieu de l'attente d'une autre commande. Le rappel `mqttapp_on_message()` se contente donc de le **copier dans une file** de 4 messages ; il est traité plus tard, au tour de boucle suivant, par `mqttapp_poll()`.

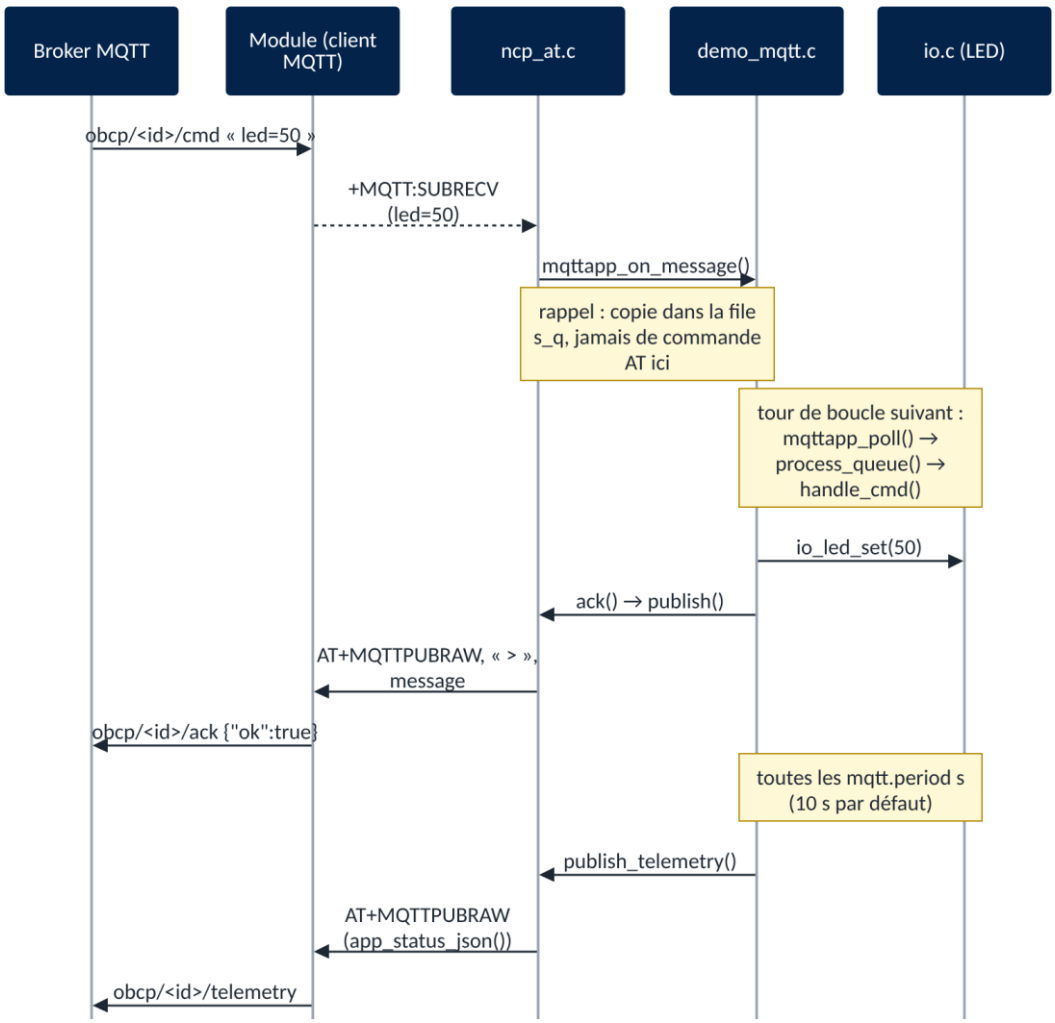


Figure 7 – Commande MQTT reçue, réponse et télémétrie

> **Attention :** règle du pilote : **jamais de commande AT dans un rappel** (`xxx_urc()`, `mqttapp_on_message()`…). Le moteur AT ne traite qu'une commande à la fois ; un rappel est appelé pendant l'attente d'une réponse : envoyer une commande à ce moment-là la ferait refuser (`AT_BUSY`).

3.7 Démarrage automatique des services (version 1.2)

Sur batterie, sans terminal, la carte peut lancer seule ses services à la mise sous tension, avec la connexion enregistrée. Trois paramètres le décident (chapitre 8), réglables par la commande `auto` ou la page web (mode d'emploi : guide d'utilisation, 5.6) :

Paramètre	Effet à la mise sous tension	Code
`wifi.auto = 1`	connexion à la box enregistrée (`wifi.ssid`), puis reconnexion automatique en cas de perte	`netapp_boot()`, `netapp_poll()`
`web.auto = 1`	point d'accès `ObCP-WiFi-xxxxxx` et serveur web, lancés une fois la tentative de connexion à la box terminée	`web_boot()`, `web_poll()`
`mqtt.auto = 1`	connexion au broker dès que la carte a obtenu son adresse IP sur la box, télémétrie	`mqttapp_sta_up()`, `mqttapp_poll()`

Le lancement du serveur web est **différé** : `web_boot()` le programme seulement (`s_pending`) et `web_poll()` l'exécute au moins 1,5 s plus tard, et pas tant qu'une connexion à la box est en cours (35 s au plus). Le module

règle ainsi son point d'accès sur le canal de la box, et deux séquences de commandes AT ne s'entremêlent pas. Si le module redémarre de lui-même, le point d'accès et le serveur sont relancés de la même façon (4.4).

Les messages affichés en tâche de fond pendant ce lancement (adresse, mot de passe du point d'accès) sont encadrés par `console_async_begin()` et `console_async_end()`, qui effacent puis réaffichent proprement la ligne de saisie.

4. Dialogue avec le module Wi-Fi

4.1 Un coprocesseur piloté par commandes AT

Le ST67W611M1 est un **coprocesseur réseau** (NCP, « Network Co-Processor ») : il contient la radio Wi-Fi 6, mais aussi toute la pile réseau (TCP/IP, DHCP, DNS, SNTP, serveur TCP, client MQTT), exécutée par son propre processeur avec le firmware « mission T01 ». Le STM32 n'a donc pas de pile TCP/IP : il envoie au module des **commandes AT**, des lignes de texte qui commencent par `AT+`, et lit ses réponses. Exemple de connexion à une box :

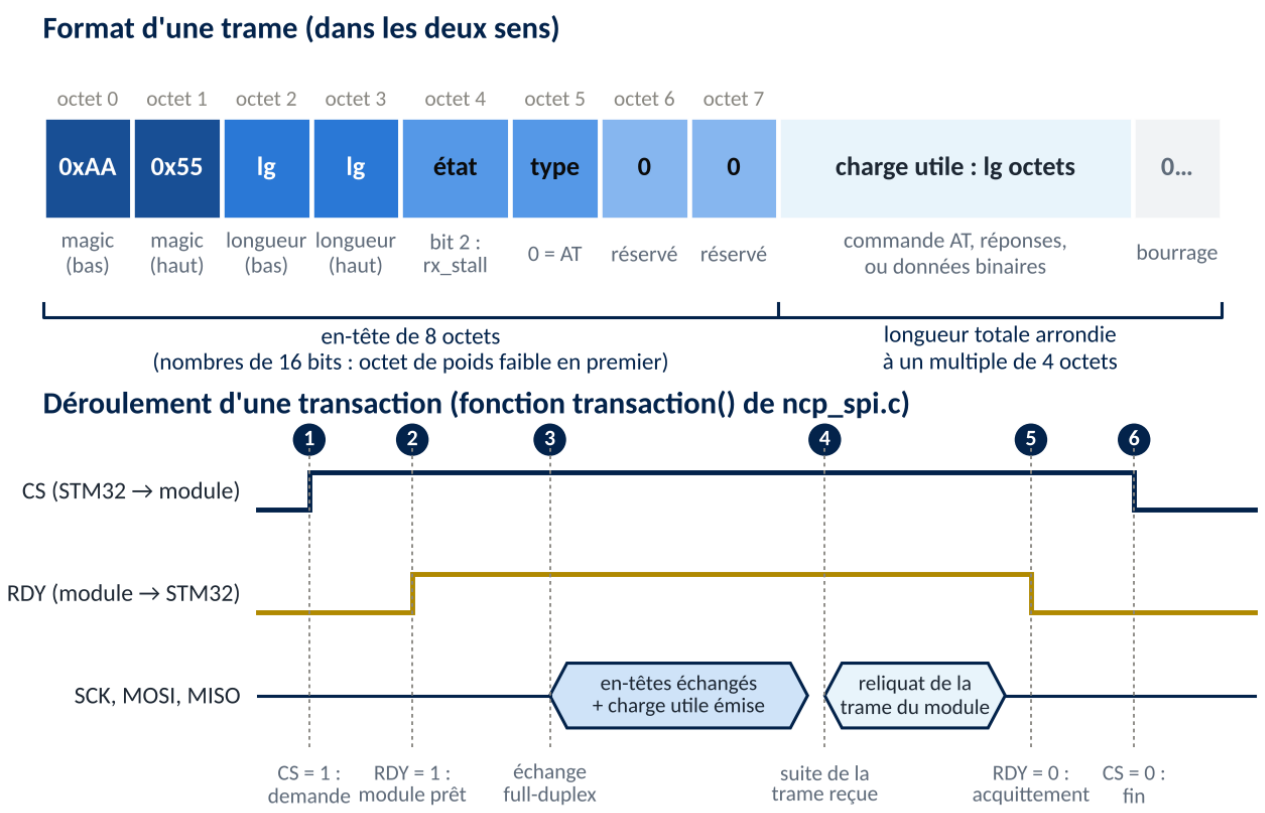
```
→ AT+CWJAP="MaBox","motdepasse",,0    wifi_connect_start() : demande de connexion
← OK                                     commande acceptée (la connexion continue)
← +CW:CONNECTED                          associé à la box : événement non sollicité
← +CW:GOTIP                              adresse IP obtenue (DHCP) : liaison utilisable
```

Chaque commande reçoit une réponse finale, `OK` ou `ERROR`, éventuellement précédée de lignes de résultat (`+CWJAP:…`). Le module envoie aussi, à tout moment, des **événements non sollicités** (URC, « Unsolicited Result Code ») : connexion perdue, appareil connecté au point d'accès, données reçues, message MQTT… La commande `log on` du terminal affiche tout ce trafic (mots de passe masqués) : c'est le meilleur moyen de voir le pilote travailler.

Le pilote (`Core/Src/st67w6x`) est organisé en couches, de la plus basse à la plus haute : `ncp_port_obcp.c` (broches et SPI de la carte), `ncp_spi.c` (trames SPI), `ncp_at.c` (moteur AT), puis les services `ncp_wifi.c`, `ncp_net.c`, `ncp_mqtt.c`, `ncp_fwu.c`, et enfin `ncp.c` (démarrage du module et répartition des événements).

4.2 Les trames SPI

Les commandes et réponses voyagent sur le bus SPI1 (10 MHz, mode 0) dans des **trames** : un en-tête de 8 octets suivi de la charge utile. Deux broches s'ajoutent au SPI : **CS**, mise à 1 par le STM32 pour demander un échange (elle réveille aussi le module), et **RDY**, mise à 1 par le module quand il est prêt ou quand il a lui-même quelque chose à envoyer.



Le module lève aussi RDY de lui-même quand il a une trame à envoyer : ncp_spi_poll() la lit alors par une transaction sans charge utile.

Figure 8 – Trame SPI et déroulement d'une transaction

Sur un bus SPI, chaque octet émis fait revenir un octet reçu : le STM32 et le module échangent donc leurs en-têtes **en même temps** (« full-duplex ») et chacun y lit la longueur de ce que l'autre a à transmettre. Si le module n'a plus de place pour recevoir, il le signale par le bit `rx_stall` : `ncp_spi_send()` renvoie alors la trame un peu plus tard (jusqu'à 20 essais en 3 s). Les transferts sont bloquants (sans DMA) et bornés par des délais de garde ; les compteurs de la liaison (trames, erreurs, refus) sont affichés par la commande `stats`.

> **Bon à savoir :** si l'en-tête reçu ne commence pas par `0x55AA`, le module n'exécute pas le bon firmware : `app_module_start()` le détecte (`ncp_spi_stats()->hdr_errors`) et affiche la marche à suivre (valeur `0x5501` : firmware trop ancien).

4.3 Le moteur AT : réponses et événements

`ncp_at.c` transforme les trames reçues en lignes et aiguille chacune vers son destinataire (figure ci-après). Une seule commande peut être en cours à la fois : `at_cmd()` envoie la commande puis appelle `at_poll()` en boucle jusqu'à la réponse finale ou la fin du délai ; pendant cette attente, les événements continuent d'être traités.

Les lignes qui commencent par le préfixe attendu par la commande en cours sont passées à une fonction de rappel fournie par l'appelant. Exemple tiré de `ncp_at.h` :

```
at_cmd("AT+CWJAP?", "+CWJAP:", on_line, &ctx, 2000);
   envoie la commande, appelle on_line(args, &ctx) pour chaque ligne « +CWJAP:... »
   (args = la ligne sans son préfixe), rend AT_OK quand le module a répondu OK,
   AT_ERROR s'il a répondu ERROR, AT_TIMEOUT au bout de 2000 ms.
```

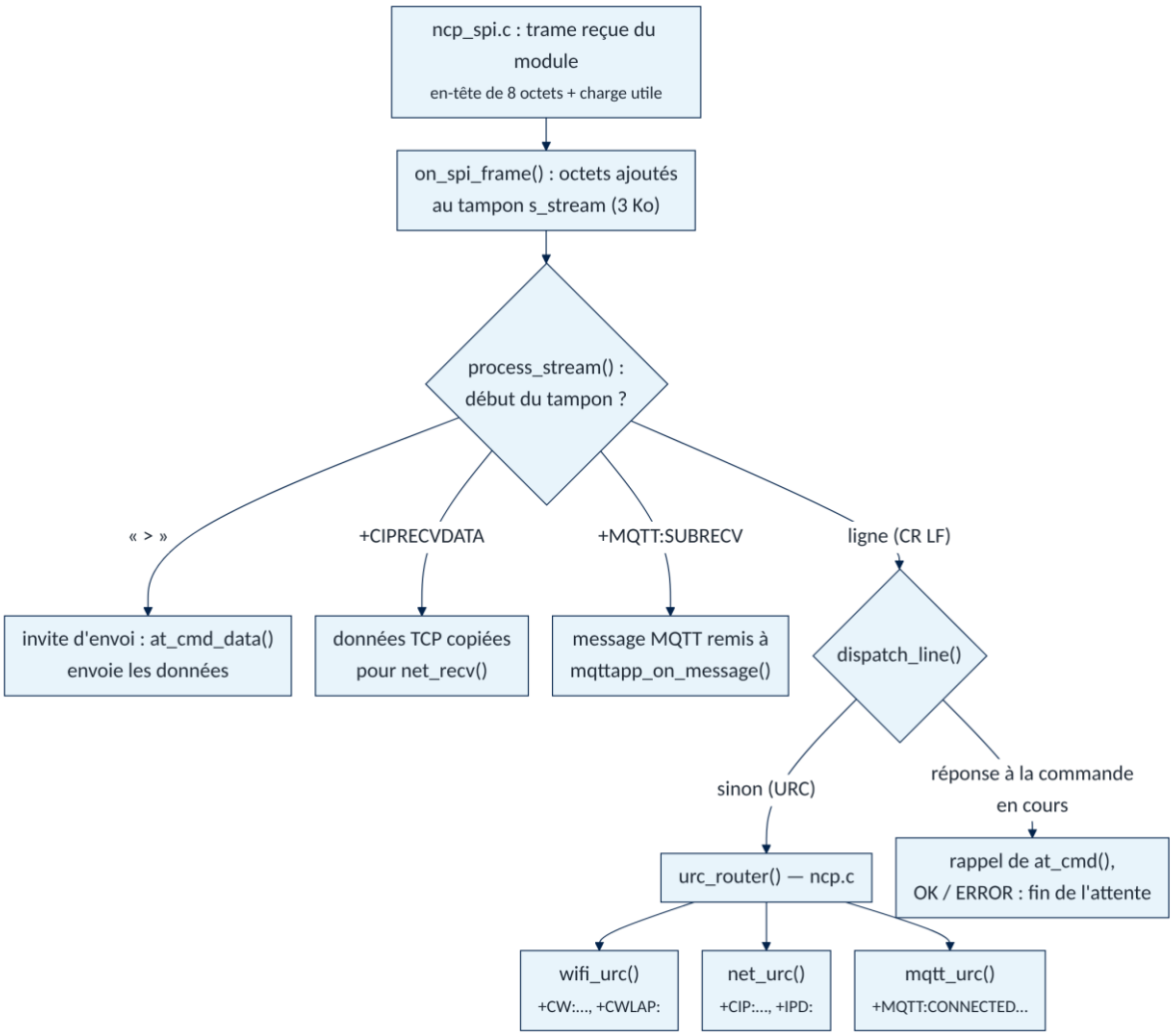


Figure 9 – Aiguillage des données reçues par le moteur AT

Deux réponses transportent des **données binaires** (qui peuvent contenir n'importe quel octet, y compris des retours à la ligne) : `+CIPRECVDATA:<n>,<données>` (données TCP lues par `net_recv()`) et `+MQTT:SUBRECV:…` (message MQTT reçu). Leur longueur est annoncée en tête : `process_stream()` les extrait d'un bloc, sans les découper en lignes. Inversement, pour envoyer des données (`AT+CIPSEND`, `AT+MQTTPUBRAW`, `AT+OTASEND`), `at_cmd_data()` envoie la commande, attend l'invite `>` du module, envoie les octets puis attend la confirmation.

Événement du module	Traité par	Effet
`+CWLAP:(…)`	`wifi_urc()`	un réseau trouvé, rangé dans `g_wifi.scan[]`
`+CW:SCAN_DONE`	`wifi_urc()`	fin du balayage (`evt_scan_done`)
`+CW:CONNECTING`, `CONNECTED`, `GOTIP`	`wifi_urc()`	progression de la connexion de la station (`g_wifi.sta`)
`+CW:DISCONNECTED`, `+CW:ERROR,<raison>`	`wifi_urc()`	liaison perdue ou échec, avec le code de raison (`last_reason`)
`+CW:STA_CONNECTED`, `STA_DISCONNECTED`, `DIST_STA_IP`	`wifi_urc()`	appareil connecté au point d'accès, parti, ou adresse attribuée
`+CIP:<lien>,CONNECTED` / `DISCONNECTED`	`net_urc()`	connexion TCP ouverte ou fermée (`g_net.link[]`)
`+IPD:<lien>,<n>`	`net_urc()`	n octets arrivés sur un lien, gardés par le module
`+PING:<ms>`	`net_urc()`	écho reçu (ou `TIMEOUT`), transmis au rappel de `net_ping()`

Événement du module	Traité par	Effet
`+MQTT:CONNECTED` / `DISCONNECTED`	`mqtt_urc()`	état du client MQTT (`g_mqtt.connected`)
`+MQTT:SUBRECV:…`	`ncp_at.c`	message reçu, remis au rappel `mqttapp_on_message()`
`ready`	`ncp_at.c`, `ncp.c`	le module a (re)démarré ; s'il ne l'a pas demandé, `ncp_restarted()` le signale à `app_idle()`

4.4 Les machines à états

Une connexion Wi-Fi peut prendre plusieurs secondes, une connexion au broker MQTT aussi. Pour attendre sans bloquer la boucle principale, chaque tâche de fond est une **machine à états** : une variable mémorise où en est l'opération ; à chaque appel, la tâche regarde cet état et les drapeaux d'événements arrivés, agit si besoin (une commande AT rapide) et change d'état. Les trois figures suivantes décrivent celles du programme.

Connexion à la box (demo_net.c, netapp_poll())

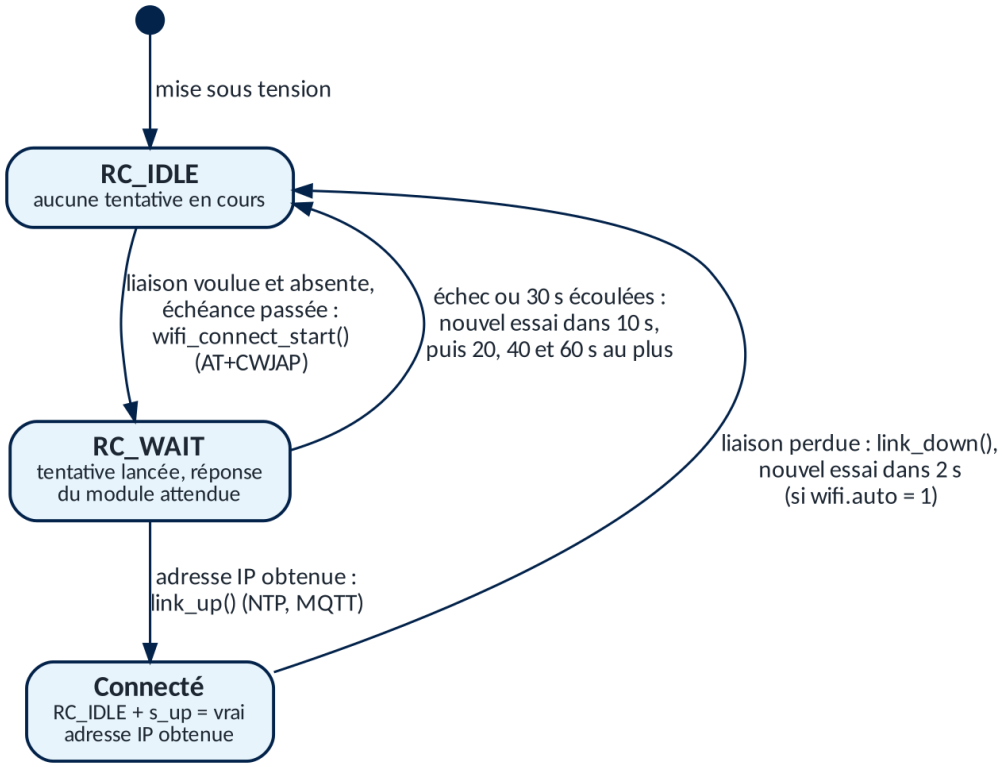


Figure 10 – Connexion et reconnexion automatiques à la box

Après un échec, le délai avant le nouvel essai double (10, 20, 40 puis 60 s au plus) pour ne pas saturer la box ; il revient à 10 s après une connexion réussie. La commande `connect`, elle, attend le résultat pour afficher le détail de la liaison : `netapp_connect()` appelle `wifi_connect()`, qui scrute le module (`at_poll()`) jusqu'à 30 s ; les autres tâches de fond sont suspendues pendant cette attente. Côté pilote, `g_wifi.sta` suit les étapes annoncées par le module : `WIFI_STA_IDLE`, `CONNECTING`, `CONNECTED` (associée), `GOT_IP` (adresse obtenue).

Point d'accès et serveur web (demo_web.c, web_poll())

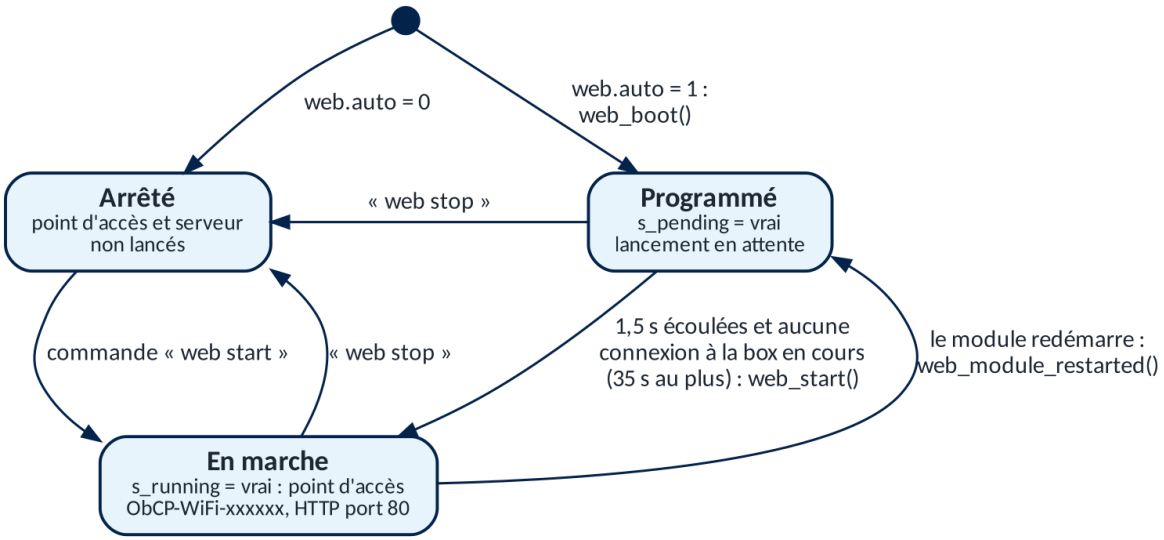


Figure 11 – Lancement du point d'accès et du serveur web

Client MQTT (demo_mqtt.c, mqttapp_poll())

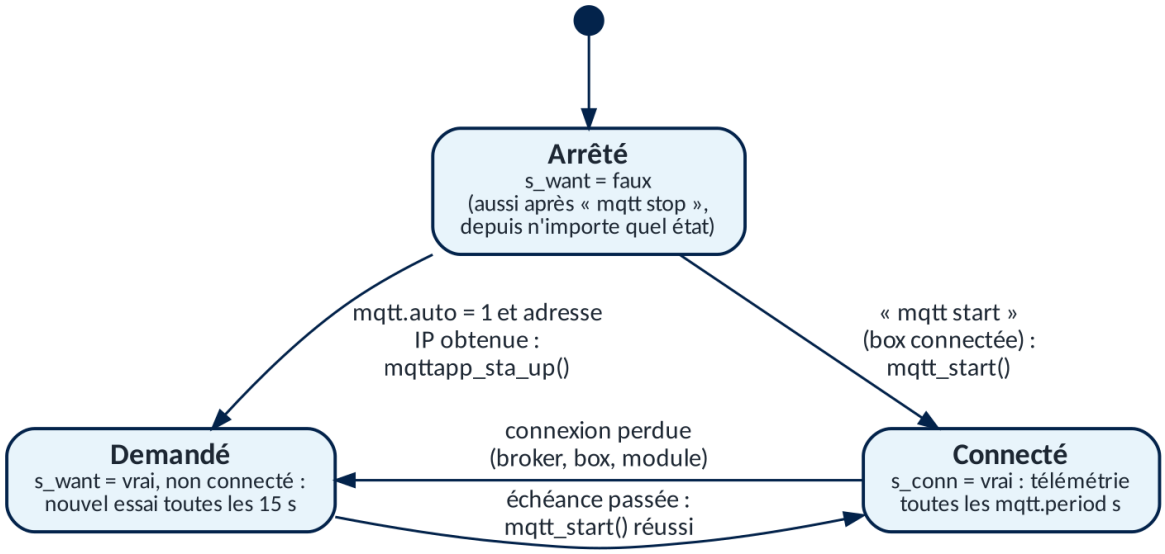


Figure 12 – Connexion au broker MQTT

À la connexion, `mqtt_start()` configure le client avec un **testament** (« LWT ») : si la carte disparaît sans prévenir, le broker publie `offline` sur `<racine>/status` à sa place. Elle publie ensuite `online` (message retenu), s'abonne à `<racine>/cmd` et programme la première télémétrie.

4.5 Commandes AT utilisées

Liste extraite du code du pilote : chaque commande AT et la fonction qui l'envoie. Les fonctions sont décrites au chapitre 7 ; la syntaxe complète des commandes est dans la documentation du firmware « mission T01 » (X-CUBE-ST67W61).

Commande	Envoyée par	Rôle
`AT+CWNETMODE`	`ncp_start()`	contrôle de l'architecture du firmware (T01)
`AT+GET_CLOCK`	`ncp_start()`	lecture de la source d'horloge du module

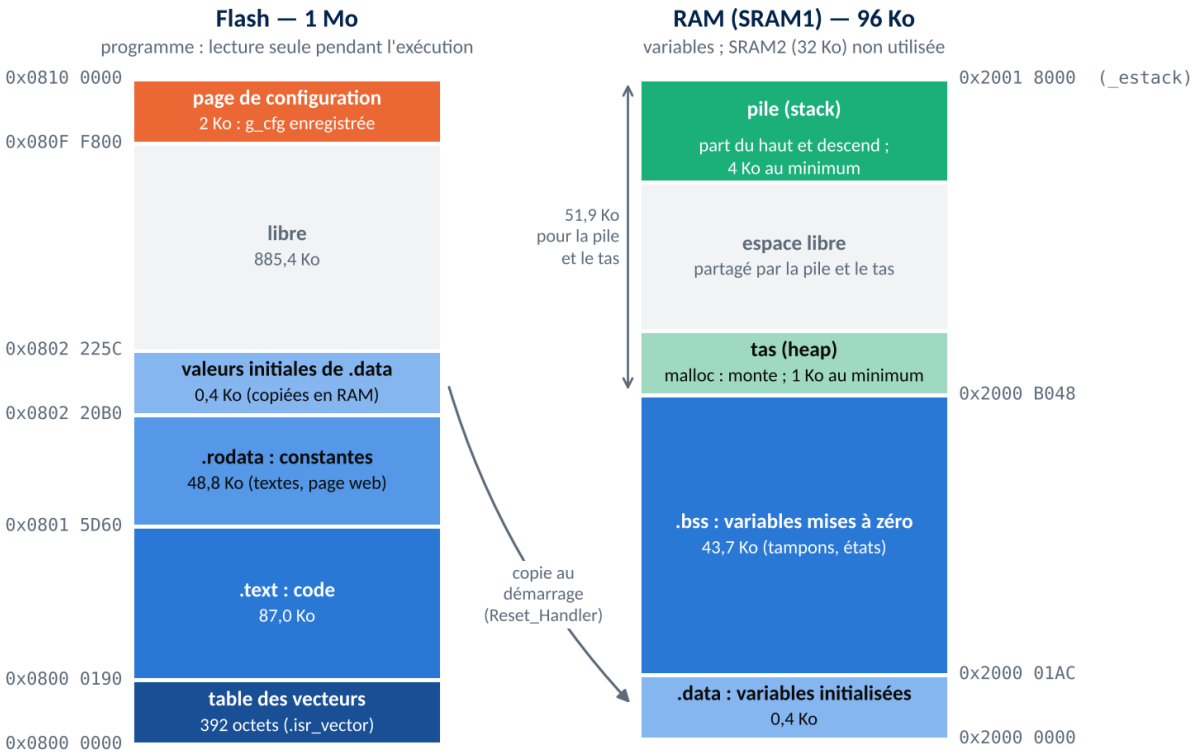
Commande	Envoyée par	Rôle
AT+GMR	ncp_start()	versions du firmware du module
AT+PWR	ncp_start()	pas d'économie d'énergie du module
AT+SET_CLOCK	ncp_start()	choix de la source d'horloge (le module redémarre)
AT+CWAPPROTO	wifi_ap_start()	normes du point d'accès (b/g/n/ax)
AT+CWAUTOCONN	wifi_init()	pas de connexion automatique décidée par le module
AT+CWCOUNTRY	wifi_init()	pays (canaux autorisés)
AT+CWJAP	wifi_connect_start(), wifi_link_info()	connexion à un réseau, ou caractéristiques de la liaison (?)
AT+CWLAP	wifi_scan()	balayage des réseaux
AT+CWLAPOPT	wifi_scan()	options du balayage
AT+CWLIF	wifi_ap_clients()	appareils connectés au point d'accès
AT+CWMODE	wifi_ap_start(), wifi_ap_stop(), wifi_init()	mode Wi-Fi : station seule ou station + point d'accès
AT+CWQAP	wifi_disconnect()	déconnexion
AT+CWRECONNCFG	wifi_connect_start()	pas de reconnexion par le module (gérée par l'application)
AT+CWSAP	wifi_ap_start()	création du point d'accès
AT+CWSTATE	wifi_conn_details()	état et norme de la liaison
AT+SLWKDTIM	wifi_connect_start(), wifi_init()	intervalle de réveil DTIM 1 (réactivité)
AT+CIPAP	net_ap_ip(), net_init()	adresse du point d'accès
AT+CIPCLOSE	net_close()	fermeture d'un lien
AT+CIPDINFO	net_init()	adresse et port de l'expéditeur dans +IPD
AT+CIPDNS	net_dns_servers(), net_set_dns()	serveurs DNS
AT+CIPDOMAIN	net_resolve()	résolution d'un nom (DNS)
AT+CIPMUX	net_init()	plusieurs connexions simultanées
AT+CIPRECVBUF	net_init()	tampon de réception de chaque lien (4 Ko)
AT+CIPRECVDATA	net_recv()	lecture des données reçues sur un lien
AT+CIPRECVMODE	net_init()	réception passive (+IPD puis lecture)
AT+CIPSEND	net_send()	envoi de données sur un lien
AT+CIPSERVER	net_server_start(), net_server_stop()	démarrage ou arrêt du serveur TCP
AT+CIPSERVERMAXCONN	net_server_start()	nombre de connexions du serveur TCP
AT+CIPSNTPCFG	net_sntp_config()	client de mise à l'heure (SNTP)
AT+CIPSNTPTIME	net_sntp_time()	lecture de l'heure
AT+CIPSTA	net_set_static(), net_sta_ipcfg()	adresse IP de la station (lecture ou IP fixe)
AT+CIPSTAMAC	net_sta_mac()	adresse MAC de la station
AT+CIPV6	net_init()	IPv4 seulement

Commande	Envoyée par	Rôle
AT+CWDHCP	net_init(), net_set_dhcp()	client DHCP (station) et serveur DHCP (point d'accès)
AT+CWHOSTNAME	net_init()	nom d'hôte annoncé à la box
AT+PING	net_ping()	ping
AT+MQTTCLEAN	mqtt_disconnect()	déconnexion du broker
AT+MQTTCONN	mqtt_connect()	connexion au broker
AT+MQTTCONNCFG	mqtt_configure()	« signe de vie » et testament (LWT)
AT+MQTTPUBRAW	mqtt_publish()	publication d'un message
AT+MQTTSUB	mqtt_subscribe()	abonnement à un topic
AT+MQTTUSERCFG	mqtt_configure()	identifiant du client, nom d'utilisateur, mot de passe
AT+OTAFIN	ncp_fwu_finish()	vérification de l'image et redémarrage du module
AT+OTASEND	send_buf()	envoi d'un bloc du nouveau firmware
AT+OTASTART	ncp_fwu_abort(), ncp_fwu_begin()	ouverture (ou abandon) d'une mise à jour du firmware

5. Mémoire

5.1 Plan de la mémoire

Le STM32L476RG a 1 Mo de **flash** (le programme, conservé hors tension) et 128 Ko de **RAM** (les variables, perdues hors tension) : SRAM1 (96 Ko, utilisée par le programme) et SRAM2 (32 Ko, libre). Le placement est décidé par le script d'édition de liens `STM32L476RGTX_FLASH.ld`, généré par CubeMX et non modifié. La figure suivante en donne le plan ; les adresses sont celles du programme compilé.



Hauteurs non proportionnelles aux tailles ; adresses relevées dans ObCP_WiFi_Demo.elf (script STM32L476RGTX_FLASH.ld).

Figure 13 – Plan de la mémoire : flash et RAM

- **.text** : le code des fonctions ; **.rodata** : les constantes, en particulier les textes (messages, aide des commandes) et la page web (10,4 Ko).
- **.data** : les variables globales ou `static` qui ont une valeur initiale (`static int x = 5;`) : leurs valeurs sont en flash et copiées en RAM au démarrage.
- **.bss** : les autres variables globales ou `static`, mises à zéro au démarrage. Les tampons (USB, SPI, requêtes HTTP…) y sont tous : leur taille est donc fixée à la compilation, sans allocation dynamique.
- **Pile** : variables locales des fonctions et adresses de retour ; elle part du haut de la RAM et descend. Un tableau local (`char buf[384]` dans `console_event()`) n'y occupe de la place que pendant l'exécution de la fonction.
- **Tas** : mémoire allouée par `malloc()` ; le programme n'en alloue pas lui-même, seule la bibliothèque C peut s'en servir.

Le script réserve au minimum 4 Ko de pile et 1 Ko de tas (`_Min_Stack_Size`, `_Min_Heap_Size`) : l'édition de liens échoue si les variables ne laissent pas cette place. Il reste ici 51,9 Ko entre la fin de .bss et le haut de la RAM.

5.2 Occupation par partie du programme

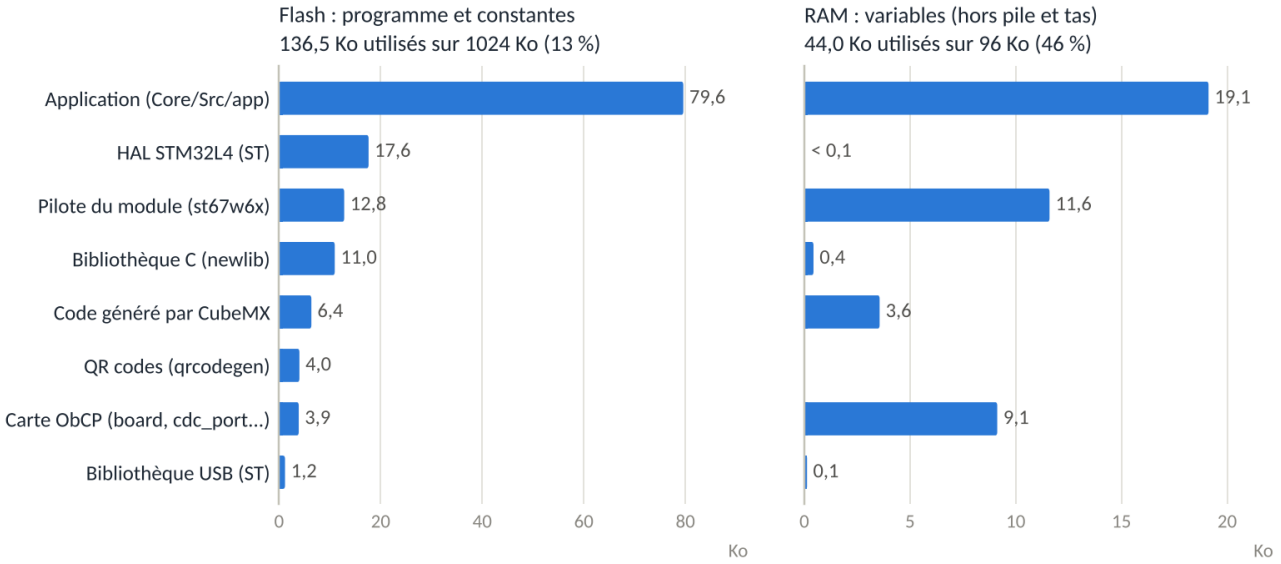


Figure 14 – Flash et RAM utilisées, par partie du programme

Dans la flash, l'application domine : `cli.c` (20,6 Ko, surtout les textes d'aide des commandes) et `web_page.c` (10,4 Ko : la page HTML, son style et son JavaScript). Dans la RAM, ce sont les tampons : requêtes et réponses du serveur web, tampons USB, trames SPI. Le tableau suivant détaille les fichiers de la démo, triés par place en flash.

Fichier	Flash	RAM
`cli.c`	20,6 Ko	0 octets
`web_page.c`	10,4 Ko	0 octets
`demo_net.c`	8,4 Ko	68 octets
`demo_mqtt.c`	5,6 Ko	1,8 Ko
`demo_web.c`	5,6 Ko	12,6 Ko
`app.c`	5,2 Ko	229 octets
`demo_scan.c`	4,8 Ko	6 octets
`module_ota.c`	4,2 Ko	788 octets
`ncp_wifi.c`	3,8 Ko	1,8 Ko
`cfg_codec.c`	3,6 Ko	0 octets
`ncp_net.c`	3,0 Ko	152 octets
`ncp_at.c`	2,9 Ko	3,6 Ko
`util.c`	2,2 Ko	0 octets
`ymodem.c`	2,0 Ko	1,0 Ko
`console.c`	1,6 Ko	399 octets
`board.c`	1,4 Ko	8 octets

Valeurs relevées dans le fichier `.map` produit par l'éditeur de liens ; les textes identiques étant regroupés en une seule copie, leur place est répartie entre les fichiers au prorata (valeurs approchées à quelques pour cent).

5.3 Les plus grosses variables

Variable	Fichier	Taille	Rôle
s_conn	demo_web.c	7 720 o	requêtes HTTP en cours de réception, une par lien (5 × 1,5 Ko)
s_resp	demo_web.c	5 120 o	réponse HTTP en préparation (en-tête + JSON)
s_tx	cdc_port.c	4 096 o	tampon circulaire d'émission USB (messages vers le terminal)
s_stream	ncp_at.c	3 072 o	flux reçu du module, en attente d'analyse
s_rx	ncp_spi.c	2 060 o	trame SPI reçue (en-tête + 2 Ko au plus)
s_tx	ncp_spi.c	2 060 o	trame SPI à émettre
s_buf	ncp_fwu.c	2 048 o	bloc de firmware en cours d'envoi au module
s_rx	cdc_port.c	2 048 o	tampon circulaire de réception USB (caractères tapés)
s_tx	uart_bridge.c	2 048 o	passerelle module boot : octets du PC vers le module
g_wifi	ncp_wifi.c	1 844 o	état Wi-Fi et résultats du dernier balayage
hpcd_USB_OTG_FS	usbd_conf.c	1 252 o	état du périphérique USB (HAL)
s_q	demo_mqtt.c	1 192 o	file des messages MQTT reçus (4 places)
blob	config.c	1 024 o	image de la configuration à écrire en flash
s_buf	ymodem.c	1 024 o	bloc YMODEM en cours de réception
s_rx_dma	uart_bridge.c	1 024 o	passerelle module boot : réception par DMA
s_ota	module_ota.c	788 o	état de la mise à jour du module

`blob` est une variable `static` déclarée à l'intérieur d'une fonction (`config_save()`) : elle est rangée avec les autres dans .bss et garde sa valeur entre deux appels.

5.4 La configuration enregistrée en flash

La configuration (`g_cfg`, structure `cfg_t` de 544 octets) est enregistrée dans la **dernière page de la flash** : 2 Ko à l'adresse `0x080F F800` (banque 2, page 255). `config_save()` efface la page (`config_erase()`) puis l'écrit par mots de 64 bits ; `config_load()` la relit au démarrage. Le programme (environ 140 Ko) n'atteint jamais cette page, et `config_erase()` refuse de l'effacer si c'était le cas.

Octets	Contenu
0 à 3	signature `CFG_MAGIC` = 0x43373657 (« W67C ») : page vierge ou étrangère si elle manque
4 et 5	version du format (`CFG_VERSION` = 2 depuis le programme 1.2.0)
6 et 7	taille de la structure enregistrée
8 à 11	CRC32 de la structure : détecte une configuration abîmée
12 à 15	réservé
16 et suivants	la structure `cfg_t`, puis 0xFF (flash effacée)

Au démarrage, `cfg_unpack()` part des valeurs d'usine puis recopie la structure lue : une configuration enregistrée par une version plus ancienne (plus courte) reste lisible, les champs ajoutés depuis gardant leur valeur d'usine. Une configuration d'un format plus récent que le programme est ignorée (message « configuration en flash

invalide ou d'une version plus récente »). La recette du chapitre 11 montre comment ajouter un paramètre sans perdre les configurations existantes.

6. Matériel et brochage

6.1 Broches du STM32L476RG

Les broches sont réglées dans STM32CubeMX (`ObCP_WiFi_Demo.ioc`) et portent une **étiquette** (« User Label ») : `NCP_CS`, `Led_Red`, `User1`… CubeMX en tire des noms dans `main.h` (`NCP_CS_Pin`, `NCP_CS_GPIO_Port`), que `board.h` renomme pour la démo (`NCP_CS_PIN`, `NCP_CS_PORT`). Déplacer une broche dans CubeMX en gardant son étiquette suffit donc : aucun fichier de la démo n'est à modifier.

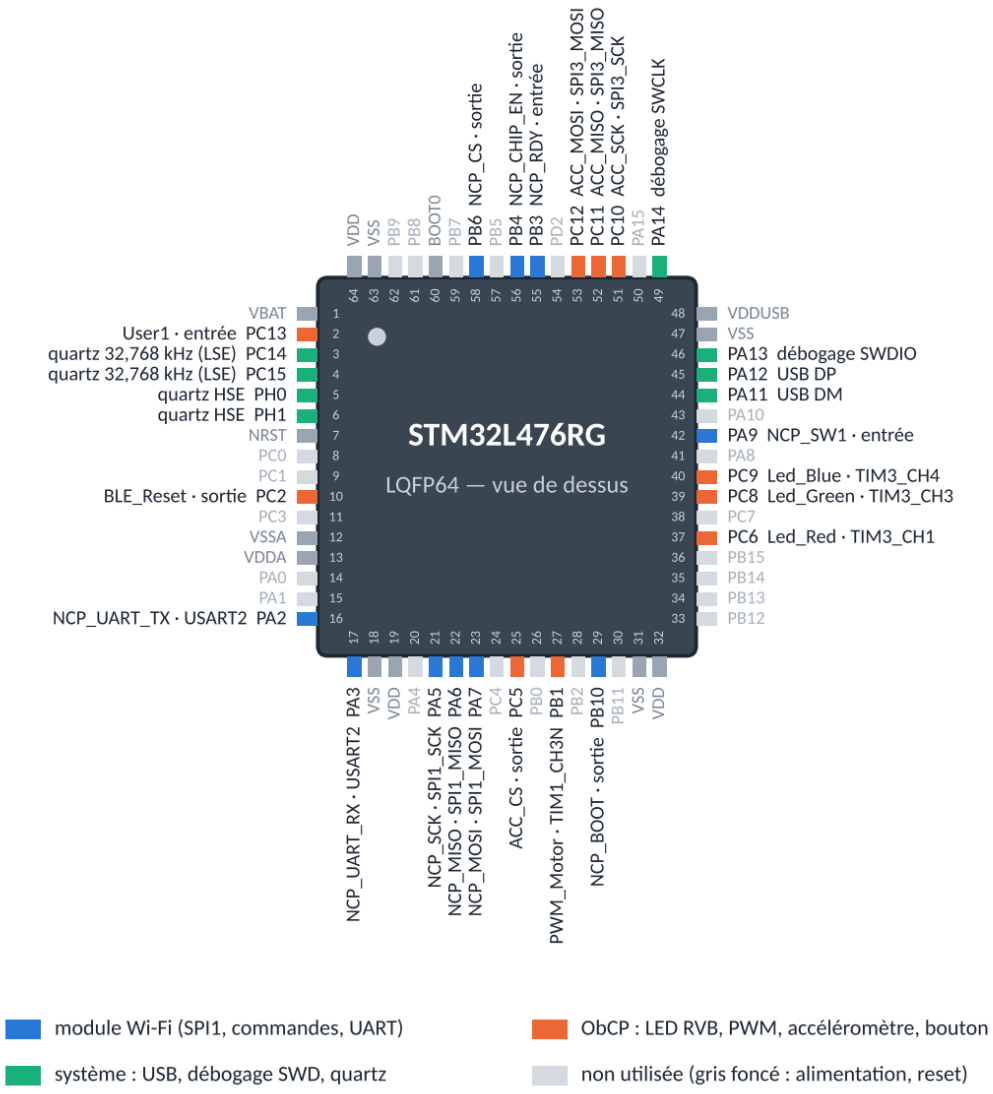


Figure 15 – Broches du STM32L476RG utilisées par la démo (boîtier LQFP64, vu de dessus)

6.2 Périphériques et fichiers qui les pilotent

Périphérique	Réglage	Usage	Piloté par
Horloges	quartz 8 MHz → PLL → 80 MHz ; USB 48 MHz (PLLSAI1) ; quartz 32,768 kHz	cadence du programme, USB, RTC	`main.c` (SystemClock_Config), `board_lse_start()`

Périphérique	Réglage	Usage	Piloté par
SPI1	maître, mode 0, 10 MHz, transferts bloquants	module Wi-Fi (trames)	`ncp_port_obcp.c`
PB6, PB3, PB4, PB10	sorties CS, CHIP_EN, BOOT ; entrée RDY	commande du module	`ncp_port_obcp.c`
SPI3 + PC5	maître, mode 3, 5 MHz	accéléromètre LIS3DH / LIS3DSH	`io.c`, `lis3dh.c`
TIM3 CH1, CH3, CH4	PWM 1 kHz	LED RVB	`io.c`, `color.c`
TIM1 CH3N	PWM 1 kHz	sortie PWM du bornier J13	`io.c`
ADC1	12 bits, capteur de température et VREFINT internes	température, tension VDDA	`io.c`
RTC	horloge LSE, registres de sauvegarde	date et heure UTC, rapport de faute	`board.c`, `fault_handlers.c`
USB OTG FS	périphérique, classe CDC	console (port COM virtuel)	`cdc_port.c`, `console.c`
USART2 + DMA1	débit choisi par le PC	passerelle `module boot`	`uart_bridge.c` (hors CubeMX)
Flash	page 255 de la banque 2	configuration	`config.c`
PC13, PA9	entrées	boutons User1 et SW1	`io.c`

> **Bon à savoir :** l'USART2 et son DMA ne sont pas déclarés dans CubeMX : `uart_bridge.c` les configure lui-même, seulement pendant `module boot`, puis remet PA2 et PA3 en entrées. Ne pas les activer dans CubeMX : la broche TX piloterait l'UART du module en permanence.

7. Référence des fonctions

Ce chapitre décrit chaque module : son rôle, ses fichiers, les modules qu'il utilise et ceux qui l'utilisent, puis ses **295 fonctions publiques** (déclarées dans l'en-tête .h, utilisables par les autres fichiers) et ses **175 fonctions internes** (`static` : visibles seulement dans leur fichier .c). Les textes sont ceux des commentaires du code, extraits automatiquement.

Lecture d'une fiche de fonction : la première ligne est le **prototype** (type rendu, nom en gras, paramètres) ; viennent ensuite le rôle de la fonction, la signification de chaque paramètre et la valeur rendue. Les modules sont présentés couche par couche, de l'application vers le matériel ; l'annexe A les classe par ordre alphabétique.

Couche	Modules
Application : le cœur	app, console, demos (commandes et démos), config, cfg_codec, app_config
Application : services	io, color, lis3dh, http_parse, web_page, qr_term, module_ota, ymodem, bridge_ctl, timeutil, util
Carte ObCP	board, cdc_port, uart_bridge, ncp_port (ncp_port_obcp.c)
Pilote du module	ncp, ncp_spi, ncp_at, ncp_wifi, ncp_net, ncp_mqtt, ncp_fwu
Code généré par CubeMX	main.c, stm32l4xx_it.c, usbd_cdc_if.c, usbd_conf.c, fault_handlers.c (7.29)

7.1 app.h — démarrage, boucle principale, état général

Application de démonstration : initialisation, boucle principale, tâches de fond et fonctions communes aux quatre démos.

Fichiers : `Core/Inc/app.h`, `Core/Src/app/app.c`

Utilise : board, cfg_codec, cli, color, config, console, demo_mqtt, demo_net, demo_web, io, ncp, ncp_at, ncp_mqtt, ncp_net, ncp_spi, ncp_wifi, util, HAL

Utilisé par : cli, demo_mqtt, demo_net, demo_scan, demo_web, main, module_ota

Point d'entrée de l'application, appelé par `main.c` (code généré) :

```
app_init()  une fois, après l'initialisation du matériel ;
app_loop()  sans fin, dans la boucle while (1) de main().
```

Le programme n'utilise pas de système d'exploitation temps réel (RTOS) : tout se déroule dans cette boucle, qui ne doit jamais rester bloquée longtemps. Chaque tour lit le terminal, exécute la commande éventuelle, puis appelle `app_idle()` : chaque démo y fait avancer ses « tâches de fond » (reconnexion Wi-Fi, requêtes web, télémétrie MQTT...). Les commandes longues (scan, ping, track...) appellent elles aussi `app_idle()` pendant leurs attentes.

Fonctions publiques (9)

`void` **`app_init`**`(void)`

Initialisation de l'application (appelée par `main.c`, USER CODE 2) : console, configuration en flash, entrées / sorties, bannière, démarrage du module Wi-Fi, connexion et services automatiques.

`void` **`app_loop`**`(void)`

Un tour de la boucle principale (appelée sans fin par `main.c`) : lecture du terminal, exécution d'une commande, tâches de fond.

`void` **`app_idle`**`(void)`

Tâches de fond : événements du module, reconnexion Wi-Fi, NTP, serveur web, MQTT, boutons. Appelée par la boucle principale et par les commandes longues pendant leurs attentes. Protégée contre les appels imbriqués.

`int app_module_start(bool verbose)`

(Re)démarre le module Wi-Fi et initialise les couches Wi-Fi et réseau.

`verbose` : true : affiche la progression et les versions du firmware.

Retour : `AT_OK` (0) si le module répond, sinon un code d'erreur négatif.

`void app_print_module_info(void)`

Affiche les versions du firmware et l'adresse MAC du module.

`const char *app_err_str(int err)`

Message en français pour tout code d'erreur du pilote ou de l'application (AT_xxx, WIFI_ERR_xxx, APP_ERR_xxx...).

`int app_report(int err, const char *what)`

Affiche « ✗ <what> : <raison> » en rouge dans le terminal.

Retour : err (pour écrire : return app_report(st, "connexion");).

`size_t app_status_json(char *out, size_t len, bool with_rssi)`

État de la carte en JSON (page web /api/status, télémétrie MQTT) : capteurs, LED, PWM, boutons, liaison Wi-Fi, heure.

`out` : tampon de destination.

`len` : taille du tampon.

`with_rssi` : true : interroge le module pour le niveau du signal.

Retour : longueur du texte JSON, 0 si le tampon est trop petit.

`void app_autostart_summary(char *out, size_t len)`

Services lancés à la mise sous tension, en clair (bannière, commandes « info » et « auto ») : « connexion à « Box », serveur web, MQTT ».

Fonctions internes (4)

Fonction	Rôle
`mask_quoted()`	Remplace par des '*' le texte du index-ième champ entre guillemets : `AT+CWJAP="Box","secret"` -> `AT+CWJAP="Box","******"` (index = 2)
`print_banner()`	Bannière de démarrage : version, identifiant, horloges, capteurs
`print_fault_report()`	Rapport d'une faute survenue avant le dernier redémarrage
`connection_poll()`	Terminal (ré)ouvert sur le port COM virtuel : le contenu du tampon USB (message de démarrage...) lui est envoyé automatiquement ; à partir de la deuxième ouverture, rappel de l'identité de la carte et de l'invite.

7.2 console.h — console sur le port COM virtuel

Console sur le port COM virtuel USB de l'ObCP (classe CDC, voir `cdc_port.h`) : émission tamponnée (les messages produits avant l'ouverture du terminal y sont conservés), éditeur de ligne (effacement, rappel de la dernière commande avec la flèche haut, Ctrl-C) et couleurs ANSI désactivables.

Fichiers : `Core/Inc/console.h`, `Core/Src/app/console.c`

Utilise : cdc_port, util, HAL

Utilisé par : app, cli, demo_mqtt, demo_net, demo_scan, demo_web, module_ota, qr_term

Fonctions publiques (27)

`const char *C_OK(void)`

Séquences ANSI de couleur (vides si les couleurs sont désactivées). Exemple : console_printf("%sOK%s\n", `C_OK()`, `C_RST()`) affiche OK en vert.

`const char *C_ERR(void)`

vert

`const char *C_WARN(void)`

rouge

`const char *C_INFO(void)`

jaune

`const char *C_BOLD(void)`

cyan

`const char *C_DIM(void)`

gras

`const char *C_RST(void)`

atténué

`void console_init(void)`

Initialise la console (saisie vide, `printf()` redirigé vers l'USB).

`void console_set_color(bool on)`

Active ou désactive les couleurs ANSI (paramètre `term.color`).

`bool console_color(void)`

Couleurs actives ?

`void console_write(const char *s, size_t n)`

Écrit n octets de texte ; chaque '\n' devient "\r\n" (retour à la ligne du terminal).

`void console_write_raw(const uint8_t *data, size_t n)`

Octets bruts, sans conversion (protocole de transfert de fichier YMODEM).

`void console_puts(const char *s)`

Écrit une chaîne.

`int console_printf(const char *fmt, ...)`

Écrit un texte mis en forme comme `printf()` (384 caractères au plus).

`void console_flush(void)`

Attend que tout soit parti vers le terminal ouvert (2 s au plus).

`void console_pad(const char *s, size_t width)`

Écrit s puis des espaces jusqu'à width colonnes (alignement de tableaux, texte UTF-8).

`void console_event(const char *fmt, ...)`

Message asynchrone (événement de fond : connexion perdue, message MQTT...) : efface la ligne en cours de saisie, affiche le message puis réaffiche l'invite et la saisie, qui n'est pas perdue.

`void console_async_begin(void)`

Début d'un affichage de plusieurs lignes en tâche de fond : efface la ligne en cours de saisie (les messages s'écrivent ensuite normalement, puis `console_async_end()`).

`void console_async_end(void)`

Fin d'un affichage en tâche de fond : réaffiche l'invite et la saisie.

`int console_getc(void)`

Lit un caractère reçu. @retval le caractère, ou -1 si aucun.

`void console_rx_flush(void)`

Oublie les octets reçus non lus.

`bool console_abort(void)`

Vrai si une touche a été frappée (vide le tampon) : arrêt d'une commande longue.

`bool console_readline(char *out, size_t len)`

Éditeur de ligne : traite les caractères reçus (effacement, flèche haut, Ctrl-C...). Non bloquant.

`out` : reçoit la ligne tapée quand l'utilisateur appuie sur Entrée.

`len` : taille de out.

Retour : true quand une ligne complète est disponible.

`void console_prompt(void)`

Affiche l'invite « obcp> » et vide la saisie.

`void console_set_prompt_visible(bool on)`

Indique si l'invite est affichée (false pendant l'exécution d'une commande).

`bool console_new_connection(bool *first)`

Détecte une nouvelle ouverture du port par un terminal.

`first` : reçoit true pour la première ouverture depuis le démarrage.

Retour : true une fois à chaque nouvelle ouverture.

`uint32_t console_take_dropped(void)`

Octets perdus (tampon plein alors qu'aucun terminal n'était ouvert), remis à zéro.

Fonctions internes (6)

Fonction	Rôle
`out_raw()`	Terminal fermé : les octets restent dans le tampon USB (4 Ko) jusqu'à son ouverture, au-delà ils sont perdus (comptés). Terminal ouvert : attente de place, sauf s'il ne lit plus rien pendant `TX_STALL_MS`.
`redraw_input()`	Réaffiche l'invite et la saisie en cours (après un message de fond)
`erase_chars()`	Efface n caractères à gauche du curseur : retour arrière, espace, retour arrière
`backspace()`	Touche retour arrière : retire le dernier caractère de la saisie
`utf8_columns()`	Nombre de colonnes occupées par n octets UTF-8 (les octets de continuation 10xxxxxx ne comptent pas)
`replace_line()`	Remplace la saisie affichée par txt (rappel de la commande précédente)

7.3 demos.h — commandes et tâches de fond des quatre démos

Commandes du terminal et services des quatre démonstrations.

Fichiers : `Core/Inc/demos.h`, `Core/Src/app/cli.c`, `Core/Src/app/demo_scan.c`, `Core/Src/app/demo_net.c`, `Core/Src/app/demo_web.c`, `Core/Src/app/demo_mqtt.c`

cli.c utilise : app, board, cfg_codec, color, config, console, demo_mqtt, demo_net, demo_scan, demo_web, io, module_ota, ncp, ncp_at, ncp_port_obcp, ncp_spi, ncp_wifi, uart_bridge, util, HAL

cli.c est utilisé par : app

demo_scan.c utilise : app, color, config, console, demo_net, io, ncp_wifi, util, HAL

demo_scan.c est utilisé par : cli, demo_web

demo_net.c utilise : app, board, cfg_codec, config, console, demo_mqtt, ncp_net, ncp_wifi, timeutil, util, HAL

demo_net.c est utilisé par : app, cli, demo_mqtt, demo_scan, demo_web, module_ota

demo_web.c utilise : app, cfg_codec, color, config, console, demo_net, demo_scan, http_parse, io, ncp_net, ncp_wifi, qr_term, util, web_page, HAL

demo_web.c est utilisé par : app, cli, module_ota

demo_mqtt.c utilise : app, color, config, console, demo_net, io, ncp_mqtt, util, HAL

demo_mqtt.c est utilisé par : app, cli, demo_net, module_ota

- Démo 1 (`demo_scan.c`) : scanner Wi-Fi, occupation des canaux, suivi RSSI
- Démo 2 (`demo_net.c`) : connexion, IP (DHCP / fixe), DNS, ping, SNTP -> RTC
- Démo 3 (`demo_web.c`) : point d'accès (SoftAP) + serveur web
- Démo 4 (`demo_mqtt.c`) : télémétrie et commandes MQTT

Fonctions publiques (41)

```
int cli_exec(char *line)
```

Exécute une ligne tapée dans le terminal : découpe en mots, recherche de la commande dans la table `s_cmds[]` (`cli.c`), appel de sa fonction.

`line` : ligne tapée (modifiée par le découpage).
Retour : code de retour de la commande.

```
int cmd_scan(int argc, char **argv)
```

Commande « scan » : réseaux visibles (signal, canal, sécurité, norme).

```
int cmd_channels(int argc, char **argv)
```

Commande « channels » : occupation des canaux 1 à 13 et canal conseillé.

```
int cmd_track(int argc, char **argv)
```

Commande « track » : suivi du niveau de signal en direct (LED RVB, CSV).

```
uint32_t scan_age_s(void)
```

Âge du dernier balayage en secondes, `UINT32_MAX` s'il n'y en a pas eu.

```
int scan_run(bool quiet)
```

Balayage complet sans affichage détaillé (page web) ; quiet : sans message.

```
const char *scan_std_label(uint8_t proto)
```

Norme la plus récente annoncée par un réseau : « Wi-Fi 6 », « Wi-Fi 4 »...

```
int cmd_connect(int argc, char **argv)
```

Commande « connect [ssid [mot de passe]] » : connexion à un réseau Wi-Fi.

```
int cmd_disconnect(int argc, char **argv)
```

Commande « disconnect » : déconnexion (reconnexion automatique suspendue).

```
int cmd_status(int argc, char **argv)
```

Commande « status » : état de la liaison Wi-Fi et configuration IP.

```
int cmd_ip(int argc, char **argv)
```

Commande « ip » : adressage DHCP ou adresse fixe.

```
int cmd_dns(int argc, char **argv)
```

Commande « dns <nom> » : résolution d'un nom en adresse IP.

```
int cmd_ping(int argc, char **argv)
```

Commande « ping <hôte> [nombre] [intervalle] » : test d'accessibilité.

```
int cmd_ntp(int argc, char **argv)
```

Commande « ntp [serveur] » : mise à l'heure de la RTC par SNTP.

```
int cmd_time(int argc, char **argv)
```

Commande « time » : heure de la RTC (UTC et locale).

```
void netapp_boot(void)
```

Au démarrage : programme la connexion automatique à la box enregistrée (`wifi.auto`).

`void` **`netapp_poll`**`(void)`

Tâche de fond : événements Wi-Fi, (re)connexion automatique, resynchronisation NTP.

`void` **`netapp_module_restarted`**`(void)`

Le module a redémarré : la liaison voulue sera rétablie.

`bool` **`netapp_sta_ready`**`(void)`

Vrai si la station est connectée et a une adresse IP.

`bool` **`netapp_connecting`**`(void)`

Vrai pendant une tentative de connexion de la station en tâche de fond.

`const char *` **`netapp_ip`**`(void)`

Adresse IP de la station en texte ("" si non connectée).

`int` **`netapp_connect`**`(bool verbose)`

Connexion bloquante avec le SSID et le mot de passe de la configuration.

`verbose` : true : affiche le détail de la liaison obtenue.

`void` **`netapp_request_connect`**`(void)`

Connexion différée (après la réponse HTTP de la page de configuration).

`void` **`netapp_disconnect`**`(void)`

Déconnexion volontaire (plus de reconnexion automatique).

`bool` **`netapp_time_synced`**`(void)`

Vrai si l'heure de la RTC a été réglée par NTP.

`bool` **`netapp_local_time`**`(dt_t *local, int32_t *offset_s)`

Heure locale (fuseau `time.tz` et heure d'été `time.dst` appliqués).

`local` : reçoit la date et l'heure locales.

`offset_s` : reçoit le décalage par rapport à l'UTC en secondes (peut être NULL).

`void` **`netapp_iso_utc`**`(char *out, size_t len)`

Heure UTC au format ISO 8601 (« 2026-10-23T08:15:00Z »), "" si non synchronisée.

`void` **`netapp_print_time`**`(void)`

Affiche la ligne « Heure » des commandes info et time.

`int` **`cmd_web`**`(int argc, char **argv)`

Commande « web start | stop | status » : point d'accès et serveur web.

`void` **`web_boot`**`(void)`

Au démarrage : programme le lancement automatique du serveur web (`web.auto`).

`void` **`web_poll`**`(void)`

Tâche de fond : connexions et requêtes HTTP, événements du point d'accès.

`bool` **`web_running`**`(void)`

Vrai si le serveur web est actif.

`void` **`web_module_restarted`**`(void)`

Le module a redémarré : serveur et point d'accès relancés s'ils étaient actifs.

`int` **`cmd_mqtt`**`(int argc, char **argv)`

Commande « mqtt start | stop | status | pub | sub | trace ».

`void` **`mqttapp_poll`**`(void)`

Tâche de fond : (re)connexion au broker, télémétrie périodique, commandes reçues.

`void mqttapp_on_message(const char *topic, const uint8_t *msg, uint16_t len)`

Rappel du pilote pour chaque message reçu : le message est seulement copié dans une file, traité ensuite par `mqttapp_poll()`.

`void mqttapp_button(uint8_t btn)`

Publie l'appui d'un bouton (IO_BTN_xxx) sur <racine>/event si MQTT est connecté.

`void mqttapp_sta_up(void)`

La station vient d'obtenir son adresse IP : démarrage si `mqtt.auto` = 1.

`void mqttapp_sta_down(void)`

La station a perdu sa liaison : reconnexion au broker programmée.

`void mqttapp_module_restarted(void)`

Le module a redémarré : reconnexion au broker programmée.

`bool mqttapp_connected(void)`

Vrai si le client MQTT est connecté au broker.

Fonctions internes (70)

`cli.c`

Fonction	Rôle
`prefix_nocase()`	Vrai si s commence par p, sans tenir compte des majuscules
`find_cmd()`	Commande de la table par son nom ou son alias, NULL si inconnue
`print_usage()`	Affiche « Usage : <syntaxe> » d'une commande
`on_raw_line()`	Rappel de `ncp_raw()` : chaque ligne de réponse du module est affichée
`at_raw()`	Commande AT brute tapée par l'utilisateur (« at `AT+GMR` » ou « `AT+GMR` »)
`help_demo()`	« help demo » : guide rapide des quatre démos, avec les noms et adresses propres à cette carte (point d'accès, topics MQTT)
`cmd_help()`	« help » : liste des commandes par rubrique ; « help <commande> » : détail
`cmd_info()`	« info » : résumé de l'état de la carte, du module et des services
`cmd_config()`	« config [filtre] » : liste des paramètres (ceux dont la clé contient le filtre)
`set_hint()`	Après « set » : applique tout de suite ce qui peut l'être et dit quand le nouveau réglage sera pris en compte
`cmd_set()`	« set <clé> <valeur> » : modifie un paramètre en RAM (« save » l'enregistre)
`cmd_save()`	« save » : enregistre tous les paramètres en flash
`cmd_auto()`	« auto [web | mqtt | wifi] [on | off] » : raccourci pour les paramètres `web.auto`, `mqtt.auto` et `wifi.auto`, puis résumé de ce qui démarre seul
`cmd_factory()`	« factory [erase] » : valeurs d'usine en RAM (erase : efface aussi la flash)
`cmd_reboot()`	« reboot » : redémarre la carte
`print_led()`	label : début de ligne (« LED RVB : » seul, ou aligné dans « sensors »)
`cmd_led()`	« led [0-100 | on | off] [couleur] » : les arguments peuvent être donnés dans n'importe quel ordre ; v = 101 signifie « rallumer au dernier niveau »
`cmd_pwm()`	« pwm <0-100 | on | off> » : rapport cyclique de la sortie du bornier
`fmt_g()`	Valeur en g avec signe : « +0.98 »
`print_sensors()`	Affiche toutes les mesures : sur plusieurs lignes, ou sur une seule ligne réécrite sur place (« sensors suivi » : \r ramène le curseur en début de ligne)

Fonction	Rôle
`cmd_sensors()`	« sensors [suivi] » : mesures, une fois ou en continu (4 par seconde) jusqu'à l'appui d'une touche ; `app_idle()` garde les services actifs pendant ce temps
`cmd_dfu()`	« dfu » : après confirmation (touche o), redémarre dans le chargeur de ST
`cmd_at()`	« at » : présent dans la table pour l'aide ; la ligne est traitée par `cli_exec()`
`cmd_log()`	« log on | off » : journal du trafic AT avec le module
`cmd_stats()`	« stats » : compteurs des couches SPI et AT du pilote (diagnostic)
`cmd_module()`	« module [reset | ota | boot] » : versions, redémarrage, mise à jour du module

`demo_scan.c`

Fonction	Rôle
`rssi_color()`	Couleur d'affichage d'un niveau : vert >= -60 dBm, jaune >= -75 dBm, rouge
`safe_ssid()`	SSID affichable : caractères de contrôle remplacés, coupé à cols colonnes (UTF-8) avec « … » si nécessaire.
`before()`	Vrai si le réseau a doit être affiché avant b selon le critère choisi
`sort_index()`	Trie les numéros des réseaux (idx) sans déplacer les résultats eux-mêmes
`print_table()`	Tableau des réseaux trouvés, avec barre de qualité et bilan
`find_ssid()`	Niveau (dBm) du point d'accès le plus fort portant ce SSID dans le dernier balayage, et son canal ; 0 si le réseau est absent
`track_timestamp()`	Horodatage d'une mesure : heure locale si la RTC est réglée, sinon temps écoulé

`demo_net.c`

Fonction	Rôle
`fmt_offset()`	Décalage horaire en texte : 7200 -> « UTC+2 », 19800 -> « UTC+5:30 »
`set_rtc_from()`	Règle la RTC (UTC) avec l'heure SNTP lue dans le module
`ntp_start()`	Lance une synchronisation NTP en tâche de fond (le module interroge le serveur ; `ntp_poll()` lit ensuite son heure toutes les secondes)
`ntp_poll()`	Tâche de fond NTP : resynchronisation toutes les 6 h ; pendant une synchronisation, lecture de l'heure du module jusqu'à ce qu'elle soit valide (année >= 2024) ou que 20 s se soient écoulées.
`apply_ip_mode_before()`	Avant la connexion : client DHCP actif (adresse donnée par la box) ou coupé
`apply_static_after()`	Après l'association, en IP fixe : adresse, passerelle, masque et DNS
`print_link_details()`	Détail de la liaison : réseau, canal, signal, sécurité, adresses
`link_up()`	La liaison vient d'être établie (adresse IP obtenue) : mémorise l'adresse, prévient MQTT, lance la synchronisation NTP
`link_down()`	La liaison est perdue ou coupée
`print_hint()`	Conseil adapté à la raison de l'échec de connexion signalée par le module
`on_ping()`	Rappel de `net_ping()` pour chaque écho : affichage et statistiques

`demo_web.c`

Fonction	Rôle
`header()`	En-tête de réponse HTTP : code (200, 404...), type du contenu, longueur. « Connection: close » : une requête par connexion, plus simple à gérer.

Fonction	Rôle
`send_buffered()`	Corps déjà écrit dans `s_resp` + `WEB_HDR_ROOM` : l'en-tête est placé juste devant pour tout envoyer en une seule fois.
`send_text()`	Réponse courte (texte d'erreur...) : copiée après la place de l'en-tête
`send_page()`	Page HTML (10 Ko, en flash) : en-tête puis page, envoyés directement
`scan_json()`	/api/scan : réseaux du dernier balayage en JSON
`auto_get()`	/api/auto?web=0|1&mqtt=0|1 : démarrage automatique, enregistré aussitôt en flash (sur batterie, la page web est souvent la seule interface).
`wifi_post()`	POST /api/wifi (formulaire « Connecter la carte à une box ») : SSID et mot de passe enregistrés dans la configuration (et en flash si save=1), puis connexion différée pour laisser partir la réponse au navigateur.
`handle()`	Aiguillage d'une requête complète selon son chemin : page, API ou 404
`conn_close()`	Ferme la connexion du lien i et libère son tampon
`conn_reset()`	Oublie toutes les connexions (serveur relancé, module redémarré)
`pending_poll()`	Lancement automatique programmé (`web_boot()` ou redémarrage du module)
`ap_events()`	Événements du point d'accès : appareil connecté, adresse attribuée, départ
`print_access()`	Comment rejoindre le serveur : réseau, mot de passe, adresses, QR code
`web_start()`	interactive : commande « web start » (QR code de connexion affiché) ; sinon lancement automatique, plus bref (le QR code remplirait le tampon de la console alors qu'aucun terminal n'est peut-être ouvert).
`web_stop()`	« web stop » : ferme les connexions, arrête le serveur et le point d'accès
`web_status()`	« web status » : état, compteurs, adresses

`demo_mqtt.c`

Fonction	Rôle
`full_topic()`	Topic complet : « telemetry » -> « obcp/A1B2C3/telemetry » ; un topic commençant par '/' est absolu (« /maison/test » -> « maison/test »)
`publish()`	Publie un texte sur <racine>/<sub> et tient les compteurs à jour
`publish_telemetry()`	Publie l'état de la carte (JSON de `app_status_json()`) sur <racine>/telemetry
`ack()`	Réponse à une commande sur <racine>/ack : {"cmd":"led","ok":true,"led":75}
`sanitize()`	Remplace les caractères de contrôle d'un message reçu par des espaces (un message malveillant ne doit pas piloter le terminal)
`handle_cmd()`	Accepte « led=50 », « led 50 », « led:50 » ou {"led":50}
`process_queue()`	Traite les messages de la file : commandes sur <racine>/cmd, simple affichage pour les autres topics souscrits
`mqtt_start()`	Connexion au broker : configuration (identifiant, testament « offline »), connexion, publication de « online » (retenu), abonnement à <racine>/cmd
`mqtt_stop()`	Déconnexion propre : « offline » publié avant de fermer
`join()`	Recolle les mots argv[from..] séparés par des espaces (message de « mqtt pub »)

7.4 config.h — configuration courante et sauvegarde en flash

Configuration courante (RAM) et sauvegarde dans la dernière page de la flash (banque 2, page 255 : 0x080FF800, 2 Ko). Le script d'édition de liens généré par CubeMX n'est pas modifié : le programme (environ 140 Ko) n'atteint jamais cette page, et `config_erase()` refuse de l'effacer si c'était le cas.

Fichiers : `Core/Inc/config.h`, `Core/Src/app/config.c`

Utilise : board, cfg_codec, util, HAL

Utilisé par : app, cli, demo_mqtt, demo_net, demo_scan, demo_web, fault_handlers

Fonctions publiques (9)

`int config_load(void)`

Charge la configuration enregistrée en flash dans `g_cfg` (valeurs d'usine si elle est absente ou invalide).

Retour : `CFG_UNPACK_OK`, `CFG_UNPACK_EMPTY`, `CFG_UNPACK_VERSION` ou `CFG_UNPACK_CRC`.

`int config_save(void)`

Enregistre `g_cfg` en flash (effacement de la page puis écriture par mots de 64 bits, relecture de contrôle).

Retour : 0 si l'enregistrement a réussi, valeur négative sinon.

`int config_erase(void)`

Efface la page de configuration (commande « factory erase »). @retval 0 si OK.

`bool config_stored(void)`

Vrai si une configuration valide est enregistrée en flash.

`bool config_ecc_nmi(void)`

Appelée par `fault_nmi()` : une double erreur ECC détectée en lisant la flash (page de configuration abîmée) déclenche une NMI.

Retour : true si c'était bien une erreur ECC (acquittée), false sinon.

`const char *config_ap_ssid(void)`

SSID du point d'accès : `ap.ssid` ou « ObCP-WiFi-<id> ».

`const char *config_hostname(void)`

Nom d'hôte DHCP : `net.hostname` ou « obcp-<id> ».

`const char *config_mqtt_root(void)`

Racine des topics MQTT : `mqtt.topic` ou « obcp/<id> ».

`const char *config_mqtt_client_id(void)`

Identifiant du client MQTT : « obcp-<id> ».

Fonctions internes (2)

Fonction	Rôle
`flash_bank()`	Banque physique contenant l'adresse (tient compte d'une éventuelle permutation des banques par l'option BFB2).
`flash_page()`	Numéro de la page (2 Ko) dans sa banque

7.5 cfg_codec.h — table des paramètres, valeurs d'usine, stockage

Configuration de la démo : structure, valeurs d'usine, accès par nom de clé (commandes « set » / « config ») et format de stockage en flash (en-tête + CRC32). Indépendant du matériel (testé sur PC).

Fichiers : `Core/Inc/cfg_codec.h`, `Core/Src/app/cfg_codec.c`

Utilise : util

Utilisé par : app, cli, config, demo_net, demo_web

Ajouter un paramètre (exemple : un seuil d'alarme) :

1. ajouter le champ À LA FIN de cfg_t (les configurations déjà enregistrées restent lisibles) ;
2. lui donner sa valeur d'usine dans `cfg_defaults()` (`cfg_codec.c`) ;
3. ajouter une ligne dans la table `s_items[]` (clé, type, bornes, aide) : les commandes set / config le gèrent alors automatiquement ;
4. augmenter `CFG_VERSION` et, dans `cfg_unpack()`, donner sa valeur d'usine au champ quand la configuration lue est plus ancienne.

Fonctions publiques (9)

```
void cfg_defaults(cfg_t *c)
```

Remplit c avec les valeurs d'usine (`app_config.h`).

```
const cfg_item_t *cfg_items(size_t *count)
```

Table de tous les paramètres ; *count reçoit leur nombre.

```
const cfg_item_t *cfg_find(const char *key)
```

Description d'un paramètre par sa clé (majuscules indifférentes), NULL si inconnu.

```
int cfg_set(cfg_t *c, const char *key, const char *value)
```

Modifie un paramètre à partir de sa valeur en texte (commande « set »).

`c` : configuration à modifier.
`key` : clé, par exemple "`wifi.ssid`".
`value` : valeur tapée, contrôlée selon le type et les bornes.
Retour : `CFG_OK`, ou `CFG_ERR_KEY` / `CFG_ERR_VALUE` / `CFG_ERR_RANGE` / `CFG_ERR_CHARS`.

```
void cfg_get(const cfg_t *c, const cfg_item_t *it, char *out, size_t len, bool masked)
```

Valeur d'un paramètre en texte.

`masked` : true : les secrets (mots de passe) sont remplacés par des '*'.

```
const char *cfg_err_str(int err)
```

Texte en français d'un code CFG_ERR_xxx.

```
void cfg_sanitize(cfg_t *c)
```

Remet en ordre une configuration lue (chaînes terminées, valeurs dans leurs bornes).

```
size_t cfg_pack(const cfg_t *c, uint8_t *buf, size_t len)
```

Prépare l'image à écrire en flash : en-tête (magic, version, taille, CRC32) + structure, complétée à un multiple de 8 octets.

Retour : taille de l'image, 0 si le tampon est trop petit.

```
int cfg_unpack(cfg_t *c, const uint8_t *buf, size_t len)
```

Relit une image enregistrée (contrôle du magic, de la version et du CRC).

Retour : `CFG_UNPACK_OK`, `CFG_UNPACK_EMPTY`, `CFG_UNPACK_VERSION` ou `CFG_UNPACK_CRC`.

Fonctions internes (3)

Fonction	Rôle
`get_int()`	Lit un champ entier de 8 ou 16 bits, signé si sa borne minimale est négative
`put_int()`	Écrit un champ entier de 8 ou 16 bits (pendant de `get_int()`)
`str_chars_ok()`	Texte sans guillemet ni caractère de contrôle (il serait recopié dans des commandes AT entre guillemets)

7.6 app_config.h — réglages de compilation et valeurs d'usine

Paramètres de compilation et valeurs par défaut de la démo. Les réglages modifiables à chaud (SSID, broker MQTT, fuseau...) se font depuis le terminal (« set », « save ») : ce fichier ne fixe que les valeurs d'usine.

Fichiers : `Core/Inc/app_config.h`

Constante	Valeur	Signification
APP_NAME	"Démonstrateur Wi-Fi 6 ST67W611M1"	
APP_BOARD	"ObCP (STM32L476RG) + X-NUCLEO-67W61M1"	
APP_VERSION	"1.2.0"	1.2 : démarrage automatique du serveur web, code commenté
DEF_COUNTRY	"EU"	CN, JP, US, EU ou 00
DEF_NTP_SERVER	"pool.ntp.org"	
DEF_TZ_MIN	60	UTC+1 (Paris)
DEF_DST_EU	1	heure d'été européenne
DEF_AP_PASS	"obcpdemo"	8 caractères minimum (WPA2)
DEF_AP_CHANNEL	6	
DEF_AP_IP	{ 10U, 19U, 96U, 1U }	sous-réseau par défaut du firmware
DEF_WEB_PORT	80	
DEF_MQTT_HOST	"broker.emqx.io"	broker public de test (pas de TLS)
DEF_MQTT_PORT	1883	
DEF_MQTT_PERIOD_S	10	
DEF_TRACK_PERIOD_MS	1000	
AP_SSID_PREFIX	"ObCP-WiFi-"	
HOSTNAME_PREFIX	"obcp-"	
MQTT_TOPIC_PREFIX	"obcp/"	
WIFI_RECONNECT_DELAY_MS	10000U	attente entre deux tentatives (doublée à chaque échec, 60 s au plus)
NTP_SYNC_TIMEOUT_MS	20000U	attente de la réponse du serveur NTP
NTP_RESYNC_PERIOD_MS	(6UL * 3600UL * 1000UL)	nouvelle synchro toutes les 6 h

7.7 io.h — LED RVB, sortie PWM, boutons, capteurs

Entrées / sorties de la démo sur l'ObCP : LED RVB (TIM3), sortie PWM du bornier (TIM1), accéléromètre LIS3DH ou LIS3DSH (SPI3), boutons User1 (ObCP) et SW1 (X-NUCLEO), capteur de température interne et VDDA.

Fichiers : `Core/Inc/io.h`, `Core/Src/app/io.c`

Utilise : color, lis3dh, code CubeMX, HAL

Utilisé par : app, cli, demo_mqtt, demo_scan, demo_web, module_ota

Fonctions publiques (13)

```
void io_init(void)
```

Initialise la LED (éteinte, blanche), la sortie PWM (0 %) et l'accéléromètre.

`void io_read(io_sensors_t *s)`

Lit tous les capteurs : température, VDDA, accéléromètre, boutons.

`void io_led_set(uint8_t pct)`

Luminosité de la LED RVB, 0 à 100 % (0 : éteinte).

`uint8_t io_led_get(void)`

Luminosité actuelle de la LED RVB (%).

`void io_led_toggle(void)`

Éteint la LED, ou la rallume au dernier niveau non nul (bouton User1).

`void io_led_color_set(const uint8_t rgb[3])`

Couleur de la LED RVB (rouge, vert, bleu de 0 à 255).

`void io_led_color_get(uint8_t rgb[3])`

Couleur actuelle de la LED RVB.

`void io_pwm_set(uint8_t pct)`

Rapport cyclique de la sortie PWM, 0 à 100 % (1 kHz).

`uint8_t io_pwm_get(void)`

Rapport cyclique actuel de la sortie PWM (%).

`uint8_t io_acc_whoami(void)`

Valeur de `WHO_AM_I` lue au démarrage (0x33 : LIS3DH, 0x3F : LIS3DSH).

`const char *io_acc_name(void)`

Nom du capteur reconnu (« LIS3DH », « LIS3DSH »), ou NULL si absent.

`uint8_t io_buttons_poll(void)`

Anti-rebond des boutons, à appeler souvent (boucle principale).

Retour : masque IO_BTN_xxx des boutons nouvellement appuyés depuis l'appel précédent (0 si aucun).

`uint32_t io_button_count(uint8_t btn)`

Nombre d'appuis comptés depuis le démarrage sur un bouton (IO_BTN_xxx).

Fonctions internes (3)

Fonction	Rôle
`acc_xfer()`	Échange SPI fourni au pilote `lis3dh.c` : CS à 0, n octets échangés (les octets reçus remplacent ceux envoyés dans buf), CS à 1
`led_apply()`	Recalcule et écrit les trois rapports cycliques de la LED RVB (registres de comparaison CCR des canaux de TIM3)
`adc_read()`	Mesure d'un canal de l'ADC (résultat 0 à 4095 sur 12 bits)

7.8 color.h — couleurs de la LED RVB

Couleurs de la LED RVB : noms, notation #RRGGBB, dégradé de qualité du signal, correction de luminosité (gamma). Indépendant du matériel (testé sur PC).

Fichiers : `Core/Inc/color.h`, `Core/Src/app/color.c`

Utilise : aucun autre module

Utilisé par : app, cli, demo_mqtt, demo_scan, demo_web, io

Fonctions publiques (6)

`bool color_parse(const char *s, uint8_t rgb[3])`

Lit une couleur : un nom (rouge, vert, bleu, jaune, cyan, magenta, blanc, orange, violet, rose ; aussi en anglais), « #RRGGBB », « RRGGBB » ou « R,G,B » (0..255).

`s` : texte à lire.
`rgb` : reçoit les composantes rouge, vert, bleu (0 à 255).
Retour : false si le texte n'est pas reconnu.

`const char *color_name(const uint8_t rgb[3])`

Nom de la couleur si elle correspond exactement à un nom connu, sinon NULL.

`void color_hex(const uint8_t rgb[3], char out[8])`

Écrit la couleur en notation « #RRGGBB » (8 octets avec le zéro final).

`const char *color_names(void)`

Liste des noms de couleurs connus (texte d'aide).

`void color_quality(uint8_t q, uint8_t rgb[3])`

Couleur d'une qualité de signal : rouge (0) -> jaune (50) -> vert (100).

`q` : qualité de 0 à 100.
`rgb` : reçoit la couleur.

`uint16_t color_duty(uint8_t c, uint8_t pct, uint16_t period)`

Rapport cyclique PWM d'une composante de couleur : la composante est corrigée (gamma 2 : (c / 255)², pour des nuances qui paraissent régulières à l'œil) puis multipliée par la luminosité pct.

`c` : composante 0 à 255.
`pct` : luminosité 0 à 100 %.
`period` : période du timer (valeur maximale du rapport cyclique).
Retour : valeur à écrire dans le registre de comparaison (0 à period).

Fonctions internes (4)

Fonction	Rôle
`same_nocase()`	Égalité de deux textes sans tenir compte des majuscules
`hex_digit()`	Valeur d'un chiffre hexadécimal ('0'..'9', 'a'..'f', 'A'..'F'), -1 sinon
`parse_hex6()`	« RRGGBB » : trois octets en hexadécimal
`parse_triplet()`	« R,G,B » : trois nombres décimaux de 0 à 255 séparés par des virgules

7.9 lis3dh.h — accéléromètre

Accéléromètre 3 axes de l'ObCP (liaison SPI, mode 3) : LIS3DH ou LIS3DSH, reconnu automatiquement par son registre `WHO_AM_I` (la nomenclature de fabrication cite le LIS3DSH, le programme modèle de l'ObCP pilote un LIS3DH : les deux sont pris en charge). Indépendant du matériel : l'échange SPI est fourni par l'appelant (`io.c` sur la carte, simulation sur PC).

Fichiers : `Core/Inc/lis3dh.h`, `Core/Src/app/lis3dh.c`
Utilise : aucun autre module
Utilisé par : io

Réglage commun : 50 mesures/s, pleine échelle ±2 g, lecture des 3 axes en une rafale (BDU = 1 : octets bas et haut d'une même mesure). LIS3DH : mode haute résolution (12 bits), 1 mg par pas. LIS3DSH : 16 bits, 0,06 mg par pas.

Fonctions publiques (6)

`int lis3dh_init(lis3dh_t *d, lis3dh_xfer_t xfer, void *ctx)`

Reconnaît le capteur (`WHO_AM_I`) et le configure (50 Hz, ±2 g).

`d` : état du capteur, rempli.
`xfer` : fonction d'échange SPI (voir lis3dh_xfer_t).
`ctx` : pointeur libre transmis à xfer.
Retour : 0 si le capteur répond (`WHO_AM_I` = 0x33 ou 0x3F) et accepte la configuration.

`const char *lis3dh_name(const lis3dh_t *d)`

Nom du capteur reconnu (« LIS3DH », « LIS3DSH ») ou NULL.

`int lis3dh_read_mg(lis3dh_t *d, int16_t mg[3])`

Lit les accélérations x, y, z en mg (1000 mg = 1 g = pesanteur).

Retour : 0 si la lecture a réussi.

`int16_t lis3dh_raw_to_mg(uint8_t lo, uint8_t hi)`

Conversion d'une mesure LIS3DH (16 bits justifiés à gauche, 12 utiles) en mg.

`int16_t lis3dsh_raw_to_mg(uint8_t lo, uint8_t hi)`

Conversion d'une mesure LIS3DSH (0,06 mg par pas) en mg.

`void lis3dh_tilt(const int16_t mg[3], int16_t *pitch_x10, int16_t *roll_x10)`

Inclinaisons déduites de la pesanteur, carte immobile : tangage = rotation autour de Y, roulis = rotation autour de X.

`mg` : accélérations x, y, z en mg.
`pitch_x10` : reçoit le tangage en 1/10 de degré.
`roll_x10` : reçoit le roulis en 1/10 de degré.

Fonctions internes (3)

Fonction	Rôle
`write_reg()`	Écriture d'un registre : adresse (bit 7 à 0) puis valeur
`read_regs()`	Lecture de n registres consécutifs. Le LIS3DH n'incrémente l'adresse que si le bit 6 (MS) est à 1 ; sur le LIS3DSH, ce bit fait partie de l'adresse et l'incrément est réglé par `CTRL_REG6` : il ne faut donc pas le mettre.
`write_check()`	Écrit un registre puis le relit : vérifie que l'écriture a été prise en compte

7.10 http_parse.h — analyse des requêtes HTTP

Analyse minimale des requêtes HTTP/1.x pour le serveur web de la démo 3 (indépendant du matériel, testé sur PC).

Fichiers : `Core/Inc/http_parse.h`, `Core/Src/app/http_parse.c`

Utilise : aucun autre module

Utilisé par : demo_web

Une requête HTTP est un texte envoyé par le navigateur :

```
GET /api/led?v=40&c=00FF00 HTTP/1.1      <- méthode, chemin, paramètres
Host: 10.19.96.1                         <- en-têtes, un par ligne
(ligne vide)                             <- fin des en-têtes
[corps, pour un POST]
```

Elle peut arriver en plusieurs morceaux : `http_parse_request()` indique s'il faut attendre la suite.

Fonctions publiques (4)

`int http_parse_request(const char *buf, size_t len, size_t cap, http_req_t *req)`

Analyse les octets reçus d'une connexion.

`buf` : octets reçus (pas forcément terminés par '\0').
`len` : nombre d'octets reçus.
`cap` : taille du tampon (au-delà : `HTTP_PARSE_TOO_BIG`).
`req` : reçoit la requête analysée.
Retour : `HTTP_PARSE_OK`, `HTTP_PARSE_MORE` (attendre la suite), `HTTP_PARSE_BAD` ou `HTTP_PARSE_TOO_BIG`.

`bool http_get_param(const char *query, const char *name, char *out, size_t outlen)`

Cherche name=valeur dans une chaîne « a=1&b=2 » et décode la valeur (%XX et '+' pour l'espace).

Retour : false si le paramètre est absent.

`size_t http_url_decode(char *out, size_t outlen, const char *in, size_t inlen)`

Décodage URL (%XX, '+') ; retourne la longueur écrite (troncature sûre).

`const char *http_status_text(int code)`

Texte d'un code de réponse HTTP : 200 -> « OK », 404 -> « Not Found »...

Fonctions internes (3)

Fonction	Rôle
`find()`	Recherche bornée (memmem n'est pas disponible partout)
`hexval()`	Valeur d'un chiffre hexadécimal, -1 si ce n'en est pas un
`content_length()`	Valeur de l'en-tête Content-Length (insensible à la casse)

7.11 web_page.h — page web embarquée

Page HTML du serveur web de la démo 3 (HTML, CSS et JavaScript dans une seule chaîne de caractères, stockée en flash). Le JavaScript de la page interroge ensuite /api/status... (`demo_web.c`).

Fichiers : `Core/Inc/web_page.h`, `Core/Src/app/web_page.c`

Utilise : aucun autre module

Utilisé par : demo_web

Données publiques : `WEB_PAGE_INDEX[]`, le texte de la page (HTML, style CSS et JavaScript, environ 10 Ko), et `WEB_PAGE_INDEX_LEN`, sa longueur. `send_page()` (`demo_web.c`) l'envoie telle quelle à chaque demande de « / ».

7.12 qr_term.h — QR code dans le terminal

Affichage d'un QR code dans le terminal (demi-blocs UTF-8 et couleurs ANSI), via la bibliothèque QR-Code-generator de Project Nayuki (licence MIT, Core/Src/qrcodegen).

Fichiers : `Core/Inc/qr_term.h`, `Core/Src/app/qr_term.c`

Utilise : console, qrcodegen

Utilisé par : demo_web

Fonctions publiques (1)

`bool qr_print(const char *text)`

Affiche le QR code d'un texte dans le terminal (deux lignes du code par ligne de texte, grâce aux caractères « demi-bloc »).

`text` : texte à coder, par exemple « WIFI:T:WPA;S:ObCP-WiFi-A1B2C3;P:obcpdemo;; » (connexion automatique d'un téléphone au point d'accès).
Retour : false si le texte est trop long ou si les couleurs sont désactivées.

7.13 module_ota.h — mise à jour du firmware du module

Commande « module ota » : mise à jour du firmware du ST67W611M1 avec un fichier .ota envoyé depuis le PC par le terminal (YMODEM) et transmis au module par le lien SPI (`AT+OTASTART` / `AT+OTASEND` / `AT+OTAFIN`). Aucune liaison matérielle supplémentaire n'est nécessaire, mais le module doit déjà exécuter un firmware « mission T01 » 2.0.x.

Fichiers : `Core/Inc/module_ota.h`, `Core/Src/app/module_ota.c`

Utilise : app, console, demo_mqtt, demo_net, demo_web, io, ncp_at, ncp_fwu, ymodem, HAL

Utilisé par : cli

Fonctions publiques (1)

```
int module_ota_run(void)
```

Commande « module ota » : attend un fichier .ota envoyé en YMODEM par le terminal, le contrôle (nom, en-tête, taille), le transmet au module au fil de la réception puis demande son installation.

Retour : 0 si la mise à jour a réussi, sinon un code d'erreur négatif (`APP_ERR_CANCELED`, `APP_ERR_TRANSFER`, `APP_ERR_BADFILE`...).

Fonctions internes (11)

Fonction	Rôle
`io_getc()`	Lecture d'un octet du terminal avec délai d'attente ; les tâches de fond continuent pendant l'attente (`app_idle()`)
`io_putc()`	Envoi d'un octet de contrôle YMODEM (ACK, NAK, 'C'...) au PC
`io_tick()`	Horloge en ms pour le protocole
`contains_nocase()`	Vrai si needle apparaît dans s, sans tenir compte des majuscules
`ends_with_nocase()`	Vrai si s se termine par suffix (« .ota »), sans tenir compte des majuscules
`current_version()`	Version « 2.0.106 » du firmware en place (champ SDK, sinon début de « AT version »)
`version_cmp()`	Comparaison de versions « a.b.c » : <0, 0 ou >0
`on_file()`	Rappel YMODEM, bloc 0 (nom et taille du fichier) : refuse ce qui n'est pas un firmware « mission T01 » (.ota, ni T02 ni mfg), ouvre la session de mise à jour du module
`on_data()`	Rappel YMODEM pour chaque bloc de données : le premier bloc doit contenir un en-tête .ota valide ; tout est transmis au module (ncp_fwu_write) et la progression s'affiche (LED bleue)
`print_instructions()`	Comment envoyer le fichier depuis le PC (Tera Term, script, sz)
`end_of_transfer_cleanup()`	Après un transfert interrompu : laisse l'émetteur se taire et vide la réception

7.14 ymodem.h — réception de fichier YMODEM

Réception d'un fichier par le protocole YMODEM (CRC 16 bits, blocs de 128 ou 1024 octets), indépendante du matériel : les octets passent par les fonctions getc / putc fournies par l'appelant.

Fichiers : `Core/Inc/ymodem.h`, `Core/Src/app/ymodem.c`

Utilise : aucun autre module

Utilisé par : module_ota

Déroulement côté récepteur :

```
récepteur -> 'C'                      répété toutes les 2 s jusqu'au début
émetteur  -> bloc 0 « nom\0taille »   récepteur -> ACK puis 'C'
émetteur  -> blocs 1, 2, ...           récepteur -> ACK (NAK : à renvoyer)
émetteur  -> EOT                       récepteur -> NAK
émetteur  -> EOT                       récepteur -> ACK puis 'C'
émetteur  -> bloc 0 vide (fin du lot) récepteur -> ACK
```

Bloc : SOH (128 o) ou STX (1024 o), n, 255-n, données, CRC-16/XMODEM (polynôme 0x1021, poids fort en premier). Annulation : CAN CAN.

Logiciels émetteurs : Tera Term (Fichier > Transfert > YMODEM > Envoyer, en anglais File > Transfer > YMODEM > Send), ExtraPuTTY, sz --ymodem (lrzsz), `Tools/maj_module_ota.py`.

Fonctions publiques (3)

```
int ymodem_receive(const ymodem_cfg_t *cfg, ymodem_result_t *res)
```

Reçoit un fichier (bloquant) : chaque bloc reçu sans erreur est transmis à cfg->on_data, dans l'ordre.

`cfg` : entrées / sorties et destinataire.

`res` : reçoit le nom, la taille et les compteurs.

Retour : `YM_OK` ou un code YM_ERR_xxx (négatif).

```
uint16_t ymodem_crc16(const uint8_t *data, uint32_t len)
```

CRC-16/XMODEM (polynôme 0x1021) d'un bloc.

```
const char *ymodem_status_str(int st)
```

Texte en français d'un code ymodem_status_t.

Fonctions internes (7)

Fonction	Rôle
`rx()`	Lit un octet (ou -1 après ms millisecondes) par la fonction de l'appelant
`tx()`	Envoie un octet de contrôle par la fonction de l'appelant
`purge()`	Jette les octets reçus jusqu'à un silence de `PURGE_SILENCE_MS` (après une erreur, pour se resynchroniser sur le début du bloc suivant)
`cancel()`	Annule le transfert : plusieurs CAN envoyés à l'émetteur
`read_body()`	Lit la fin d'un bloc dont l'octet de tête (SOH ou STX) vient d'être reçu.
`read_packet()`	Attend le prochain élément : bloc de données, EOT, annulation ou silence
`parse_header_block()`	Bloc 0 : « nom\0taille [date mode ...]\0 »

7.15 bridge_ctl.h — fin de la passerelle « module boot »

Décision de fin de la passerelle « module boot » (port COM virtuel <-> UART du module), indépendante du matériel pour être testée sur PC. `uart_bridge.c` relaie les octets et appelle `bridge_ctl_step()` à chaque tour de boucle.

Fichiers : `Core/Inc/bridge_ctl.h`, `Core/Src/app/bridge_ctl.c`

Utilise : aucun autre module

Utilisé par : uart_bridge

La fin ne dépend pas de l'état du signal DTR seul : QConn_Flash (option do_reset) manipule DTR et RTS pour tenter de redémarrer la puce et peut les laisser à 0 pendant toute la programmation. Fins retenues :

- bouton SW1 (X-NUCLEO) ou User1 (ObCP) maintenu 50 ms ;
- terminal rouvert : une fois que le PC et le module ont échangé des octets, front montant de DTR survenu après le dernier échange, suivi d'au moins 2 s de silence ;

- silence de 2 min après les premiers échanges (effacement complet de la flash du module compris), ou de 15 min sans aucun échange.

Fonctions publiques (2)

```
void bridge_ctl_init(bridge_ctl_t *c, bool dtr, uint32_t now)
```

Début de session.

`c` : état à initialiser.
`dtr` : état du signal DTR (terminal ouvert).
`now` : temps courant (ms).

```
bridge_end_t bridge_ctl_step(bridge_ctl_t *c, uint32_t now, bool dtr, uint32_t in, uint32_t out, bool button)
```

Un tour de boucle de la passerelle : faut-il terminer la session ?

`c` : état de la session.
`now` : temps courant (ms).
`dtr` : état du signal DTR.
`in` : octets relayés PC -> module depuis le tour précédent.
`out` : octets relayés module -> PC depuis le tour précédent.
`button` : un bouton (User1 ou SW1) est appuyé.
Retour : `BRIDGE_END_NONE` tant que la session continue, sinon sa cause de fin.

7.16 timeutil.h — dates, fuseau horaire, heure d'été

Calculs calendaires (UTC <-> heure locale, heure d'été européenne). Indépendant du matériel (testé sur PC).

Fichiers : `Core/Inc/timeutil.h`, `Core/Src/app/timeutil.c`

Utilise : aucun autre module

Utilisé par : board, demo_net

La RTC de la carte reste en temps universel (UTC) ; l'heure locale est calculée à l'affichage : UTC + fuseau (`time.tz`) + 1 h en été (`time.dst`). « Époque » : nombre de secondes écoulées depuis le 1er janvier 1970 à 0 h UTC ; elle rend les calculs (ajout d'un décalage...) très simples.

Fonctions publiques (7)

```
int64_t dt_to_epoch(const dt_t *dt)
```

Date -> secondes depuis le 1er janvier 1970 (« époque »).

```
void epoch_to_dt(int64_t t, dt_t *dt)
```

Secondes depuis 1970 -> date (jour de la semaine compris).

```
uint8_t dt_weekday(uint16_t y, uint8_t m, uint8_t d)
```

Jour de la semaine d'une date : 1 = lundi ... 7 = dimanche.

```
bool eu_dst_active(int64_t utc)
```

Heure d'été de l'Union européenne en vigueur ? Du dernier dimanche de mars 01:00 UTC au dernier dimanche d'octobre 01:00 UTC.

`utc` : instant en secondes depuis 1970.

```
int32_t local_offset_s(int64_t utc, int16_t tz_min, bool dst_eu)
```

Décalage de l'heure locale par rapport à l'UTC.

`utc` : instant en secondes depuis 1970.
`tz_min` : fuseau horaire en minutes (60 pour Paris).
`dst_eu` : true : ajoute 1 h pendant l'heure d'été européenne.
Retour : décalage en secondes (3600 en hiver à Paris, 7200 en été).

```
void dt_format(const dt_t *dt, char *out, size_t len)
```

Écrit la date au format « AAAA-MM-JJ hh:mm:ss ».

`const char *dt_wday_fr(uint8_t wday)`

Nom français du jour de la semaine (1 = « lundi »).

Fonctions internes (3)

Fonction	Rôle
`days_from_civil()`	Nombre de jours entre le 1er janvier 1970 et la date y-m-d (négatif avant)
`civil_from_days()`	Opération inverse : date du jour numéro z depuis le 1er janvier 1970
`last_sunday_31()`	Date (jour du mois) du dernier dimanche d'un mois de 31 jours

7.17 util.h — fonctions utilitaires

Fonctions utilitaires indépendantes du matériel (testées sur PC) : chaînes, analyse et mise en forme de nombres et d'adresses, découpage de la ligne de commande, construction de JSON.

Fichiers : `Core/Inc/util.h`, `Core/Src/app/util.c`

Utilise : aucun autre module

Utilisé par : app, cfg_codec, cli, config, console, demo_mqtt, demo_net, demo_scan, demo_web

Convention : les fonctions qui écrivent du texte reçoivent le tampon de destination (out) et sa taille (len) ; elles ne débordent jamais et terminent toujours le texte par '\0' (texte tronqué si nécessaire).

Fonctions publiques (31)

`uint32_t util_crc32(const void *data, size_t len)`

CRC32 (polynôme de l'Ethernet et du ZIP) : empreinte de la configuration en flash.

`size_t util_strlcpy(char *dst, const char *src, size_t size)`

Copie bornée d'une chaîne (toujours terminée) ; retourne strlen(src).

`bool util_strieq(const char *a, const char *b)`

Comparaison de deux chaînes sans tenir compte des majuscules.

`size_t util_utf8_len(const char *s)`

Nombre de caractères (colonnes) d'un texte UTF-8 (« é » compte pour 1).

`uint32_t util_isqrt(uint64_t v)`

Racine carrée entière (pas de calcul en virgule flottante).

`bool util_parse_ip(const char *s, uint8_t ip[4])`

Lit une adresse IPv4 « a.b.c.d ». @retval false si invalide.

`bool util_parse_long(const char *s, long *out)`

Lit un entier décimal (signe admis). @retval false si invalide.

`bool util_parse_bool(const char *s, bool *out)`

Lit un booléen : 1/0, on/off, oui/non, true/false. @retval false si invalide.

`void util_fmt_ip(char *out, size_t len, const uint8_t ip[4])`

Adresse IPv4 en texte « a.b.c.d ».

`void util_fmt_mac(char *out, size_t len, const uint8_t mac[6])`

Adresse MAC en texte « AA:BB:CC:DD:EE:FF ».

`void util_fmt_fixed(char *out, size_t len, int32_t value, unsigned decimals)`

Nombre à virgule fixe : (245, 1) -> "24.5" ; (-5, 1) -> "-0.5".

`void util_fmt_uptime(char *out, size_t len, uint32_t seconds)`

Durée en secondes en texte « 3 j 04:05:06 ».

`size_t util_json_escape(char *out, size_t len, const char *in)`

Échappement JSON (", \, caractères de contrôle) ; retourne la longueur écrite.

`size_t util_html_escape(char *out, size_t len, const char *in)`

Échappement HTML (<, >, &, ", ') ; retourne la longueur écrite.

`int util_tokenize(char *line, char **argv, int max)`

Découpe une ligne de commande en arguments séparés par des espaces ("..." pour un argument contenant des espaces). Modifie la ligne.

`line` : ligne tapée (les séparateurs sont remplacés par '\0').

`argv` : reçoit un pointeur vers chaque argument.

`max` : taille du tableau argv.

Retour : nombre d'arguments (argc), ou -1 s'il y en a plus que max.

`int util_rssi_quality(int rssi)`

Qualité d'un signal de rssi dBm, de 0 % (-100 dBm) à 100 % (-50 dBm).

`const char *util_rssi_label(int rssi)`

Appréciation d'un signal : « excellent », « bon », « moyen », « faible »...

`void util_bar(char *out, size_t len, int value, int vmin, int vmax, int width)`

Barre de niveau en UTF-8 (▉ et ▒) de width cellules pour value entre vmin et vmax.

`uint16_t util_channel_mhz(uint8_t channel)`

Fréquence centrale d'un canal Wi-Fi de la bande 2,4 GHz (canal 6 -> 2437 MHz).

`size_t util_wifi_qr(char *out, size_t len, const char *ssid, const char *pass)`

Texte d'un QR code de connexion Wi-Fi : WIFI:T:WPA;S:<ssid>;P:<mdp>;; (caractères \ ; , : " échappés).

Retour : longueur écrite, 0 si le tampon est trop petit.

`void jb_init(jbuf_t *j, char *buf, size_t cap)`

Commence un document JSON dans le tampon buf de cap octets.

`void jb_raw(jbuf_t *j, const char *fmt, ...)`

Ajoute du texte brut, mis en forme comme avec `printf()`.

`void jb_key(jbuf_t *j, const char *key)`

Ajoute ,"key": (la virgule seulement si nécessaire).

`void jb_str(jbuf_t *j, const char *s)`

Ajoute une chaîne entre guillemets, échappée.

`void jb_open(jbuf_t *j, char c)`

Ouvre un objet '{' ou un tableau '['.

`void jb_close(jbuf_t *j, char c)`

Ferme un objet '}' ou un tableau ']'.

`void jb_item(jbuf_t *j)`

Séparateur avant un nouvel élément de tableau.

`void jb_kv_str(jbuf_t *j, const char *key, const char *val)`

Ajoute "key":"val" (chaîne échappée).

`void jb_kv_int(jbuf_t *j, const char *key, long val)`

Ajoute "key":val (entier).

`void jb_kv_bool(jbuf_t *j, const char *key, bool val)`

Ajoute "key":true ou "key":false.

`void jb_kv_raw(jbuf_t *j, const char *key, const char *raw)`

Ajoute "key":raw, raw étant déjà écrit en JSON (nombre formaté, null...).

Fonctions internes (3)

Fonction	Rôle
`put()`	Ajoute n octets si la place le permet (le résultat reste toujours terminé).
`qr_put_escaped()`	Ajoute s au texte du QR code en échappant \ ; , : " par un '\'
`jb_put()`	Ajoute n octets au document JSON ; signale le débordement sans dépasser

7.18 board.h — carte ObCP

Carte ObCP (STM32L476RG, SUPMICROTECH-ENSMM, version 2.3) + X-NUCLEO-67W61M1 enfichée sur ses connecteurs Arduino Uno.

Fichiers : `Core/Inc/board.h`, `Core/Src/board.c`

Utilise : cdc_port, main, timeutil, code CubeMX, HAL

Utilisé par : app, cli, config, demo_net, fault_handlers, main

Les broches, horloges et périphériques sont réglés dans STM32CubeMX (`ObCP_WiFi_Demo.ioc`) ; ce fichier (qui n'est pas généré) donne aux modules de la démo des noms indépendants de CubeMX, construits à partir des étiquettes (« User Label ») des broches dans le .ioc. Déplacer une broche dans CubeMX en gardant son étiquette suffit donc à la démo.

```
Signal X-NUCLEO      Arduino   STM32L476   Étiquette CubeMX
------------------   -------   ---------   ----------------------------
SPI_CLK              D13       PA5         NCP_SCK   (SPI1)
SPI_MISO             D12       PA6         NCP_MISO  (SPI1)
SPI_MOSI             D11       PA7         NCP_MOSI  (SPI1)
SPI_CS (réveil)      D10       PB6         NCP_CS    sortie, actif HAUT
SPI_RDY (+ LED1)     D3        PB3         NCP_RDY   entrée
CHIP_EN              D5        PB4         NCP_CHIP_EN
BOOT                 D6        PB10        NCP_BOOT  0 = mode mission
Bouton SW1           D8        PA9         NCP_SW1   entrée, actif bas
UART du module       D0/D1     PA3/PA2     NCP_UART_RX / NCP_UART_TX :

entrées au repos ; l'USART2 ne
les pilote que pendant
« module boot » (uart_bridge.c).
```

Composants de l'ObCP utilisés par la démo :

```
Console (USB device, CDC)  PA11/PA12       port COM virtuel « Périphérique série USB »
LED RVB (via 2N2222)       PC6/PC8/PC9     Led_Red/Led_Green/Led_Blue : TIM3 CH1, CH3, CH4
Sortie PWM (bornier J13)   PB1             PWM_Motor : TIM1_CH3N, MOSFET (charge sur 5 V)
Accéléromètre LIS3DH/LIS3DSH SPI3 PC10/11/12, ACC_CS = PC5 (modèle reconnu au démarrage)
Bouton User1 (SW1 ObCP)    PC13            User1, actif bas (rappel externe R1)
Quartz 8 MHz (HSE)         PH0/PH1         horloge système (80 MHz) et USB (48 MHz)
Quartz 32,768 kHz (LSE)    PC14/PC15       RTC
Module BLE (BlueNRG-M0)    PC2             BLE_Reset : maintenu en reset (non utilisé)
Interrupteur Prog/Run SW4  BOOT0           Prog = chargeur USB DFU de ST
```

Fonctions publiques (16)

```
void board_early_init(void)
```

À appeler en tout premier dans `main()` (USER CODE 1) : saute dans le chargeur USB DFU de ST si la commande « dfu » l'a demandé avant le redémarrage (indicateur conservé dans un registre de la RTC).

```
void board_lse_start(void)
```

Avant `SystemClock_Config()` (USER CODE Init) : démarre le quartz 32 kHz avec l'amplification « medium high » qu'il demande. Si le quartz ne démarre pas, il est arrêté et la configuration de CubeMX décide de la suite (voir `board.c`).

`void board_post_init(void)`

Après les `MX_xxx_Init()` (USER CODE 2) : active les SPI, démarre les PWM, étalonne l'ADC, règle la date de la RTC à la première mise sous tension et calcule l'identifiant de la carte.

`void board_error_blink(void)`

Appelée par `Error_Handler()` : LED rouge clignotante, ne revient pas. 2 éclats : quartz 8 MHz (HSE) ; 3 éclats : quartz 32 kHz (LSE) ; clignotement rapide continu : autre erreur d'initialisation.

`bool board_lse_ok(void)`

Vrai si la RTC est cadencée par le quartz 32 kHz (LSE).

`bool board_hse_ok(void)`

Vrai si la PLL (horloge système) est alimentée par le quartz 8 MHz (HSE).

`const char *board_uid_short(void)`

Identifiant court de la carte : 6 caractères hexadécimaux tirés de l'identifiant unique du STM32 (« A1B2C3 »). Sert à nommer le point d'accès (ObCP-WiFi-A1B2C3) et les topics MQTT (obcp/A1B2C3/...).

`void board_uid_full(char *out, size_t len)`

Identifiant unique complet du STM32 (96 bits) en hexadécimal.

`uint32_t board_image_end(void)`

Adresse de fin du programme en flash (contrôle avant d'effacer la page de configuration).

`void board_reset(void)`

Redémarre la carte (l'USB est d'abord arrêté proprement).

`void board_reboot_to_dfu(void)`

Redémarre dans le chargeur USB DFU (programmation par STM32CubeProgrammer). Ne revient pas (sur la carte).

`void board_fault_save(const board_fault_t *f)`

Enregistre un rapport de faute dans les registres de sauvegarde (`fault_handlers.c`).

`bool board_fault_take(board_fault_t *f)`

Lit et efface le rapport de faute du démarrage précédent.

Retour : true (une seule fois) si une faute a eu lieu avant ce démarrage.

`bool board_rtc_get(dt_t *dt)`

Lit la date et l'heure UTC de la RTC. @retval false si la lecture échoue.

`bool board_rtc_set(const dt_t *dt)`

Règle la date et l'heure UTC de la RTC (après une synchronisation SNTP).

`bool board_rtc_synced(void)`

Vrai si l'heure a été réglée par SNTP depuis la mise sous tension de la RTC.

Fonctions internes (4)

Fonction	Rôle
`backup_write()`	Écrit un registre de sauvegarde. Le domaine de sauvegarde est protégé en écriture (bit DBP de PWR->CR1) : la protection est levée le temps de l'écriture, puis rétablie si elle était en place.
`delay_cycles()`	Boucle d'attente sans SysTick (les interruptions sont coupées)
`rtc_first_date()`	Date de départ de la RTC à la première mise sous tension
`uid_init()`	Identifiant court : empreinte FNV-1a des 96 bits de l'identifiant unique du STM32, réduite à 24 bits (6 chiffres hexadécimaux).

7.19 cdc_port.h — port COM virtuel USB

Port COM virtuel de l'ObCP (classe USB CDC) : tampons circulaires d'émission (4 Ko) et de réception (2 Ko) avec contrôle de flux, détection de l'ouverture du port par le terminal (signal DTR), codage de ligne demandé par le PC (utilisé par la passerelle « module boot »).

Fichiers : `Core/Inc/cdc_port.h`, `Core/Src/cdc_port.c`

Utilise : code CubeMX, bibliothèque USB

Utilisé par : board, console, uart_bridge, usbd_cdc_if, code CubeMX

Émission : les données sont gardées dans le tampon tant que le terminal n'est pas ouvert, puis envoyées à son ouverture (le message de démarrage n'est donc pas perdu). Réception : le point d'accès OUT n'est réarmé que s'il reste au moins 64 octets libres ; sinon l'hôte est mis en attente (NAK) : aucun octet n'est perdu, même pendant un transfert YMODEM.

Fichier non généré. Le code généré par CubeMX l'appelle depuis ses sections « USER CODE » :

```
USB_DEVICE/App/usbd_cdc_if.c   rappels de la classe CDC
                               (cdc_port_itf_*) ;
USB_DEVICE/Target/usbd_conf.c  veille / reprise de l'USB
                               (cdc_port_on_suspend).
```

Fonctions publiques (23)

```
bool cdc_port_configured(void)
```

Vrai si le PC a reconnu (énuméré) la carte et n'est pas en veille.

```
bool cdc_port_open(void)
```

Vrai si un terminal a ouvert le port (signal DTR ou données reçues).

```
uint32_t cdc_port_open_count(void)
```

Nombre d'ouvertures du port par un terminal (+1 à chaque ouverture).

```
size_t cdc_port_write(const uint8_t *data, size_t n)
```

Ajoute des octets au tampon d'émission (non bloquant).

Retour : nombre d'octets acceptés (moins que n si le tampon est plein).

```
size_t cdc_port_tx_free(void)
```

Place libre dans le tampon d'émission (octets).

```
bool cdc_port_tx_empty(void)
```

Vrai si le tampon d'émission est vide et aucun transfert USB en cours.

```
void cdc_port_tx_kick(void)
```

Relance l'émission USB si des octets attendent dans le tampon.

```
void cdc_port_tx_discard(void)
```

Oublie les octets pas encore envoyés.

```
void cdc_port_set_always_send(bool on)
```

Passerelle « module boot » : émission même si DTR est à 0 (l'outil du PC peut manipuler DTR/RTS pendant la programmation).

```
int cdc_port_getc(void)
```

Lit un octet reçu. @retval l'octet (0 à 255), ou -1 si rien n'a été reçu.

```
size_t cdc_port_read(uint8_t *buf, size_t max)
```

Lit jusqu'à max octets reçus. @retval nombre d'octets lus.

```
size_t cdc_port_rx_count(void)
```

Nombre d'octets reçus en attente de lecture.

`void cdc_port_rx_flush(void)`
Oublie les octets reçus non lus.

`uint32_t cdc_port_baudrate(void)`
Débit demandé par le programme du PC (sans effet sur l'USB lui-même).

`uint32_t cdc_port_line_coding_count(void)`
Nombre de changements de débit demandés (+1 à chaque `SET_LINE_CODING`).

`bool cdc_port_dtr(void)`
État du signal DTR (le terminal a ouvert le port).

`void cdc_port_usb_stop(void)`
Arrêt de l'USB avant un redémarrage : le PC voit le port disparaître.

`int8_t cdc_port_itf_init(void)`
Classe CDC démarrée (`usbd_cdc_if.c`, CDC_Init_FS).

`int8_t cdc_port_itf_deinit(void)`
Classe CDC arrêtée (CDC_DeInit_FS).

`int8_t cdc_port_itf_control(uint8_t cmd, uint8_t *pbuf, uint16_t length)`
Requête de contrôle du PC : codage de ligne, DTR/RTS (CDC_Control_FS).

`int8_t cdc_port_itf_receive(uint8_t *buf, uint32_t *len)`
Paquet reçu du PC (CDC_Receive_FS) : copié dans le tampon de réception.

`int8_t cdc_port_itf_tx_complete(uint8_t *buf, uint32_t *len, uint8_t epnum)`
Paquet émis vers le PC (CDC_TransmitCplt_FS) : envoi de la suite.

`void cdc_port_on_suspend(bool suspended)`
Hôte en veille (true) ou réveillé (false), câble débranché (`usbd_conf.c`).

Fonctions internes (6)

Fonction	Rôle
`irq_save()`	Section critique : masque les interruptions et rend l'état précédent
`irq_restore()`	Fin de section critique : rétablit l'état rendu par `irq_save()`
`rx_free()`	Place libre dans le tampon de réception (une case reste toujours vide)
`mark_open()`	Le terminal vient d'ouvrir le port : compté une fois par ouverture
`tx_start_locked()`	Lance un transfert USB vers le PC avec les octets en attente (au plus `TX_CHUNK`, sans dépasser la fin du tableau), si rien n'est déjà en cours. À appeler interruptions masquées ou depuis l'interruption USB.
`rx_rearm_if_room()`	Réautorise la réception d'un paquet USB dès qu'il y a de la place pour lui : tant que ce n'est pas fait, le PC attend (contrôle de flux par NAK).

7.20 uart_bridge.h — passerelle USB ↔ UART du module

Passerelle port COM virtuel (USB) <-> UART du module ST67W611M1 (USART2, PA2/PA3 = D1/D0), pour la première programmation du module avec l'outil QConn_Flash de ST, sans adaptateur USB-UART.

Fichiers : `Core/Inc/uart_bridge.h`, `Core/Src/uart_bridge.c`
Utilise : bridge_ctl, cdc_port, code CubeMX, HAL
Utilisé par : cli

Sur l'ObCP, les broches D0/D1 du connecteur Arduino sont reliées directement à PA3/PA2 : l'UART du module (`UART_TXD` sur D0, `UART_RXD` sur D1) arrive donc sur l'USART2 du STM32. Le débit suit celui que le programme du PC choisit pour le port COM (500 kbit/s puis 2 Mbit/s pour QConn_Flash) ; l'échange se fait par DMA.

Fonctions publiques (1)

```
void uart_bridge_run(bridge_result_t *r)
```

Relaie les octets entre le port COM virtuel et l'UART du module jusqu'à la fin de la session (bouton, terminal rouvert, 2 min sans échange : voir `bridge_ctl.h`). Bloquante. Le module doit avoir été placé en mode téléchargement par l'appelant (`cmd_module()`).

`r` : reçoit le bilan de la session.

Fonctions internes (8)

Fonction	Rôle
`uart_set_baud()`	Change le débit de l'USART2 (après la fin de l'émission en cours) : BRR = fréquence de l'horloge / débit, arrondi.
`uart_start()`	Configure PA2/PA3 en USART2, l'USART2 et les deux canaux DMA
`uart_stop()`	Arrête l'USART2 et le DMA, remet PA2/PA3 en entrées (état de repos)
`tx_pump()`	Octets PC -> module : tampon circulaire vidé par le DMA
`usb_to_uart()`	Copie les octets reçus du PC (port COM virtuel) dans le tampon d'émission
`uart_to_usb()`	Octets module -> PC : lus dans le tampon circulaire du DMA
`button_pressed()`	Bouton User1 (ObCP) ou SW1 (X-NUCLEO) appuyé : fin de session demandée
`led_magenta()`	LED RVB en magenta pendant la session (rouge + bleu)

7.21 ncp_port.h — adaptation du pilote à la carte

Couche d'adaptation matérielle du pilote ST67W6X_Lite.

Fichiers : `Core/Inc/ncp_port.h`, `Core/Src/ncp_port_obcp.c`

Utilise : code CubeMX, HAL

Utilisé par : cli, ncp, ncp_at, ncp_fwu, ncp_mqtt, ncp_net, ncp_spi, ncp_wifi

Le pilote (`ncp_spi.c`, `ncp_at.c`, ...) ne dépend que de ces quelques fonctions : il suffit de les réimplémenter pour porter le pilote sur une autre carte STM32. Sur l'ObCP, elles sont écrites dans `Core/Src/ncp_port_obcp.c` (broches nommées dans `Core/Inc/board.h`) ; pour les tests sur PC, dans `Tests/sim_module.c` (module simulé).

Signaux du ST67W611M1 (tous actifs à l'état HAUT) :

- `CHIP_EN` : alimentation logique / reset du module
- BOOT : 0 = mode mission (fonctionnement normal)
- `SPI_CS` : sélection SPI (et réveil du module)
- `SPI_RDY` : le module demande / accepte une transaction SPI

Fonctions publiques (7)

```
void ncp_port_chip_enable(bool on)
```

Commande la broche `CHIP_EN` (true : module en marche, false : en reset).

```
void ncp_port_boot(bool on)
```

Commande la broche BOOT (true pendant un reset : chargeur en ROM du module).

`void ncp_port_cs(bool active)`

Commande la sélection SPI (true : CS actif, à l'état haut sur ce module).

`bool ncp_port_rdy(void)`

Lit la broche RDY du module.

Retour : true si le module a des données à envoyer ou accepte une transaction.

`int ncp_port_spi_txrx(const uint8_t *tx, uint8_t *rx, uint16_t len)`

Transfert SPI full-duplex : len octets émis depuis tx pendant que len octets sont reçus dans rx (mode 0, poids fort en premier).

`tx` : octets à émettre.

`rx` : tampon des octets reçus (même longueur).

`len` : nombre d'octets.

Retour : 0 si OK, une valeur négative en cas d'erreur.

`uint32_t ncp_port_tick(void)`

Temps écoulé depuis le démarrage, en millisecondes (`HAL_GetTick()`).

`void ncp_port_delay(uint32_t ms)`

Attente de ms millisecondes (attente active : les interruptions, dont celles de l'USB, continuent d'être servies).

7.22 ncp.h — démarrage du module, événements

Pilote « ST67W6X_Lite » du coprocesseur Wi-Fi 6 / BLE ST67W611M1 (carte X-NUCLEO-67W61M1), version bare-metal sans RTOS.

Fichiers : `Core/Inc/ncp.h`, `Core/Src/st67w6x/ncp.c`

Utilise : ncp_at, ncp_mqtt, ncp_net, ncp_port_obcp, ncp_wifi

Utilisé par : app, cli

NCP = « Network Co-Processor » : le module ST67W611M1 contient sa propre pile TCP/IP. Le STM32 ne fait que lui envoyer des commandes texte (« AT ») par le bus SPI et lire ses réponses ; c'est le module qui gère le Wi-Fi, les adresses IP, les connexions TCP et MQTT.

Prérequis : module programmé avec le firmware « mission T01 » (pile TCP/IP dans le module) fourni par X-CUBE-ST67W61 (st67w611m_mission_t01_v2.x.y.bin).

Organisation du pilote (Core/Src/st67w6x/), de la couche la plus basse à la plus haute :

```
ncp_port.h  : adaptation matérielle (GPIO, SPI, temps), écrite pour
              chaque carte (ici Core/Src/ncp_port_obcp.c)
ncp_spi.c   : trames SPI (en-tête 8 octets, magic 0x55AA)
ncp_at.c    : commandes AT, réponses, évènements, données binaires
ncp_wifi.c  : scan, station, point d'accès
ncp_net.c   : IP, DNS, ping, SNTP, serveur TCP
ncp_mqtt.c  : client MQTT du module
ncp_fwu.c   : mise à jour du firmware du module (OTA par le SPI)
ncp.c       : démarrage du module, boucle de traitement (ce fichier)
```

Utilisation type (voir `app_module_start()` dans `app.c`) :

```
ncp_start(&info);   puis, dans la boucle principale :   ncp_poll();
```

Fonctions publiques (5)

`int ncp_start(ncp_info_t *info)`

Démarre le module : impulsion sur `CHIP_EN` (reset), attente du message « ready », lecture des versions et vérification que le firmware est bien un « mission T01 ».

`info` : structure remplie avec les versions lues (obligatoire).

Retour : `AT_OK` (0) si le module est prêt, sinon `NCP_ERR_NOREADY`, `NCP_ERR_ARCH` ou une erreur AT (valeur négative).

`void ncp_poll(void)`

Traite les trames reçues du module (réponses, évènements Wi-Fi, données TCP, messages MQTT). Non bloquante : à appeler le plus souvent possible depuis la boucle principale (`app_idle()`).

`bool ncp_restarted(void)`

Indique que le module a redémarré de lui-même (message « ready » reçu hors de `ncp_start()`), par exemple après une chute de tension.

Retour : true une seule fois par redémarrage (l'indicateur est remis à zéro).

`const char *ncp_err_str(int err)`

Texte en français d'un code d'erreur NCP_ERR_xxx ou NCP_FWU_ERR_xxx.

`err` : code d'erreur (négatif).

Retour : chaîne constante, jamais NULL.

`int ncp_raw(const char *cmd, at_line_cb_t cb, void *ctx, uint32_t timeout_ms)`

Accès bas niveau (commande « at » du terminal) : envoie une ligne AT brute et transmet chaque ligne reçue en réponse à cb.

`cmd` : commande sans CR/LF, par exemple "`AT+GMR`".

`cb` : appelée pour chaque ligne de réponse.

`ctx` : pointeur libre transmis à cb.

`timeout_ms` : délai maximal d'attente de OK / ERROR.

Retour : `AT_OK`, `AT_ERROR` ou `AT_TIMEOUT`.

Fonctions internes (5)

Fonction	Rôle
`urc_router()`	Distribue une ligne non sollicitée (URC) à la couche qui la comprend.
`on_int()`	Rappel de `at_cmd()` : réponse de la forme « +XXX:<entier> » -> *ctx
`copy_field()`	Copie bornée d'une chaîne, toujours terminée par '\0'
`on_gmr()`	Rappel de `AT+GMR` : relève les trois versions affichées par la bannière
`boot_and_sync()`	Attend « ready » puis vérifie que le module répond « OK » à « AT » (3 essais) : le dialogue est alors établi.

7.23 ncp_spi.h — trames SPI

Transport SPI vers le coprocesseur réseau ST67W611M1.

Fichiers : `Core/Inc/ncp_spi.h`, `Core/Src/st67w6x/ncp_spi.c`

Utilise : ncp_port_obcp

Utilisé par : app, cli, ncp_at

Format d'une trame (identique au pilote officiel ST67W6X_Network_Driver, X-CUBE-ST67W61 v1.3.0) : en-tête de 8 octets suivi de la charge utile, longueur totale arrondie au multiple de 4 supérieur.

```
octet 0-1 : magic 0x55AA (petit-boutiste : AA 55)
octet 2-3 : longueur de la charge utile (petit-boutiste)
octet 4   : bits 0-1 version (=0), bit 2 rx_stall, bits 3-7 flags
octet 5   : type de trafic (0 = commandes AT)
octet 6-7 : réservé (=0)
```

Une transaction : CS=1, attente RDY=1, échange full-duplex des en-têtes (+ charges utiles), lecture du reliquat éventuel envoyé par le module, attente RDY=0 (acquittement), CS=0.

Pour un débutant : sur un bus SPI, chaque octet émis fait revenir un octet reçu (échange « full-duplex »). Le STM32 et le module s'envoient donc leurs en-têtes en même temps ; chacun y lit la longueur de ce que l'autre a à transmettre. « Petit-boutiste » (little-endian) : l'octet de poids faible d'un nombre de 16 bits est envoyé en

premier. « rx_stall » : le module signale qu'il n'a plus de place pour recevoir ; la trame émise est alors renvoyée plus tard (`ncp_spi_send()` réessaie).

Fonctions publiques (4)

```
void ncp_spi_init(ncp_spi_rx_cb_t cb)
```

Initialise la couche SPI (compteurs à zéro, CS relâché).

`cb` : fonction appelée pour chaque trame reçue.

```
int ncp_spi_send(const uint8_t *data, uint16_t len)
```

Émet une charge utile vers le module, en réessayant tant qu'il la refuse (rx_stall), 3 s au plus. Les données reçues pendant ces transactions sont transmises au callback.

`data` : octets à émettre (une commande AT, des données TCP...).

`len` : 1 à `NCP_SPI_TX_MAX` octets.

Retour : `NCP_SPI_OK`, ou NCP_SPI_ERR_xxx (valeur négative).

```
int ncp_spi_poll(void)
```

Si le module signale des données (RDY = 1), effectue une transaction de lecture. Non bloquante si RDY = 0.

Retour : 1 si une transaction a eu lieu, 0 sinon, < 0 en cas d'erreur.

```
const ncp_spi_stats_t *ncp_spi_stats(void)
```

Compteurs de la liaison (commande « stats »).

Fonctions internes (3)

Fonction	Rôle
`put_header()`	Écrit l'en-tête de 8 octets d'une trame de len octets (format : `ncp_spi.h`)
`wait_rdy()`	Attend que la broche RDY prenne le niveau voulu ; -1 au bout du délai
`transaction()`	Une transaction complète. data == NULL : lecture seule. *accepted = true si la charge utile émise a été acceptée par le module. Étapes : 1) CS = 1 et attente de RDY = 1 (le module est prêt) ; 2) échange des en-têtes (et de notre charge utile) ; 3) analyse de l'en-tête reçu : magic, longueur, rx_stall ; 4) lecture du reste de la trame du module s'il en reste ; 5) attente de RDY = 0 (acquittement) puis CS = 0 ; 6) remise de la trame reçue au callback (`ncp_at.c`).

7.24 ncp_at.h — moteur de commandes AT

Moteur de commandes AT du ST67W611M1 (firmware « mission T01 »).

Fichiers : `Core/Inc/ncp_at.h`, `Core/Src/st67w6x/ncp_at.c`

Utilise : ncp_port_obcp, ncp_spi

Utilisé par : app, cli, module_ota, ncp, ncp_fwu, ncp_mqtt, ncp_net, ncp_wifi

Les commandes AT sont des lignes de texte envoyées au module (par exemple « `AT+CWJAP="MaBox"`,"motdepasse" » pour se connecter à un réseau). Le module répond par des lignes de texte, puis par « OK » ou « ERROR ».

Le flux AT reçu du module est traité comme un flot d'octets :

- lignes terminées par CR/LF : réponses (OK, ERROR, +XXX:...) ou événements non sollicités (URC : `+CW:`, `+CIP:`, `+IPD:`, `+MQTT:`, ...) ;
- trames « directes » contenant des données binaires : `+CIPRECVDATA`:<n>,<n octets> `+MQTT:SUBRECV`:<lien>,<lg topic>,<lg msg>,"<topic>",<msg>

```
>   (invite d'envoi de données après AT+CIPSEND / AT+MQTTPUBRAW)
```

Les fonctions at_cmd*() sont bloquantes (avec délai de garde) ; pendant l'attente, les URC continuent d'être traités. Un gestionnaire d'URC ne doit JAMAIS envoyer lui-même de commande AT (il ne fait que mettre à jour des drapeaux / variables d'état).

Exemple :

```
at_cmd("AT+CWJAP?", "+CWJAP:", on_line, &ctx, 2000)
```

envoie la commande, appelle `on_line()` pour chaque ligne « +CWJAP:... » reçue (sans le préfixe) et rend `AT_OK` quand le module a répondu OK.

Fonctions publiques (19)

```
void at_init(at_urc_cb_t urc_cb)
```

Initialise le moteur AT et la couche SPI (appelée par `ncp_start()`).

`urc_cb` : fonction recevant les événements non sollicités.

```
void at_set_mqtt_cb(at_mqtt_cb_t cb)
```

Désigne la fonction qui recevra les messages MQTT (`demo_mqtt.c`).

```
void at_set_log(bool on)
```

Active ou coupe le journal du trafic AT (commande « log on|off »).

```
bool at_get_log(void)
```

État du journal du trafic AT.

```
bool at_busy(void)
```

Vrai si une commande AT est en cours (attente de sa réponse).

```
void at_poll(void)
```

Lit les trames en attente dans le module (16 au plus) et traite leur contenu. À appeler régulièrement (boucle principale) ; elle est aussi appelée pendant l'attente des réponses.

```
at_status_t at_wait_ready(uint32_t timeout_ms)
```

Attend la ligne « ready » émise par le module après son démarrage.

`timeout_ms` : délai maximal.

Retour : `AT_OK` ou `AT_TIMEOUT`.

```
void at_clear_ready(void)
```

Oublie un « ready » déjà reçu (avant de redémarrer le module).

```
at_status_t at_cmd(const char *cmd, const char *prefix, at_line_cb_t cb, void *ctx, uint32_t timeout_ms)
```

Envoie une commande (sans CR/LF) et attend OK / ERROR.

`cmd` : commande, par exemple "`AT+CWJAP?`".

`prefix` : préfixe des lignes de réponse à transmettre à cb ("`+CWJAP:`"), "" pour recevoir toutes les lignes, NULL si aucune.

`cb` : rappel recevant ces lignes (peut être NULL).

`ctx` : pointeur libre transmis à cb.

`timeout_ms` : délai maximal ; 0 : la commande est envoyée sans attendre de réponse.

Retour : `AT_OK`, `AT_ERROR`, `AT_TIMEOUT`, `AT_BUSY`, `AT_PARAM` ou `AT_IO`.

```
at_status_t at_cmdf(uint32_t timeout_ms, const char *prefix, at_line_cb_t cb, void *ctx, const char *fmt, ...)
```

Comme `at_cmd()`, la commande étant construite à la manière de `printf()` : at_cmdf(2000, NULL, NULL, NULL, "`AT+CWCOUNTRY=0`,\"%s\"", pays).

`at_status_t **at_cmd_data**(const char *cmd, const uint8_t *data, uint16_t len, uint32_t timeout_ms)`

Commande suivie d'un bloc de données (`AT+CIPSEND=...`, `AT+MQTTPUBRAW=...`) : attend l'invite '>', envoie les données (en plusieurs trames si besoin) puis attend « Recv n bytes » ou « SEND OK ».

`cmd` : commande annonçant les données.
`data` : données à envoyer.
`len` : nombre d'octets (1 ou plus).
`timeout_ms` : délai maximal d'attente de la confirmation.
Retour : `AT_OK`, `AT_SENDFAIL` (données refusées), `AT_TIMEOUT`, `AT_ERROR`...

`void **at_set_recv_sink**(uint8_t *buf, uint16_t max, uint16_t *got)`

Désigne le tampon où ranger les données de la prochaine trame `+CIPRECVDATA` (lecture de données TCP reçues, voir `net_recv()`).

`buf` : tampon de destination.
`max` : taille du tampon.
`got` : reçoit le nombre d'octets copiés.

`const at_stats_t ***at_stats**(void)`

Compteurs du flux AT (commande « stats »).

`const char ***at_status_str**(at_status_t st)`

Texte en français d'un code at_status_t.

`int **at_split**(char *s, char **argv, int max)`

Découpe une liste "a","b",c en champs (séparateur ',', guillemets retirés). Modifie la chaîne en place.

`s` : chaîne à découper (modifiée).
`argv` : tableau recevant un pointeur vers chaque champ.
`max` : taille du tableau argv.
Retour : nombre de champs trouvés.

`bool **at_starts**(const char *s, const char *prefix)`

Vrai si la chaîne s commence par prefix.

`bool **at_parse_ip**(const char *s, uint8_t ip[4])`

Lit une adresse IPv4 « a.b.c.d » (guillemet initial accepté).

Retour : true si l'adresse est valide (ip rempli).

`bool **at_parse_mac**(const char *s, uint8_t mac[6])`

Lit une adresse MAC « aa:bb:cc:dd:ee:ff » (guillemet initial accepté).

Retour : true si l'adresse est valide (mac rempli).

`void **ncp_at_log_hook**(const char *dir, const char *text, size_t len)`

Journal du trafic AT, appelé pour chaque ligne émise ou reçue quand le journal est actif. Fonction « faible » (weak) : la version vide du pilote est remplacée par celle de `app.c`, qui affiche les lignes dans le terminal en masquant les mots de passe.

`dir` : "->" (vers le module) ou "<-" (reçu du module).
`text` : texte de la ligne (pas forcément terminé par '\0').
`len` : longueur du texte.

Fonctions internes (11)

Fonction	Rôle
`log_txt()`	Transmet une ligne au journal s'il est actif
`consume()`	Retire les n premiers octets du tampon (les suivants sont décalés au début)
`stream_starts()`	Vrai si le flux commence par prefix (complet).
`read_uint()`	Lit un entier décimal à partir de *pos ; retourne 1 si un séparateur ',' le suit, 0 s'il manque des octets, -1 si le format est invalide.

Fonction	Rôle
`direct_recvdata()`	`+CIPRECVDATA`:<n>,<données> — retourne 1 si traité, 0 si absent, -2 si incomplet
`direct_subrecv()`	`+MQTT:SUBRECV`:<lien>,<lg topic>,<lg msg>,"<topic>",<message>
`dispatch_line()`	Aiguille une ligne complète : 1) « ready » : le module (re)démarre ; 2) si une commande est en cours : OK / ERROR la terminent, les lignes commençant par le préfixe attendu vont au rappel de la commande ; pour un envoi de données, « Recv n bytes » / « SEND OK » le confirment ; 3) sinon : événement non sollicité, transmis à `urc_router()` (`ncp.c`).
`process_stream()`	Découpe le tampon en éléments complets, dans l'ordre d'arrivée : invite '>', trames de données (`+CIPRECVDATA`, `+MQTT:SUBRECV`) ou lignes. S'arrête dès qu'un élément est incomplet : la suite arrivera dans une prochaine trame SPI.
`on_spi_frame()`	Rappel de la couche SPI : chaque trame reçue est ajoutée au tampon puis analysée.
`start_cmd()`	Prépare l'état de la commande puis l'envoie (texte + CR LF) au module
`end_cmd()`	Fin de la commande : l'état est effacé, une autre peut être envoyée

7.25 ncp_wifi.h — Wi-Fi : scan, station, point d'accès

Fonctions Wi-Fi du ST67W611M1 : scan, station (STA), point d'accès (SoftAP). Toutes les fonctions retournent 0 (`AT_OK`) en cas de succès, une valeur négative sinon (at_status_t ou WIFI_ERR_xxx).

Fichiers : `Core/Inc/ncp_wifi.h`, `Core/Src/st67w6x/ncp_wifi.c`

Utilise : ncp_at, ncp_port_obcp

Utilisé par : app, cli, demo_net, demo_scan, demo_web, ncp

Vocabulaire :

- station (STA) : le module se connecte, comme un téléphone, au réseau d'une box ou d'un routeur ;
- point d'accès (SoftAP) : le module crée son propre réseau Wi-Fi, que rejoint le téléphone (démo 3) ; les deux peuvent fonctionner ensemble ;
- SSID : nom du réseau ; BSSID : adresse MAC du point d'accès ;
- RSSI : puissance reçue en dBm (-40 : excellent, -80 : très faible).

L'état de la station est tenu à jour par les événements `+CW`: du module (`wifi_urc()`) dans la variable globale `g_wifi`, lue par l'application.

Fonctions publiques (17)

```
bool wifi_urc(const char *line)
```

Traite un événement `+CW`: ou une ligne de scan `+CWLAP`: (appelée par `urc_router()` dans `ncp.c`).

Retour : true si la ligne concerne le Wi-Fi (traitée), false sinon.

```
int wifi_init(const char *country)
```

Initialise le Wi-Fi : mode station, pas de reconnexion par le module (c'est l'application qui décide), pays (canaux autorisés).

`country` : "EU", "US", "JP", "CN" ou "00" (mondial).

```
int wifi_scan(bool passive, uint8_t channel, uint32_t timeout_ms)
```

Balayage des réseaux visibles ; résultat dans `g_wifi`.scan[] (`g_wifi`.scan_count réseaux).

`passive` : true : écoute des balises seulement (plus lent, discret).
`channel` : canal 1 à 13, ou 0 pour tous les canaux.
`timeout_ms` : délai maximal.

`int wifi_connect_start(const char *ssid, const char *pass)`

Lance une connexion de la station SANS attendre le résultat : la suite arrive par les événements (`g_wifi`.sta, evt_gotip, evt_reason). Utilisée pour la (re)connexion en tâche de fond (`demo_net.c`).

`ssid` : nom du réseau (le caractère " est interdit).
`pass` : mot de passe ("" ou NULL : réseau ouvert).

`int wifi_connect(const char *ssid, const char *pass, bool wait_ip, uint32_t timeout_ms)`

Connexion de la station avec attente du résultat (bloquante).

`wait_ip` : true : attend aussi l'adresse IP (DHCP).
`timeout_ms` : délai maximal (0 : 25 s).
Retour : `AT_OK`, `WIFI_ERR_REASON` (raison dans `g_wifi`.last_reason), `WIFI_ERR_TIMEOUT`, `WIFI_ERR_NOIP`...

`int wifi_disconnect(void)`

Déconnecte la station du réseau (attend la confirmation, 5 s au plus).

`int wifi_link_info(wifi_link_t *link)`

Lit les caractéristiques de la liaison en cours (SSID, BSSID, canal, RSSI).

Retour : `AT_OK`, ou `WIFI_ERR_NOTCONN` si la station n'est pas connectée.

`int wifi_rssi(int16_t *rssi)`

Lit la puissance reçue (dBm) de la liaison en cours.

`int wifi_conn_details(uint8_t *security, uint8_t *proto_idx)`

Sécurité et norme de la liaison en cours (`AT+CWSTATE?`).

`security` : reçoit WIFI_SEC_xxx.
`proto_idx` : reçoit 1 (b), 2 (g), 3 (n, Wi-Fi 4) ou 4 (ax, Wi-Fi 6).

`int wifi_ap_start(const char *ssid, const char *pass, uint8_t channel, uint8_t max_conn)`

Démarre le point d'accès (mode station + point d'accès ; une connexion de la station en cours est conservée).

`ssid` : nom du réseau créé.
`pass` : "" : réseau ouvert, sinon WPA2 (8 caractères au moins).
`channel` : canal 1 à 13 (imposé par la box si la station est connectée).
`max_conn` : nombre maximal d'appareils (1 à 4).

`int wifi_ap_stop(void)`

Arrête le point d'accès (retour au mode station seule).

`int wifi_ap_clients(uint8_t *count)`

Compte les appareils connectés au point d'accès (`AT+CWLIF`).

`bool wifi_is_connected(void)`

Vrai si la station est associée à un réseau (avec ou sans adresse IP).

`const char *wifi_security_str(uint8_t sec)`

Nom d'un type de sécurité WIFI_SEC_xxx (« WPA2 »...).

`const char *wifi_proto_str(uint8_t proto_mask)`

Normes annoncées par un réseau (masque 1 b, 2 g, 4 n, 8 ax) : « b/g/n/ax »...

`const char *wifi_reason_str(int32_t reason)`

Explication en français d'un code de raison d'échec (`+CW:ERROR`,<raison>).

`const char *wifi_err_str(int err)`

Texte en français d'un code WIFI_ERR_xxx (ou at_status_t).

Fonctions internes (5)

Fonction	Rôle
`parse_scan()`	Décode une ligne de résultat de scan et l'ajoute à `g_wifi`.scan[]

Fonction	Rôle
`has_quote()`	Un guillemet dans un SSID ou un mot de passe casserait la commande AT
`on_cwjap()`	Rappel de `AT+CWJAP?` : remplit la structure wifi_link_t passée dans ctx
`on_cwstate()`	Rappel de `AT+CWSTATE?` : « <état>,"<ssid>",<sécurité>,<norme> »
`on_cwlif()`	Rappel de `AT+CWLIF` : une ligne par appareil connecté, on les compte

7.26 ncp_net.h — réseau IP : DNS, ping, SNTP, TCP

Services réseau du ST67W611M1 en architecture T01 (pile TCP/IP dans le module) : configuration IP, DNS, ping, SNTP, serveur TCP, réception passive (`+IPD` / `AT+CIPRECVDATA`) et émission (`AT+CIPSEND`).

Fichiers : `Core/Inc/ncp_net.h`, `Core/Src/st67w6x/ncp_net.c`

Utilise : ncp_at, ncp_port_obcp

Utilisé par : app, demo_net, demo_web, ncp

Vocabulaire :

- DHCP : la box attribue automatiquement une adresse IP à la carte ;
- DNS : traduit un nom (www.st.com) en adresse IP ;
- SNTP : protocole de mise à l'heure par Internet ;
- lien (link) : une connexion TCP gérée par le module, numérotée de 0 à 4.

Serveur web (démo 3) : le module accepte les connexions des navigateurs (`AT+CIPSERVER`) et annonce chaque connexion (`+CIP`:<lien>,CONNECTED) puis les données arrivées (`+IPD`:<lien>,<n>,...). L'application lit ces données avec `net_recv()`, répond avec `net_send()` et ferme avec `net_close()`.

Fonctions publiques (19)

`bool net_urc(const char *line)`

Traite les événements réseau `+CIP`: (connexion ouverte / fermée), `+IPD`: (données arrivées) et +PING: (appelée par `urc_router()`).

Retour : true si la ligne a été reconnue.

`int net_init(const char *hostname, const uint8_t ap_ip[4])`

Initialise le réseau : IPv4, multi-connexions, réception passive, DHCP (client de la station et serveur du point d'accès), nom d'hôte et adresse du point d'accès.

`hostname` : nom présenté à la box (DHCP).

`ap_ip` : adresse du point d'accès (seuls les 3 premiers octets comptent : il prend toujours l'adresse x.y.z.1).

`int net_sta_ipcfg(net_ipcfg_t *cfg)`

Lit l'adresse IP, la passerelle et le masque de la station.

`int net_ap_ip(uint8_t ip[4])`

Lit l'adresse IP du point d'accès.

`int net_sta_mac(uint8_t mac[6])`

Lit l'adresse MAC de la station.

`int net_dns_servers(char dns[3][16])`

Lit les serveurs DNS utilisés (texte « a.b.c.d », "" si absent).

`int net_set_dhcp(bool on)`

Active (true) ou désactive (false) le client DHCP de la station.

```
int net_set_static(const uint8_t ip[4], const uint8_t gw[4], const uint8_t mask[4])
```

Adresse IP fixe de la station. À appeler une fois la station associée (état CONNECTED), avec le client DHCP désactivé (net_set_dhcp(false)), comme le fait le pilote officiel.

```
int net_set_dns(const uint8_t dns[4])
```

Serveur DNS manuel (dns != NULL) ou automatique / DHCP (dns == NULL).

```
int net_resolve(const char *host, char *ip, size_t len)
```

Résolution DNS : adresse IPv4 d'un nom d'hôte.

- `host` : nom à résoudre (www.st.com).
- `ip` : reçoit l'adresse en texte.
- `len` : taille de ip (8 au moins).

```
int net_ping(const char *host, uint16_t count, uint16_t interval_ms, net_ping_cb_t cb, uint16_t *replies)
```

Ping : envoie count échos ICMP et attend les réponses.

- `host` : nom ou adresse.
- `count` : nombre d'échos (0 : 4).
- `interval_ms` : intervalle entre deux échos (100 à 3500 ms, sinon 1000).
- `cb` : appelée pour chaque écho (peut être NULL).
- `replies` : reçoit le nombre de réponses (peut être NULL).

```
int net_sntp_config(bool enable, int tz_hours, const char *server)
```

Configure le client SNTP du module (mise à l'heure par Internet).

- `enable` : true : client actif.
- `tz_hours` : fuseau appliqué par le module (0 : l'application garde l'UTC).
- `server` : nom du serveur de temps (pool.ntp.org).

```
int net_sntp_time(net_time_t *t)
```

Lit l'heure SNTP du module.

Retour : `AT_OK` si une heure valide a été lue, `AT_ERROR` sinon (pas encore synchronisé...).

```
bool net_parse_asctime(const char *s, net_time_t *t)
```

Décode une date au format « asctime » : "Thu Oct 23 08:45:12 2026".

Retour : true si la date est valide.

```
int net_server_start(uint16_t port, uint8_t max_conn)
```

Démarre le serveur TCP du module (serveur web de la démo 3).

- `port` : port d'écoute (80 pour HTTP).
- `max_conn` : connexions simultanées acceptées.

```
int net_server_stop(void)
```

Arrête le serveur TCP et ferme ses connexions.

```
int net_recv(uint8_t link, uint8_t *buf, uint16_t max, uint16_t *got)
```

Lit des données reçues sur un lien (signalées par `+IPD`).

- `link` : numéro du lien (0 à `NET_LINKS`-1).
- `buf` : tampon de destination.
- `max` : taille du tampon (limitée à `NET_RECV_MAX`).
- `got` : reçoit le nombre d'octets lus.

```
int net_send(uint8_t link, const uint8_t *data, uint32_t len)
```

Envoie len octets sur un lien (découpés en blocs de `NET_SEND_CHUNK`).

```
int net_close(uint8_t link)
```

Ferme la connexion d'un lien.

Fonctions internes (7)

Fonction	Rôle
`on_cipsta()`	Rappel de `AT+CIPSTA?` : lignes « ip:"..." », « gateway:"..." », « netmask:"..." »
`on_cipap()`	Rappel de `AT+CIPAP?` : seule la ligne « ip:"..." » est utile
`on_mac()`	Rappel de `AT+CIPSTAMAC?`
`on_dns()`	Rappel de `AT+CIPDNS?` : « <mode>,"dns1","dns2","dns3" »
`on_domain()`	Rappel de `AT+CIPDOMAIN` : « "a.b.c.d" »
`index_of()`	Rang de tok (3 premières lettres) dans un tableau de noms, -1 si absent
`on_time()`	Rappel de `AT+CIPSNTPTIME?` : date au format asctime

7.27 ncp_mqtt.h — client MQTT du module

Client MQTT intégré au ST67W611M1 (architecture T01).

Fichiers : `Core/Inc/ncp_mqtt.h`, `Core/Src/st67w6x/ncp_mqtt.c`

Utilise : ncp_at, ncp_port_obcp

Utilisé par : app, demo_mqtt, ncp

MQTT en bref : un serveur central, le « broker », relaie des messages. Un client « publie » un message sur un « topic » (un nom hiérarchique, par exemple obcp/A1B2C3/telemetry) ; tous les clients « abonnés » à ce topic le reçoivent. La carte publie sa télémétrie et s'abonne au topic .../cmd pour recevoir des ordres (démo 4, `demo_mqtt.c`). « LWT » (Last Will and Testament) : message que le broker publie à la place de la carte si elle disparaît sans prévenir (« offline »).

Fonctions publiques (7)

```
bool mqtt_urc(const char *line)
```

Traite les événements `+MQTT:CONNECTED` / DISCONNECTED (appelée par `urc_router()`).

Retour : true si la ligne commence par « `+MQTT`: ».

```
void mqtt_set_msg_cb(mqtt_msg_cb_t cb)
```

Désigne la fonction qui recevra les messages des topics souscrits.

```
int mqtt_configure(uint8_t scheme, const char *client_id, const char *user, const char *pass, uint16_t keepalive_s, const char *lwt_topic, const char *lwt_msg, bool lwt_retain)
```

Configure le client MQTT avant la connexion.

`scheme` : 0 : TCP sans authentification, 1 : TCP + identifiant / mot de passe.

`client_id` : identifiant unique du client auprès du broker.

`user, pass identifiant et mot de passe ("" si scheme = 0).` :

`keepalive_s` : période de « signe de vie » envoyée au broker (s).

`lwt_topic` : topic du message de dernière volonté.

`lwt_msg` : message publié par le broker si la carte disparaît.

`lwt_retain` : true : le broker conserve ce message pour les nouveaux abonnés.

```
int mqtt_connect(const char *host, uint16_t port, uint32_t timeout_ms)
```

Connexion au broker (bloquante) : attend l'événement `+MQTT:CONNECTED`.

`host` : adresse du broker (broker.emqx.io).

`port` : port (1883 : MQTT sans chiffrement).

`timeout_ms` : délai maximal (0 : 10 s).

Retour : `AT_OK` ou `MQTT_ERR_NOTCONN`.

```
int mqtt_disconnect(void)
```

Ferme la connexion au broker.

```
int mqtt_subscribe(const char *topic, uint8_t qos)
```

Abonnement à un topic (les jokers + et # sont admis).

`qos` : qualité de service : 0 (au plus une fois) ou 1 (au moins une fois).

```
int mqtt_publish(const char *topic, const uint8_t *msg, uint16_t len, uint8_t qos, bool retain)
```

Publie un message (données binaires admises).

`topic` : topic complet.
`msg` : contenu du message.
`len` : longueur (1 ou plus).
`qos` : qualité de service (0 ou 1).
`retain` : true : le broker garde ce message pour les futurs abonnés.
Retour : `AT_OK`, `MQTT_ERR_NOTCONN`, `AT_SENDFAIL`...

Fonctions internes (1)

Fonction	Rôle
`bad_str()`	Chaîne absente ou contenant un guillemet (interdit dans une commande AT)

7.28 ncp_fwu.h — mise à jour du firmware par SPI

Mise à jour du firmware du ST67W611M1 par le lien SPI (« FWU », firmware update, appelée aussi OTA) : l'image est transmise au module par blocs, le module la vérifie puis redémarre dessus.

Fichiers : `Core/Inc/ncp_fwu.h`, `Core/Src/st67w6x/ncp_fwu.c`

Utilise : ncp_at, ncp_port_obcp

Utilisé par : module_ota

Séquence reprise du pilote officiel ST67W6X_Network_Driver (`W6X_FWU_*`, exemple `ST67W6X_FOTA` de X-CUBE-ST67W61) :

```
AT+OTASTART=0 puis AT+OTASTART=1      ouverture d'une session propre
AT+OTASEND=512   + en-tête de 512 o   (en une seule fois, exigé par le module)
AT+OTASEND=<n>   + données            (blocs multiples de 256 o conseillés)
AT+OTAFIN                              vérification, puis redémarrage
                                       sur le nouveau firmware (« ready »)
```

`AT+OTASEND` suit le schéma de `AT+CIPSEND` : OK, invite '>', données, « Recv n bytes ».

Fichiers acceptés : st67w611m_mission_t01_vX.Y.Z.bin.ota ou .xz.ota (dossier Projects/ST67W6X_Scripts/Binaries/NCP_Binaries de X-CUBE-ST67W61). En-tête de 512 octets :

```
0..15   "QCC74X_OTAVer1.0"
16..19  type d'image "RAW " ou "XZ  " (compressée)
20..23  longueur de l'image qui suit l'en-tête (petit-boutiste)
32..47  nom du module ("QCC_Module_v1.1")
48..63  version ("2.0.106")
64..95  SHA-256 de l'image (vérifiée par le module)
96..511 0xFF
```

L'en-tête ne distingue pas les firmwares T01, T02 et « mfg » : seul le nom du fichier le fait (contrôle effectué par l'application).

Fonctions publiques (8)

```
bool ncp_fwu_parse_header(const uint8_t *h, size_t len, ncp_fwu_header_t *out)
```

Analyse l'en-tête d'une image .ota.

`h` : début du fichier.
`len` : octets disponibles (96 au moins).
`out` : reçoit le type, la longueur, le module, la version, le SHA-256.
Retour : false si ce n'est pas un en-tête .ota valide.

`int ncp_fwu_begin(void)`

Ouvre une session de mise à jour (`AT+OTASTART=0` puis =1).

`int ncp_fwu_write(const uint8_t *data, uint32_t len)`

Transmet une partie de l'image (en-tête compris), dans l'ordre du fichier, au fur et à mesure de sa réception. Les données sont regroupées : 512 octets pour l'en-tête, puis blocs de 2048 octets.

`int ncp_fwu_end(void)`

Transmet le dernier bloc, incomplet, en fin de fichier.

`int ncp_fwu_abort(void)`

Abandonne la session (`AT+OTASTART=0`) ; le firmware en place est conservé.

`int ncp_fwu_finish(uint32_t ready_timeout_ms)`

Demande l'installation (`AT+OTAFIN`) et attend le redémarrage du module sur le nouveau firmware (« ready »).

`ready_timeout_ms` : attente maximale du redémarrage.

Retour : `AT_OK`, `NCP_FWU_ERR_REJECTED` si le module refuse l'image (empreinte SHA-256 fausse...), `AT_TIMEOUT` s'il ne redémarre pas.

`uint32_t ncp_fwu_sent(void)`

Octets effectivement transmis au module depuis `ncp_fwu_begin()`.

`bool ncp_fwu_active(void)`

Vrai si une session de mise à jour est ouverte.

Fonctions internes (2)

Fonction	Rôle
`copy_text()`	Copie un champ texte de l'en-tête (terminé par 0 ou bourrage 0xFF).
`send_buf()`	Envoie les n premiers octets de `s_buf` au module (`AT+OTASEND`)

7.29 Fichiers générés par CubeMX et gestionnaires de fautes

Ces fichiers n'ont pas d'en-tête propre à la démo. Les fichiers générés (`main.c`, `stm32l4xx_it.c`, `usbd_cdc_if.c`, `usbd_conf.c`) ne contiennent, en plus du code de CubeMX, que quelques **points d'accroche** placés dans leurs sections USER CODE : ce sont les seuls liens entre le code généré et celui de la démo. `Tools/verif_cubemx.py` vérifie leur présence après chaque régénération (chapitre 11).

Fichier	Section USER CODE	Point d'accroche	Rôle
`main.c`	Includes	`#include "board.h", "app.h"`	déclarations de la démo
`main.c`	1	`board_early_init();`	saut éventuel dans le chargeur USB DFU
`main.c`	Init	`board_lse_start();`	démarrage du quartz 32 kHz
`main.c`	2	`board_post_init(); app_init();`	fin de l'initialisation, démarrage de l'application
`main.c`	WHILE	`app_loop();`	un tour de la boucle principale
`main.c`	Error_Handler_Debug	`board_error_blink();`	erreur fatale : LED rouge clignotante
`stm32l4xx_it.c`	PFP, NonMaskableInt_IRQn 0	`fault_nmi();`	NMI (erreur ECC de la flash)

Fichier	Section USER CODE	Point d'accroche	Rôle
`usbd_cdc_if.c`	INCLUDE, 3 à 7, 13	`cdc_port_itf_init()`, `_deinit()`, `_control()`, `_receive()`, `cdc_port_write()`, `cdc_port_itf_tx_complete()`	rappels de la classe USB CDC renvoyés vers `cdc_port.c`
`usbd_conf.c`	Includes, 2, 3	`cdc_port_on_suspend(true / false)`	mise en veille et reprise de l'USB

fault_handlers.c — fautes du processeur

Gestionnaires des fautes du Cortex-M4 (HardFault, MemManage, BusFault, UsageFault) et traitement de la NMI.

L'ObCP n'a pas de liaison série « directe » (la console passe par l'USB, qui ne fonctionne plus pendant une faute) : en cas de faute, l'adresse de l'instruction fautive et les registres d'état sont rangés dans les registres de sauvegarde de la RTC, la LED rouge clignote 2 secondes puis la carte redémarre ; le rapport est affiché au démarrage suivant.

Ces gestionnaires doivent lire la pile au moment de la faute : ils sont écrits en assembleur (fonctions « naked ») et ne peuvent donc pas se trouver dans les sections USER CODE de `stm32l4xx_it.c`. Leur génération est désactivée dans CubeMX (Pinout & Configuration > System Core > NVIC > Code generation : « Generate IRQ handler » décoché pour Hard fault, Memory management fault, Prefetch fault / memory access fault et Undefined instruction). La NMI reste générée : `NMI_Handler()` appelle `fault_nmi()` depuis sa section USER CODE.

Fonction	Rôle
`HardFault_Handler()`, `MemManage_Handler()`, `BusFault_Handler()`, `UsageFault_Handler()`	gestionnaires des quatre fautes, en assembleur (« naked ») : récupèrent l'adresse des registres empilés par le processeur et appellent `hardfault_report()` avec le numéro de la faute
`hardfault_report()`	range l'adresse de l'instruction fautive et les registres d'état dans les registres de sauvegarde de la RTC, fait clignoter la LED rouge puis redémarre la carte
`fault_nmi()`	appelée par `NMI_Handler()` : une double erreur ECC en lisant la page de configuration est acquittée et signalée à `config.c` ; toute autre NMI est traitée comme une faute
`red_blink()` (interne)	Clignotement de la LED rouge sans timer ni interruption (fautes)

8. Paramètres de configuration

Les paramètres sont rangés dans `g_cfg` (structure `cfg_t`) et décrits par la table `s_items[]` de `cfg_codec.c` : clé, type, bornes, texte d'aide. Les commandes du terminal les manipulent tous de la même façon, sans code particulier :

Commande	Effet
`config`	liste des paramètres et de leur valeur (mots de passe masqués)
`set <clé> <valeur>`	modifie un paramètre en RAM ; `cfg_set()` contrôle le type et les bornes. La valeur peut contenir des espaces : `set wifi.ssid Ma Box`
`save`	enregistre la configuration en flash (`config_save()`) ; sinon elle est perdue à la mise hors tension
`factory [erase]`	revient aux valeurs d'usine en RAM (`cfg_defaults()`) ; avec `erase`, efface aussi la page de flash
`auto [web|mqtt|wifi] [on|off]`	raccourci pour les trois paramètres de démarrage automatique (3.7)

Dans les valeurs d'usine, `<id>` représente les 6 derniers chiffres hexadécimaux de l'identifiant unique du STM32 (`board_uid_short()`), par exemple `A1B2C3` : chaque carte a ainsi des noms différents.

Clé	Valeur d'usine	Plage	Rôle
`wifi.ssid`	(vide)	0 à 32 caractères	SSID du réseau Wi-Fi (box, routeur...)
`wifi.pass` (secret)	(vide)	0 à 64 caractères	mot de passe Wi-Fi (vide = réseau ouvert)
`wifi.auto`	`1`	0 ou 1 (on / off)	connexion au démarrage + reconnexion automatique
`wifi.country`	`EU`	2 caractères	pays : EU, US, JP, CN ou 00 (mondial)
`ip.dhcp`	`1`	0 ou 1 (on / off)	1 = DHCP, 0 = adresse IP fixe
`ip.addr`	`192.168.1.50`	a.b.c.d	adresse IP fixe
`ip.mask`	`255.255.255.0`	a.b.c.d	masque de sous-réseau (IP fixe)
`ip.gw`	`192.168.1.1`	a.b.c.d	passerelle (IP fixe)
`ip.dns`	`0.0.0.0`	a.b.c.d	serveur DNS (IP fixe ; 0.0.0.0 = passerelle)
`net.hostname`	(vide)	0 à 32 caractères	nom d'hôte DHCP (vide = obcp-<id>)
`time.server`	`pool.ntp.org`	1 à 47 caractères	serveur de temps NTP
`time.tz`	`60`	-720 à 840	fuseau horaire en minutes (60 = Paris)
`time.dst`	`1`	0 ou 1 (on / off)	heure d'été européenne automatique
`time.auto`	`1`	0 ou 1 (on / off)	synchronisation NTP à chaque connexion
`ap.ssid`	(vide)	0 à 32 caractères	SSID du point d'accès (vide = ObCP-WiFi-<id>)
`ap.pass` (secret)	`obcpdemo`	0 à 63 caractères	mot de passe du point d'accès (vide = ouvert, sinon 8 car. min.)
`ap.channel`	`6`	1 à 13	canal du point d'accès (1 à 13)
`ap.ip`	`10.19.96.1`	a.b.c.d	adresse du point d'accès (x.y.z.1)
`web.port`	`80`	1 à 65535	port du serveur web
`web.auto`	`0`	0 ou 1 (on / off)	serveur web et point d'accès lancés à la mise sous tension

Clé	Valeur d'usine	Plage	Rôle
`mqtt.host`	`broker.emqx.io`	1 à 63 caractères	adresse du broker MQTT
`mqtt.port`	`1883`	1 à 65535	port du broker (1883 = MQTT sans TLS)
`mqtt.user`	(vide)	0 à 31 caractères	identifiant MQTT (vide = sans authentification)
`mqtt.pass` (secret)	(vide)	0 à 63 caractères	mot de passe MQTT
`mqtt.topic`	(vide)	0 à 63 caractères	racine des topics (vide = obcp/<id>)
`mqtt.period`	`10`	2 à 3600	période de télémétrie (s)
`mqtt.auto`	`0`	0 ou 1 (on / off)	connexion MQTT automatique après le Wi-Fi
`scan.period`	`1000`	200 à 60000	période du suivi RSSI (ms)
`term.color`	`1`	0 ou 1 (on / off)	couleurs ANSI dans le terminal

Paramètres	Lus par
`wifi.*`	`demo_net.c` (connexion, reconnexion), `ncp_wifi.c` (pays : `wifi_init()`)
`ip.*`	`demo_net.c` (DHCP ou adresse fixe, avant et après la connexion)
`net.hostname`	`app_module_start()` → `net_init()`
`time.*`	`demo_net.c` (synchronisation NTP), `timeutil.c` (heure locale)
`ap.*`, `web.*`	`demo_web.c` (point d'accès, serveur, lancement automatique)
`mqtt.*`	`demo_mqtt.c`
`scan.period`	`demo_scan.c` (commande `track`)
`term.color`	`console.c`

9. Codes d'erreur

Les fonctions qui peuvent échouer rendent 0 en cas de succès et un code négatif sinon. Chaque couche a sa plage de codes : moteur AT (−1 à −6), Wi-Fi (−20 à −24), MQTT (−30), démarrage et mise à jour du module (−40 à −43), application (−50 à −57). `app_err_str()` donne le texte français d'un code, `app_report()` l'affiche en rouge avec le nom de l'opération : *`✗ MQTT : connexion au broker impossible (…)`*.

Code	Nom	Défini dans	Message affiché
0	`AT_OK`	`ncp_at.h`	OK
−1	`AT_ERROR`	`ncp_at.h`	ERROR (refusé par le module)
−2	`AT_TIMEOUT`	`ncp_at.h`	délai dépassé
−3	`AT_BUSY`	`ncp_at.h`	commande déjà en cours
−4	`AT_IO`	`ncp_at.h`	erreur SPI
−5	`AT_PARAM`	`ncp_at.h`	paramètre invalide
−6	`AT_SENDFAIL`	`ncp_at.h`	échec d'envoi des données
−20	`WIFI_ERR_REASON`	`ncp_wifi.h`	raison donnée par le module (`+CW:ERROR,<raison>`), traduite par `wifi_reason_str()` : par exemple « échec de la négociation 4 temps (mot de passe incorrect ?) »
−21	`WIFI_ERR_TIMEOUT`	`ncp_wifi.h`	pas de connexion dans le délai
−22	`WIFI_ERR_NOIP`	`ncp_wifi.h`	associé mais pas d'adresse IP (DHCP)
−23	`WIFI_ERR_CHARS`	`ncp_wifi.h`	SSID vide ou caractère " interdit
−24	`WIFI_ERR_NOTCONN`	`ncp_wifi.h`	station non connectée
−30	`MQTT_ERR_NOTCONN`	`ncp_mqtt.h`	connexion au broker impossible (adresse, port, accès Internet ?)
−40	`NCP_ERR_NOREADY`	`ncp.h`	le module ne répond pas (alimentation, cavaliers JP1/JP2, firmware ?)
−41	`NCP_ERR_ARCH`	`ncp.h`	firmware T02 détecté : programmer le binaire mission T01
−42	`NCP_FWU_ERR_REJECTED`	`ncp_fwu.h`	image refusée par le module (contrôle d'intégrité ou version)
−43	`NCP_FWU_ERR_STATE`	`ncp_fwu.h`	mise à jour non démarrée ou image incomplète
−50	`APP_ERR_NOMODULE`	`app.h`	module Wi-Fi non disponible (voir « module reset »)
−51	`APP_ERR_NOSTA`	`app.h`	station non connectée (commande « connect »)
−52	`APP_ERR_USAGE`	`app.h`	arguments invalides
−53	`APP_ERR_FLASH`	`app.h`	écriture en flash impossible
−54	`APP_ERR_NOTFOUND`	`app.h`	introuvable
−55	`APP_ERR_CANCELED`	`app.h`	annulé
−56	`APP_ERR_TRANSFER`	`app.h`	échec du transfert du fichier
−57	`APP_ERR_BADFILE`	`app.h`	fichier refusé

D'autres codes restent internes à leur module et ne sont pas vus par l'utilisateur : `NCP_SPI_ERR_xxx` (transport SPI, rapportés par le moteur AT comme `AT_IO` ou `AT_TIMEOUT`), `CFG_ERR_xxx` (refus de `cfg_set()`, affichés par la commande `set` avec `cfg_err_str()`), `CFG_UNPACK_xxx` (lecture de la configuration au démarrage) et `HTTP_PARSE_xxx` (analyse d'une requête web).

10. Commandes du terminal et fonctions C

Chaque commande est une ligne de la table `s_cmds[]` de `cli.c`. Le tableau donne, rubrique par rubrique de l'aide, la syntaxe de la commande, la fonction C qui l'exécute et son fichier. Une commande marquée « module » est refusée par `cli_exec()` tant que le module Wi-Fi n'a pas démarré. Une ligne qui commence par `AT` est envoyée telle quelle au module (commande `at`).

Général

Commande	Fonction C	Rôle
`help [commande | demo]` (ou `aide`)	`cmd_help()` cli.c	aide, détail d'une commande, guide rapide des démos
`info`	`cmd_info()` cli.c	état de la carte, du module et des services
`config [filtre]`	`cmd_config()` cli.c	liste des paramètres (ex. « config mqtt »)
`set <clé> <valeur>`	`cmd_set()` cli.c	modifie un paramètre en RAM
`save`	`cmd_save()` cli.c	enregistre les paramètres en flash
`auto [web | mqtt | wifi] [on | off]`	`cmd_auto()` cli.c	services lancés à la mise sous tension (batterie)
`factory [erase]`	`cmd_factory()` cli.c	valeurs d'usine (erase : efface aussi la flash)
`reboot`	`cmd_reboot()` cli.c	redémarre la carte
`led [<0-100> | on | off] [couleur]`	`cmd_led()` cli.c	LED RVB de l'ObCP : luminosité et couleur
`pwm <0-100 | on | off>`	`cmd_pwm()` cli.c	sortie PWM du bornier de l'ObCP (1 kHz)
`sensors [suivi]` (ou `capteurs`)	`cmd_sensors()` cli.c	température, VDDA, accéléromètre, boutons
`time` (ou `heure`)	`cmd_time()` demo_net.c	heure de la RTC (UTC et locale)

Démo 1 – Scanner Wi-Fi et suivi du signal

Commande	Fonction C	Rôle
`scan [passif] [canal <n>] [tri ...] [bssid]` module	`cmd_scan()` demo_scan.c	recherche les réseaux Wi-Fi 2,4 GHz
`channels [appliquer]` (ou `canaux`) module	`cmd_channels()` demo_scan.c	occupation des canaux 1 à 13 et canal conseillé
`track [ssid] [csv]` (ou `suivi`) module	`cmd_track()` demo_scan.c	suivi du niveau de signal en direct (touche = arrêt)

Démo 2 – Connexion et services réseau

Commande	Fonction C	Rôle
`connect [ssid [mot de passe]]` (ou `wifi`) module	`cmd_connect()` demo_net.c	connexion à un réseau Wi-Fi
`disconnect` module	`cmd_disconnect()` demo_net.c	déconnexion du réseau
`status` (ou `etat`) module	`cmd_status()` demo_net.c	état de la liaison Wi-Fi et configuration IP
`ip [dhcp | <ip> <masque> <passerelle> [dns]]` module	`cmd_ip()` demo_net.c	adressage DHCP ou IP fixe
`dns <nom>` module	`cmd_dns()` demo_net.c	résolution d'un nom en adresse IP
`ping <hôte> [nombre] [intervalle ms]` module	`cmd_ping()` demo_net.c	test d'accessibilité (ICMP)
`ntp [serveur]` module	`cmd_ntp()` demo_net.c	mise à l'heure de la RTC par SNTP

Démo 3 – Serveur web en point d'accès (SoftAP)

Commande	Fonction C	Rôle
`web start | stop | status` module	`cmd_web()` demo_web.c	point d'accès Wi-Fi + serveur web embarqué

Démo 4 – Télémétrie MQTT

Commande	Fonction C	Rôle
`mqtt start | stop | status | pub | sub | trace` module	`cmd_mqtt()` demo_mqtt.c	client MQTT : télémétrie et commandes à distance

Outils avancés

Commande	Fonction C	Rôle
`at <commande>` module	`cmd_at()` cli.c	envoie une commande AT brute (ex. at `AT+GMR`)
`log on | off`	`cmd_log()` cli.c	affiche le trafic AT avec le module
`stats`	`cmd_stats()` cli.c	compteurs SPI et AT
`dfu`	`cmd_dfu()` cli.c	redémarre l'ObCP dans le chargeur USB DFU (programmation)
`module [reset | ota | boot]`	`cmd_module()` cli.c	versions du firmware, redémarrage, mise à jour

11. Modifier le programme

11.1 Règles à respecter

1. **Ne jamais bloquer la boucle principale** plus de quelques dizaines de millisecondes. Une attente longue s'écrit comme une machine à états dans une tâche de fond (4.4), ou en appelant `app_idle()` pendant l'attente, comme `cmd_track()`.
2. **Jamais de commande AT dans un rappel** (`xxx_urc()`, `mqttapp_on_message()`) : lever un drapeau ou copier la donnée, et traiter dans la tâche de fond.
3. **Interruptions** : rien de long, pas d'affichage ; ranger la donnée et laisser la boucle principale faire le reste.
4. **Fichiers générés** : n'écrire qu'entre `USER CODE BEGIN` et `USER CODE END`. Le code nouveau va dans de nouveaux fichiers (`Core/Src/app/`, `Core/Inc/`).
5. **Codes de retour** : tester ceux des fonctions du pilote et les afficher avec `app_report()`.
6. **Textes** : toujours avec la taille du tampon (`snprintf()`, `util_strlcpy()`), jamais `strcpy()` ni `sprintf()`. Pas de guillemet dans un texte recopié dans une commande AT (SSID, mot de passe, topic) : `cfg_set()` les refuse déjà.
7. **Messages en tâche de fond** : `console_event()` (une ligne) ou `console_async_begin()` / `console_async_end()` (plusieurs lignes), pour ne pas abîmer la ligne en cours de saisie.
8. **Après chaque modification** : compiler sans avertissement, lancer les tests sur PC (`make -C Tests`, chapitre 12), puis essayer sur la carte avec `log on` pour voir le trafic AT.

11.2 Ajouter une commande

Exemple : une commande `bonjour [nom]` dans la rubrique « Général » de l'aide. Tout se passe dans `cli.c` :

```
/* 1. le prototype, avec les autres en tête de cli.c */
static int cmd_bonjour(int argc, char **argv);

/* 2. une ligne dans la table s_cmds[], à l'endroit voulu dans l'aide :
 *    nom, alias, fonction, rubrique (0 = Général), module requis,
 *    syntaxe, aide en une ligne, aide détaillée (help bonjour) ou NULL */
{ "bonjour", "salut", cmd_bonjour, 0U, false, "bonjour [nom]", "affiche un salut", NULL },

/* 3. la fonction : argc / argv comme main() ; argv[0] = "bonjour" */
static int cmd_bonjour(int argc, char **argv)
{
  console_printf("Bonjour %s !\n", (argc >= 2) ? argv[1] : "ObCP");
  return 0;                 /* ou un code d'erreur négatif (chapitre 9) */
}
```

La commande apparaît aussitôt dans `help`. Les rubriques sont celles de `s_groups[]` (0 = Général à 5 = Outils avancés) ; mettre `true` dans le champ « module requis » si la commande utilise le module Wi-Fi. Pour une commande de démo, déclarer plutôt la fonction (sans `static`) dans `demos.h` et l'écrire dans le fichier de la démo, comme `cmd_scan()` dans `demo_scan.c`. En cas d'erreur, la fonction l'affiche elle-même (`app_report()`) puis rend le code négatif.

11.3 Ajouter un paramètre de configuration

Exemple : un seuil d'alarme de température `alarm.temp`, entier de −40 à 85 °C, valeur d'usine 30. Quatre endroits sont à compléter (la recette est rappelée en tête de `cfg_codec.h`) :

```
/* 1. cfg_codec.h : le champ, TOUJOURS à la fin de cfg_t */
  uint8_t  web_auto;          /* (dernier champ de la version 2) */
  int16_t  alarm_temp;        /* seuil d'alarme (°C) */
} cfg_t;

/* 2. cfg_codec.c, dans cfg_defaults() : la valeur d'usine */
  c->alarm_temp = 30;

/* 3. cfg_codec.c, table s_items[] : clé, type, champ, min, max, secret, aide */
  { "alarm.temp", CFG_T_INT, FIELD(alarm_temp), -40, 85, false, "seuil d'alarme (°C)" },

/* 4. cfg_codec.h : nouvelle version du format ...                    */
#define CFG_VERSION      3U
/*    ... et, dans cfg_unpack(), la valeur d'usine pour une configuration plus ancienne */
  if (h.version < 3U)
  {
    c->alarm_temp = 30;                  /* champ ajouté dans la version 3 */
  }
```

Les commandes `set alarm.temp 35`, `config` et `save` fonctionnent alors sans autre code, et le programme lit la valeur dans `g_cfg.alarm_temp`. Types possibles : `CFG_T_INT` (champ de 8 bits `uint8_t`, ou de 16 bits : `int16_t` si le minimum est négatif, `uint16_t` sinon), `CFG_T_BOOL` (`uint8_t`), `CFG_T_STR` (tableau de `char` de longueur maximale + 1) et `CFG_T_IP` (`uint8_t[4]`).

> **Attention :** ajouter le champ **à la fin** de la structure permet de relire les configurations déjà enregistrées : `cfg_unpack()` part des valeurs d'usine puis recopie les anciens octets. L'étape 4 est indispensable quand le nouveau champ occupe un octet de « bourrage » de la structure (cas de `web_auto` en version 2) : sans elle, il prendrait la valeur, quelconque, de cet octet.
>
> Les tests de la configuration (`Tests/test_app.c`) vérifient le numéro de version du format : les adapter à la version 3 (relecture d'une configuration enregistrée en version 2).

11.4 Ajouter une adresse au serveur web

Les requêtes sont aiguillées par `handle()` dans `demo_web.c`, selon leur chemin (`req->path`). Pour ajouter `/api/bonjour`, insérer avant la réponse « 404 » :

```
  if (strcmp(req->path, "/api/bonjour") == 0)        /* /api/bonjour?nom=Paul */
  {
    char   nom[24] = "ObCP";
    char   msg[40];
    jbuf_t j;
    (void)http_get_param(req->query, "nom", nom, sizeof(nom));
    (void)snprintf(msg, sizeof(msg), "Bonjour %s", nom);
    jb_init(&j, body, cap);
    jb_open(&j, '{');
    jb_kv_str(&j, "message", msg);               /* texte échappé : JSON valide */
    jb_close(&j, '}');
    return send_buffered(link, 200, "application/json", j.pos, head);
  }
```

`req->query` contient les paramètres après le `?` d'un GET, ou le corps d'un formulaire envoyé en POST ; `http_get_param()` en extrait un, décodé. La réponse est préparée dans `body` (5 Ko moins la place de l'en-tête) puis envoyée par `send_buffered()`. `handle()` s'exécute dans la tâche de fond `web_poll()` : la réponse doit être immédiate. Les fonctions `jb_xxx()` de `util.h` construisent le JSON et échappent les caractères spéciaux (guillemets…) : la réponse reste valide quel que soit le texte saisi.

Côté page, `web_page.c` contient le HTML, le style et le JavaScript dans une seule chaîne C : une requête s'écrit `fetch('/api/bonjour?nom=Paul').then(r => r.json())`. Les guillemets doubles y sont écrits `\"` ; `make -C Tests check-web` vérifie la syntaxe du JavaScript.

11.5 Publier une nouvelle mesure

L'état de la carte est mis en forme en un seul endroit, `app_status_json()` dans `app.c`. Une valeur ajoutée là apparaît **à la fois** dans `/api/status` (lu toutes les 1,5 s par la page web) et dans la télémétrie MQTT (`<racine>/telemetry`) :

```
jb_kv_int(&j, "pwm", io_pwm_get());
jb_kv_int(&j, "ma_mesure", (long)ma_mesure_lire());      /* nouvelle valeur */
jb_kv_bool(&j, "user", s.user);
```

Le JSON fait environ 310 octets ; la télémétrie MQTT est préparée dans un tampon de 512 octets (`publish_telemetry()`, `demo_mqtt.c`). Si le texte ne tient plus, `app_status_json()` rend 0 et rien n'est publié : agrandir alors ce tampon. Pour afficher la valeur sur la page web, ajouter un élément dans `web_page.c` et le remplir dans la fonction JavaScript qui traite `/api/status`.

11.6 Ajouter un périphérique avec STM32CubeMX

1. Ouvrir `ObCP_WiFi_Demo.ioc` dans STM32CubeMX (ou STM32CubeIDE), activer le périphérique, choisir ses broches et leur donner une **étiquette** (« User Label »).
2. Générer le code (Project > Generate Code). Les sections USER CODE et les fichiers de la démo sont conservés.
3. Écrire le pilote dans de nouveaux fichiers (par exemple `Core/Src/app/capteur.c` et `Core/Inc/capteur.h`), qui utilisent la poignée générée (`hspi3`, `htim3`… déclarées dans `spi.h`, `tim.h`) et les étiquettes (`main.h`).
4. Appeler son initialisation depuis `app_init()` (ou `board_post_init()` pour un réglage matériel), sa lecture depuis une commande ou une tâche de fond.
5. Lancer `python3 Tools/verif_cubemx.py` : il vérifie les 20 points d'accroche, les réglages dont la démo dépend et les gestionnaires de faute ; puis compiler et lancer les tests.

> **Attention :** à ne pas changer dans CubeMX : la génération des gestionnaires HardFault, MemManage, BusFault et UsageFault doit rester **désactivée** (System Core > NVIC > Code generation), car `fault_handlers.c` les fournit ; l'USART2 doit rester **inactif** (PA2 et PA3 en entrées), car `uart_bridge.c` ne l'utilise que pendant `module boot` ; PB3 est la broche RDY du module (la trace SWO n'est pas disponible).
>
> La procédure complète de modification et de régénération est décrite dans le guide d'utilisation (4.4).

12. Tests sur PC et outils

12.1 Le principe : un module simulé

Le pilote et la plus grande partie de l'application sont testés **sur PC**, sans carte : `Tests/sim_module.c` simule le ST67W611M1 et réalise les sept fonctions de `ncp_port.h`. Le pilote dialogue avec lui exactement comme avec le vrai module (trames SPI, broches CS et RDY, refus `rx_stall`) ; le simulateur répond aux commandes AT comme le firmware T01 (réseaux, box, serveur web, broker MQTT, mise à jour). Une fausse HAL (`Tests/host`) remplace les périphériques du STM32 : horloge, broches, flash, console.

Les tests sont compilés avec `gcc` et les contrôles d'erreurs mémoire activés (`-fsanitize=address,undefined`) : un débordement de tableau ou un accès hors limites arrête le test et indique la ligne fautive.

12.2 Les programmes de test

Test	Ce qu'il vérifie	Vérifications
`test_driver`	le pilote face au module simulé : trames SPI, commandes AT, événements, données binaires, mise à jour du module	193
`test_app`	modules indépendants du matériel : calendrier et heure d'été, utilitaires, HTTP, configuration (dont la relecture d'une version 1), YMODEM, couleurs, accéléromètre	325
`test_usb`	le port COM virtuel : tampons circulaires, contrôle de flux, ouverture du terminal (DTR)	115
`test_sim`	l'application complète : les commandes des quatre démos sont « tapées » et les réponses vérifiées ; journal dans `Tests/build/sim_transcript.txt`	284
`test_boot`	le démarrage automatique avec une configuration déjà enregistrée : box joignable, sans box, box injoignable	63 + 11 + 13
`check-web`	le JSON produit par le serveur et la télémétrie, la syntaxe du JavaScript de la page	—
`check_ota_py.py`	le script `Tools/maj_module_ota.py`, de bout en bout, face à l'application sur un pseudo-terminal	—
`check-cubemx`	`Tools/verif_cubemx.py` : points d'accroche dans le code généré	—

12.3 Lancer les tests

Sous Linux ou macOS (ou Windows avec WSL ou MSYS2), depuis le dossier du projet :

```
make -C Tests            # compile et lance tous les tests
make -C Tests verbose    # test du pilote avec le journal du trafic AT simulé
make                     # programme de la carte, sans CubeIDE (arm-none-eabi-gcc dans le PATH)
```

Chaque programme affiche le nombre de vérifications réussies et la liste des échecs (fichier et ligne). Un nouveau test s'ajoute dans le fichier correspondant avec la macro `CHECK(condition)` ou `EXPECT(texte attendu)`.

12.4 Les outils du dossier Tools

Script	Rôle
`verif_cubemx.py`	après une régénération par CubeMX : points d'accroche USER CODE, périphériques et étiquettes attendus, gestionnaires de faute, fichiers de la démo non déclarés comme générés

Script	Rôle
`maj_module_ota.py`	mise à jour du firmware du module depuis le PC : lance `module ota` et envoie le fichier .ota par YMODEM (pyserial)
`rssi_logger.py`	démo 1 : lance `track … csv`, enregistre le RSSI dans un fichier CSV et trace la courbe (pyserial, matplotlib)

13. Glossaire

Terme	Signification
AT (commande)	ligne de texte commençant par `AT+`, envoyée au module pour lui demander une action ; il répond `OK` ou `ERROR`
.bss, .data, .text, .rodata	sections de la mémoire : variables mises à zéro, variables initialisées, code, constantes (5.1)
Broker	serveur MQTT qui relaie les messages entre les clients
CDC (classe USB)	Communication Device Class : le PC voit la carte comme un port série (port COM virtuel)
CRC	code de contrôle calculé sur des données : s'il ne correspond plus, les données ont été altérées
CS, RDY	broches du lien SPI avec le module : demande d'échange (STM32), disponibilité (module)
DFU	Device Firmware Upgrade : chargeur USB intégré au STM32, pour programmer la flash sans sonde
DHCP	protocole par lequel la box attribue automatiquement une adresse IP
DMA	contrôleur qui copie des données entre un périphérique et la mémoire sans le processeur
DNS	service qui traduit un nom (`www.st.com`) en adresse IP
Drapeau	variable booléenne levée par une partie du code, lue puis remise à faux par une autre
Flash, RAM	mémoire du programme, conservée hors tension ; mémoire des variables, effacée hors tension
Full-duplex	émission et réception simultanées (bus SPI)
HAL	Hardware Abstraction Layer : bibliothèque de ST qui pilote les périphériques (`HAL_GPIO_WritePin()`…)
HSE, LSE	quartz externes : 8 MHz (horloge système) et 32,768 kHz (RTC)
Interruption	suspension du programme pour exécuter un court gestionnaire en réponse à un événement matériel
Lien (link)	une connexion TCP gérée par le module, numérotée de 0 à 4
LWT	Last Will and Testament : message que le broker publie si le client disparaît sans prévenir
Machine à états	code qui mémorise l'étape d'une opération longue et avance d'un pas à chaque appel
MQTT	protocole de messagerie par publication et abonnement, très utilisé pour les objets connectés
NCP	Network Co-Processor : coprocesseur qui porte la pile réseau (le ST67W611M1)
Petit-boutiste	little-endian : l'octet de poids faible d'un nombre est transmis ou rangé en premier
PWM	modulation de largeur d'impulsion : la puissance moyenne dépend du rapport cyclique
Rappel (callback)	fonction passée en paramètre à une autre, qui l'appellera plus tard (ligne reçue, message…)
RSSI	puissance du signal reçu, en dBm (−40 : excellent, −80 : très faible)
RTC	horloge temps réel : date et heure, entretenue par le quartz 32 kHz
RTOS	système d'exploitation temps réel (non utilisé ici)
SNTP	protocole simple de mise à l'heure par Internet
SoftAP	point d'accès Wi-Fi créé par le module (démo 3)

Terme	Signification
SPI	bus série synchrone : une horloge, deux lignes de données, une sélection
Station (STA)	le module connecté au réseau d'une box, comme un téléphone
`static`	mot-clé C : fonction ou variable visible seulement dans son fichier ; variable locale conservée entre deux appels
Super-boucle	programme organisé en une boucle sans fin qui appelle les tâches à tour de rôle
Tampon circulaire	tableau où l'écrivain et le lecteur avancent chacun leur index, qui revient au début en fin de tableau
Tâche de fond	fonction `xxx_poll()` appelée à chaque tour de boucle, qui fait avancer un service sans bloquer
Topic	nom hiérarchique d'un canal MQTT (`obcp/A1B2C3/telemetry`)
URC	Unsolicited Result Code : événement envoyé spontanément par le module
USER CODE	sections des fichiers générés réservées au code de l'utilisateur, conservées à la régénération
`volatile`	mot-clé C : la variable peut changer hors du code qui la lit ; le compilateur la relit à chaque accès
YMODEM	protocole de transfert de fichier sur liaison série, par blocs contrôlés par CRC

Annexe A – Index des fonctions publiques

Les 295 fonctions publiques par ordre alphabétique, avec le module (fichier .h) qui les déclare et dont la fiche figure au chapitre 7. Les fonctions internes (`static`) se trouvent dans la fiche de leur module.

`app_autostart_summary` app
`app_err_str` app
`app_idle` app
`app_init` app
`app_loop` app
`app_module_start` app
`app_print_module_info` app
`app_report` app
`app_status_json` app
`at_busy` ncp_at
`at_clear_ready` ncp_at
`at_cmd` ncp_at
`at_cmd_data` ncp_at
`at_cmdf` ncp_at
`at_get_log` ncp_at
`at_init` ncp_at
`at_parse_ip` ncp_at
`at_parse_mac` ncp_at
`at_poll` ncp_at
`at_set_log` ncp_at
`at_set_mqtt_cb` ncp_at
`at_set_recv_sink` ncp_at
`at_split` ncp_at
`at_starts` ncp_at
`at_stats` ncp_at
`at_status_str` ncp_at
`at_wait_ready` ncp_at
`board_early_init` board
`board_error_blink` board
`board_fault_save` board
`board_fault_take` board
`board_hse_ok` board
`board_image_end` board
`board_lse_ok` board
`board_lse_start` board
`board_post_init` board
`board_reboot_to_dfu` board
`board_reset` board
`board_rtc_get` board
`board_rtc_set` board
`board_rtc_synced` board
`board_uid_full` board
`board_uid_short` board
`bridge_ctl_init` bridge_ctl
`bridge_ctl_step` bridge_ctl
`C_BOLD` console
`C_DIM` console
`C_ERR` console
`C_INFO` console
`C_OK` console
`C_RST` console
`C_WARN` console
`cdc_port_baudrate` cdc_port
`cdc_port_configured` cdc_port
`cdc_port_dtr` cdc_port
`cdc_port_getc` cdc_port
`cdc_port_itf_control` cdc_port
`cdc_port_itf_deinit` cdc_port
`cdc_port_itf_init` cdc_port
`cdc_port_itf_receive` cdc_port
`cdc_port_itf_tx_complete` cdc_port
`cdc_port_line_coding_count` cdc_port
`cdc_port_on_suspend` cdc_port
`cdc_port_open` cdc_port
`cdc_port_open_count` cdc_port
`cdc_port_read` cdc_port
`cdc_port_rx_count` cdc_port
`cdc_port_rx_flush` cdc_port
`cdc_port_set_always_send` cdc_port
`cdc_port_tx_discard` cdc_port
`cdc_port_tx_empty` cdc_port
`cdc_port_tx_free` cdc_port
`cdc_port_tx_kick` cdc_port
`cdc_port_usb_stop` cdc_port
`cdc_port_write` cdc_port
`cfg_defaults` cfg_codec
`cfg_err_str` cfg_codec
`cfg_find` cfg_codec
`cfg_get` cfg_codec
`cfg_items` cfg_codec
`cfg_pack` cfg_codec
`cfg_sanitize` cfg_codec
`cfg_set` cfg_codec
`cfg_unpack` cfg_codec
`cli_exec` demos
`cmd_channels` demos
`cmd_connect` demos
`cmd_disconnect` demos
`cmd_dns` demos
`cmd_ip` demos
`cmd_mqtt` demos
`cmd_ntp` demos
`cmd_ping` demos
`cmd_scan` demos
`cmd_status` demos
`cmd_time` demos
`cmd_track` demos
`cmd_web` demos
`color_duty` color
`color_hex` color
`color_name` color
`color_names` color
`color_parse` color
`color_quality` color
`config_ap_ssid` config
`config_ecc_nmi` config
`config_erase` config
`config_hostname` config
`config_load` config
`config_mqtt_client_id` config
`config_mqtt_root` config
`config_save` config
`config_stored` config
`console_abort` console
`console_async_begin` console
`console_async_end` console
`console_color` console
`console_event` console
`console_flush` console
`console_getc` console
`console_init` console
`console_new_connection` console
`console_pad` console
`console_printf` console
`console_prompt` console
`console_puts` console
`console_readline` console
`console_rx_flush` console
`console_set_color` console
`console_set_prompt_visible` console
`console_take_dropped` console
`console_write` console
`console_write_raw` console
`dt_format` timeutil
`dt_to_epoch` timeutil
`dt_wday_fr` timeutil
`dt_weekday` timeutil
`epoch_to_dt` timeutil
`eu_dst_active` timeutil
`http_get_param` http_parse
`http_parse_request` http_parse
`http_status_text` http_parse
`http_url_decode` http_parse
`io_acc_name` io
`io_acc_whoami` io
`io_button_count` io
`io_buttons_poll` io
`io_init` io
`io_led_color_get` io
`io_led_color_set` io
`io_led_get` io
`io_led_set` io
`io_led_toggle` io
`io_pwm_get` io
`io_pwm_set` io
`io_read` io
`jb_close` util
`jb_init` util
`jb_item` util
`jb_key` util
`jb_kv_bool` util
`jb_kv_int` util
`jb_kv_raw` util
`jb_kv_str` util
`jb_open` util
`jb_raw` util
`jb_str` util
`lis3dh_init` lis3dh
`lis3dh_name` lis3dh
`lis3dh_raw_to_mg` lis3dh
`lis3dh_read_mg` lis3dh
`lis3dh_tilt` lis3dh
`lis3dsh_raw_to_mg` lis3dh
`local_offset_s` timeutil
`module_ota_run` module_ota
`mqtt_configure` ncp_mqtt
`mqtt_connect` ncp_mqtt
`mqtt_disconnect` ncp_mqtt
`mqtt_publish` ncp_mqtt
`mqtt_set_msg_cb` ncp_mqtt
`mqtt_subscribe` ncp_mqtt
`mqtt_urc` ncp_mqtt
`mqttapp_button` demos
`mqttapp_connected` demos
`mqttapp_module_restarted` demos
`mqttapp_on_message` demos
`mqttapp_poll` demos
`mqttapp_sta_down` demos
`mqttapp_sta_up` demos
`ncp_at_log_hook` ncp_at
`ncp_err_str` ncp
`ncp_fwu_abort` ncp_fwu
`ncp_fwu_active` ncp_fwu
`ncp_fwu_begin` ncp_fwu
`ncp_fwu_end` ncp_fwu
`ncp_fwu_finish` ncp_fwu
`ncp_fwu_parse_header` ncp_fwu
`ncp_fwu_sent` ncp_fwu

ncp_fwu_write ncp_fwu
ncp_poll ncp
ncp_port_boot ncp_port
ncp_port_chip_enable ncp_port
ncp_port_cs ncp_port
ncp_port_delay ncp_port
ncp_port_rdy ncp_port
ncp_port_spi_txrx ncp_port
ncp_port_tick ncp_port
ncp_raw ncp
ncp_restarted ncp
ncp_spi_init ncp_spi
ncp_spi_poll ncp_spi
ncp_spi_send ncp_spi
ncp_spi_stats ncp_spi
ncp_start ncp
net_ap_ip ncp_net
net_close ncp_net
net_dns_servers ncp_net
net_init ncp_net
net_parse_asctime ncp_net
net_ping ncp_net
net_recv ncp_net
net_resolve ncp_net
net_send ncp_net
net_server_start ncp_net
net_server_stop ncp_net
net_set_dhcp ncp_net
net_set_dns ncp_net
net_set_static ncp_net
net_sntp_config ncp_net
net_sntp_time ncp_net
net_sta_ipcfg ncp_net
net_sta_mac ncp_net
net_urc ncp_net
netapp_boot demos
netapp_connect demos
netapp_connecting demos
netapp_disconnect demos
netapp_ip demos
netapp_iso_utc demos
netapp_local_time demos
netapp_module_restarted demos
netapp_poll demos
netapp_print_time demos
netapp_request_connect demos
netapp_sta_ready demos
netapp_time_synced demos
qr_print qr_term
scan_age_s demos
scan_run demos
scan_std_label demos
uart_bridge_run uart_bridge
util_bar util
util_channel_mhz util
util_crc32 util
util_fmt_fixed util
util_fmt_ip util
util_fmt_mac util
util_fmt_uptime util
util_html_escape util
util_isqrt util
util_json_escape util
util_parse_bool util
util_parse_ip util
util_parse_long util
util_rssi_label util
util_rssi_quality util
util_strieq util
util_strlcpy util
util_tokenize util
util_utf8_len util
util_wifi_qr util
web_boot demos
web_module_restarted demos
web_poll demos
web_running demos
wifi_ap_clients ncp_wifi
wifi_ap_start ncp_wifi
wifi_ap_stop ncp_wifi
wifi_conn_details ncp_wifi
wifi_connect ncp_wifi
wifi_connect_start ncp_wifi
wifi_disconnect ncp_wifi
wifi_err_str ncp_wifi
wifi_init ncp_wifi
wifi_is_connected ncp_wifi
wifi_link_info ncp_wifi
wifi_proto_str ncp_wifi
wifi_reason_str ncp_wifi
wifi_rssi ncp_wifi
wifi_scan ncp_wifi
wifi_security_str ncp_wifi
wifi_urc ncp_wifi
ymodem_crc16 ymodem
ymodem_receive ymodem
ymodem_status_str ymodem